

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **#Gerenciamento de Configuração e Mudança**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10)**
- O gerenciamento de configuração de software (GCS) ou gerenciamento de configuração e mudança (GCM) é considerado uma disciplina à parte dentro do gerenciamento de projetos. Essa área é especialmente importante na indústria de software, porque componentes e produtos de software são desenvolvidos e modificados muitas vezes ao longo do tempo, durante e após o projeto. Assim, diferentes versões de um mesmo componente ou produto precisam estar disponíveis em diferentes momentos.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **#Gerenciamento de Configuração e Mudança**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10)**
- O gerenciamento de configuração, portanto, é a área que vai indicar como as diferentes versões dos artefatos envolvidos no desenvolvimento de software devem ser modificadas e identificadas.
- Pressman indica que o gerenciamento de configuração e mudança deve ter como objetivo responder às perguntas a seguir:
 - O que mudou e quando mudou?
 - Por que mudou?
 - Quem fez a mudança?
 - Pode-se reproduzir essa mudança?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **#Gerenciamento de Configuração e Mudança**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10)**
- Essas questões serão respondidas pelas três grandes atividades do gerenciamento de configuração e mudança:
 - O controle de versão vai dizer o que mudou e quando.
 - O controle de solicitações de mudança vai dizer porque as coisas mudaram.
 - A auditoria de configuração vai dizer quem fez a mudança e como ela pode ser reproduzida.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Gerenciamento de Configuração e Mudança
- (Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10)
- 10.1.1. Item de Configuração de Software
- Item de configuração de software (ICS) é um elemento unitário para efeito de controle de versão, ou, ainda, um agregado de elementos que são tratados como uma entidade única no sistema de gerenciamento de configuração.
- Não apenas o código de programação deve ser controlado pelo gerenciamento de configuração, mas também a documentação, os diagramas, os planos, as ferramentas, os casos de teste, os dados e quaisquer outros artefatos relacionados com o desenvolvimento do software.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **#Gerenciamento de Configuração e Mudança**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10)**
- Um dos problemas com o gerenciamento de configuração consiste em determinar a melhor granularidade para os itens de configuração. Ter itens demais poderá dificultar seu controle, e as configurações de software, formadas por um conjunto de ICS, serão listas muito extensas. De outro lado, ter itens de menos também poderá dificultar o controle, porque cada item terá um número muito grande de versões.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **#Gerenciamento de Configuração e Mudança**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10)**
- Em sistemas orientados a objetos, em geral, um item de configuração terá a granularidade de um pacote ou componente de classes afins. Mas, dependendo do projeto, esse tamanho poderá variar. Projetos muito grandes tenderão a ter itens maiores, com mais classes, e projetos muito pequenos poderão ter até classes individuais definidas como itens de configuração.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **#Gerenciamento de Configuração e Mudança**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10)**
- Cada item de configuração será definido por quatro elementos:
 - Um nome que é uma cadeia de caracteres curta que identifica claramente o item básico ou composto.
 - Uma descrição que consiste na definição do tipo de item (documento, diagrama, dados, código fonte etc.) e detalhes sobre ele.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **#Gerenciamento de Configuração e Mudança**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10)**
- • Uma lista de recursos que são outros itens de configuração exigidos pelo item básico. No caso de itens compostos, a lista de recursos poderá ser definida como a união das listas de seus itens básicos.
- • Uma realização que, no caso do item básico, é um ponteiro ou endereço para o artefato que efetivamente realiza o item. No caso de itens compostos, a realização é definida como o conjunto das realizações de seus itens básicos.
- Os itens de configuração podem ser básicos ou compostos. Os básicos são os artefatos arbitrados como individuais e indivisíveis para efeito de controle de versão. Os compostos podem ser formados por outros itens básicos e compostos.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **#Gerenciamento de Configuração e Mudança**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10)**
- 10.1.2. Relacionamentos de Itens de Configuração de Software
- Conforme visto, um ICS pode estar ligado a outros a partir de uma lista de recursos exigidos. Podem existir vários tipos de dependência ou relacionamento entre os itens de software. A maioria desses relacionamentos é prevista nas ferramentas CASE, que permitem desenhar diagramas de classe UML.
- Tais tipos de relacionamento podem tanto ser representados por associações de um tipo específico quanto por associações estereotipadas. A lista a seguir, embora não seja exaustiva, apresenta alguns exemplos de relacionamentos entre ICS que também devem ser mantidos por um sistema de controle de configuração de software:

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Gerenciamento de Configuração e Mudança
- (Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10)
- • Dependência: costuma indicar que um componente utiliza funções que são implementadas em outro componente. A associação pura e simples da UML não deixa de ser uma forma de dependência, nesse sentido, já que uma classe vai estar associada à outra normalmente para poder utilizar funções implementadas na outra.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **#Gerenciamento de Configuração e Mudança**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10)**
- • Agregação e composição: indicam que um componente é formado por outros.
- • Realização: indica que um componente concreto é a implementação de um componente mais abstrato.
- • Especialização: indica que um componente é uma variante mais específica de outro. Em geral, o componente mais especializado depende do componente mais genérico. O inverso só será verdade se o componente genérico tiver métodos abstratos que precisem ser necessariamente implementados no componente mais especializado. Nesse caso, pode-se dizer que existe dependência mútua entre os componentes.
- Dentre estas, especialmente a relação de dependência é importante para determinar quais testes de integração devem ser realizados quando ICS são atualizados.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **#Gerenciamento de Configuração e Mudança**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10)**
- 10.1.3. Rastreabilidade
- Rastreabilidade ou controle de rastros refere-se a um dos princípios da Engenharia de Software cuja implementação ainda implica significativa dificuldade. Manter um controle de rastros entre módulos de código não é difícil, porque as dependências entre os módulos costumam ser indicadas de forma sintática pelos próprios comandos da linguagem de programação (import ou uses, por exemplo).
- Contudo, manter controle de rastros entre artefatos de análise e design não é tão simples. A rastreabilidade é importante, porque, quando são feitas alterações em artefatos de análise, design ou código, é necessário manter a consistência com outros artefatos, caso contrário a documentação fica rapidamente desatualizada e inútil.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **#Gerenciamento de Configuração e Mudança**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10)**
- A técnica de rastreabilidade mais utilizada nas ferramentas CASE é a matriz de rastreabilidade, que associa elementos entre si, por exemplo, casos de uso e seus respectivos diagramas de sequência ou classes e módulos de programa. Porém, esse tipo de visualização matricial torna-se impraticável à medida que a quantidade de elementos cresce, especialmente se o controle das relações entre os elementos precisar ser feito manualmente (CLELAND-HUANG, ZEMONT & LUKASIK, 2004).

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de versões dos itens do projeto
- (Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10 -- tópico 10.2)
- Repositório
- Todos os arquivos referentes às realizações dos itens de configuração ficam em um local chamado *repositório*, sob a guarda do sistema de controle de versão. O repositório pode ser entendido como um local (banco de dados) em que as diferentes versões de cada item são automaticamente mantidas e identificadas.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de versões dos itens do projeto
- (Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10 -- tópico 10.2)
- **Repositório**
- Um bom sistema de controle de versões deverá ser capaz de otimizar o espaço de armazenamento do repositório. Em geral, tal sistema deverá ser capaz de armazenar apenas o artefato original e, dali para a frente, armazenar apenas as modificações que forem feitas, e não novas cópias completas do mesmo artefato. Dessa forma, uma versão qualquer será produzida pelo sistema a partir da versão original, na qual o sistema aplicará sequencialmente todas as modificações até chegar à versão desejada.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de versões dos itens do projeto
- (Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10 -- tópico 10.2)
- Repositório
- Para otimizar a velocidade de acesso, uma vez que normalmente a última versão é a que será efetivamente utilizada na maioria das vezes, o sistema pode armazenar em *cache* uma cópia completa dessa última versão, sem que haja necessidade de gerá-la a partir da original e das modificações, como as anteriores. Quando essa versão deixar de ser a atual, a cópia pode ser apagada, ficando no repositório apenas as modificações que permitem gerá-la a partir das versões anteriores e a cópia da nova versão atual.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **#Controle de versões dos itens do projeto**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10 -- tópico 10.2)**
- **Políticas de Compartilhamento de Itens**
- Os desenvolvedores não trabalham diretamente sobre os itens do repositório, mas sobre cópias desses itens. Assim, um desenvolvedor deve solicitar acesso a determinado item no repositório, obtendo uma cópia dele. Esse desenvolvedor vai fazer as alterações nesta cópia do item e, posteriormente, salvar no repositório uma nova versão dele.
- Deve-se decidir o que fazer caso mais de um desenvolvedor esteja trabalhando sobre o mesmo item (concorrência). Nesse caso, existem duas políticas usuais:

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de versões dos itens do projeto
- (Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10 -- tópico 10.2)
- • Política *trava-modifica-destrava* (*exclusive lock*), na qual um desenvolvedor que copia um item para modificá-lo deve travar o item no repositório de forma que nenhum outro desenvolvedor poderá alterá-lo até que a modificação seja concluída e a nova versão seja salva. Essa política tem como vantagem o fato de evitar colisões, isto é, situações em que dois desenvolvedores alteram um item e as alterações do primeiro a salvar acabam sendo perdidas porque o segundo vai salvar suas próprias modificações sobre uma versão anterior às modificações do primeiro. Mas a desvantagem dessa política reside no fato de que, muitas vezes, os desenvolvedores até poderiam trabalhar simultaneamente sobre um mesmo item, pois vão alterar partes diferentes dele, e mesmo assim terão de esperar, perdendo tempo ou realizando outras tarefas menos importantes. Esse problema fica mais crítico à medida que os ICS ficam maiores.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de versões dos itens do projeto
- (Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10 -- tópico 10.2)
- Política *copia-modifica-resolve* (*optimistic merges*), na qual dois desenvolvedores ou mais podem trabalhar simultaneamente sobre um mesmo ICS e, no momento de salvar a nova versão, resolver entre si possíveis interferências. Na prática, os conflitos são raros e causados por falta de comunicação entre os desenvolvedores, e a mesclagem das versões pode ser feita automaticamente pelo sistema de controle de versões.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **#Controle de versões dos itens do projeto**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10 -- tópico 10.2)**
- No caso da política copia-modifica-resolve, na maioria das vezes as alterações de um item vão ocorrer em locais diferentes e o próprio sistema poderá resolvê-las. Nas poucas vezes em que as alterações ocorrerem na mesma parte do artefato, os desenvolvedores deverão decidir como tratá-las. Esse tipo de conflito, porém, normalmente ocorre somente quando há uma divisão de trabalho inadequada.
- Quando um sistema de controle de versões é usado, normalmente o tratamento de conflitos na política copia-modifica-resolve ocorre da seguinte forma:

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de versões dos itens do projeto
- (Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10 -- tópico 10.2)
- • O desenvolvedor *A* pega uma cópia do item *X* para modificar;
- • O desenvolvedor *B* pega outra cópia do mesmo item *X* para modificar;
- • O desenvolvedor *A* salva (*commit*) uma nova versão de *X* com suas alterações (*xA*);
- • Quando o desenvolvedor *B* vai salvar sua cópia, o sistema avisa que a versão do item está desatualizada (porque já existe a versão *xA*) e que o desenvolvedor *B* deve fazer uma mesclagem (*merge*) de sua cópia com a versão existente... *xB*
- Um bom sistema vai mostrar exatamente quais linhas mudaram da versão *xA* para *xB* e também se o *merge* das duas versões implica alterar as mesmas linhas.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **#Controle de versões dos itens do projeto**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10 -- tópico 10.2)**
- Dessa forma, o desenvolvedor *B* deverá avaliar as alterações do desenvolvedor *A* e verificar se existe conflito entre estas e as que ele próprio produziu. Se necessário, os dois desenvolvedores devem conversar para resolver possíveis conflitos. Ao final do processo, o desenvolvedor *B* salvará a versão 1.3 do item.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de versões dos itens do projeto
- (Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10 -- tópico 10.2)
- Envio de Versões
- Normalmente, o *envio de versões (commit)* é feito a critério do desenvolvedor. Porém, é importante que uma nova versão de qualquer artefato só seja enviada ao repositório se estiver minimamente estável, isto é, razoavelmente livre de defeitos e revisada em relação a padrões de escrita.
- No caso de artefatos de código, isso pode ser garantido com a realização de testes de unidade, os quais devem inclusive acompanhar o código como parte do item de configuração. Estes mesmos testes de unidade poderão ser usados para compor os testes de integração quando aplicados a um *build* do sistema em vez de a um item isolado.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **#Controle de versões dos itens do projeto**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10 -- tópico 10.2)**
- Entre os mais comuns encontram-se:
- as soluções livres: CVS, Mercurial, Git e SVN (Subversion);
- e as comerciais: SourceSafe, TFS, PVCS (Serena) e ClearCase.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **#Controle de versões dos itens do projeto**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10 -- tópico 10.2)**
- O desenvolvimento de software livre utiliza mais o Git (com repositórios no [GitHub](#)), que vem substituindo o SVN, que por sua vez é um sucessor do CVS.
- Muitas empresas também adotam o Git (como a Microsoft com o código fonte do Windows) ou o SVN, embora algumas empresas prefiram uma solução comercial, optando pelo ClearCase (da [IBM](#)) ou Team Foundation Server (da [Microsoft](#)).

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **#Controle de versões dos itens do projeto**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10 -- tópico 10.2)**
- Optar por uma solução comercial geralmente está relacionada à garantia, pois as soluções livres não se responsabilizam por erros no software e perdas de informações, porém as soluções livres podem ter melhor desempenho e segurança que as comerciais.
- As soluções comerciais apesar de supostas garantias adicionais, não garantem o sucesso da implementação nem indenizam por qualquer tipo de erro mesmo que comprovadamente advindo do software.
- A eficácia do controle de versão de software é comprovada por fazer parte das exigências para melhorias do processo de desenvolvimento de certificações tais como CMMI, MPS-BR e SPICE.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- (Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10 -- tópico 10.3 + 10.4 + 10.5)
- *O controle de mudança ou gerência de solicitações de mudança é uma parte importante da gerência de configuração que permite saber por que determinada versão de um ICS foi sucedida por outra. Um bom sistema de controle de versões consegue identificar quais linhas foram alteradas em um artefato. Mas ele não saberá indicar *por que* essas linhas foram alteradas. O motivo da alteração de um artefato da versão 1.2 para a 1.3 poderia, ser, por exemplo, a correção de um defeito no código. Então, cabe ao controle de mudança manter juntamente com o controle de versão o histórico dos motivos que levaram a cada alteração nos artefatos.*

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- (Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10 -- tópico 10.3 + 10.4 + 10.5)
- Um típico controle de mudança de um sistema de software vai indicar quais funcionalidades foram adicionadas, removidas ou modificadas, quais defeitos foram corrigidos e, eventualmente, quais pendências ainda restam para uma versão futura. Por exemplo, um arquivo de controle de mudança de um produto de software poderia conter as informações a seguir:

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- (Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10 -- tópico 10.3 + 10.4 + 10.5)
- • Mudanças da versão 2.2 para a versão 2.3
 - • Correção do defeito D345.
 - • Correção do defeito D346.
 - • Adicionada a funcionalidade do requisito R43.
 - • Aprimorada a usabilidade da interface I12.
- • Pendências para uma versão posterior
 - • Defeito D347.
 - • Melhorar a usabilidade da interface I13.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- (Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10 -- tópico 10.3 + 10.4 + 10.5)
- As modificações de uma versão para outra podem ser tanto aquelas que já estavam no plano de desenvolvimento do software, como a adição de funcionalidades referentes aos requisitos ou casos de uso próprios das iterações, ou a inclusão de mudanças solicitadas pelo usuário, quando este não fica totalmente satisfeito com as funcionalidades implementadas durante os testes de aceitação, ou ainda a inclusão de mudanças referentes a características que não foram incorporadas em uma iteração por falta de tempo e foram retiradas do escopo da iteração. Na fase de produção (evolução e manutenção do software), o gerenciamento de mudança fará o controle das atividades de manutenção corretiva, adaptativa e perfectiva.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- (Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10 -- tópico 10.3 + 10.4 + 10.5)
- Além desse tipo de controle, é possível haver um controle automatizado das mudanças, pois o sistema gerenciador de versões é capaz de comparar duas versões de um artefato e indicar exatamente quais elementos foram alterados e por quem. Dessa forma, quando uma nova versão de um sistema é gerada a partir de um conjunto de itens de configuração, é possível também gerar automaticamente um descritivo das mudanças feitas.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- (Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10 -- tópico 10.3 + 10.4 + 10.5)
- Porém, essa descrição será unicamente sintática, costumando ser necessário adicionar as informações que indiquem a motivação da mudança. Por exemplo, não basta informar que a linha X teve seu código alterado para tal sequência de caracteres; é necessário informar *por que* isso foi feito. Se a ferramenta de gerenciamento de mudança for integrada à ferramenta de distribuição de tarefas entre os desenvolvedores, então, mesmo essas descrições de motivação poderão ser obtidas de forma praticamente automática, pois já estarão na descrição da tarefa a ser feita.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- (Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10 -- tópico 10.3 + 10.4 + 10.5)
- **10.4. Auditoria de Configuração**
- A *auditoria de configuração* é uma atividade que tem como objetivo verificar se os itens de configuração presentes em uma versão ou *baseline* são realmente aqueles que deveriam estar ali. Em relação à *baseline* ou versão, a auditoria deve ainda verificar se todos os itens necessários estão realmente presentes no repositório.
- A auditoria também pode ter como finalidade verificar a consistência da documentação fornecida ao usuário com a configuração de sistema entregue.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- (Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10 -- tópico 10.3 + 10.4 + 10.5)
- Uma auditoria de configuração pode ser executada com os passos a seguir:
 - Preparar um relatório que liste cada item a ser verificado na *baseline* e o procedimento de teste a ser efetuado.
 - Efetuar os testes e anotar os itens que passaram no teste.
 - Se houver algum item que não passou no teste, anotar o fato no documento de *descobertas da auditoria* para encaminhar ao setor responsável para providências.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- (Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10 -- tópico 10.3 + 10.4 + 10.5)
- Por vezes, poderá ser necessário realizar auditorias de versões antigas de um produto. Por exemplo, um determinado usuário de uma versão antiga do produto pode alegar ter enfrentado problemas e demanda indenização. Assim, se a organização desenvolvedora for capaz de gerar essa versão anterior para que seja analisada pela auditoria, a real causa do problema poderá ser identificada ou afastada.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- (Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10 -- tópico 10.3 + 10.4 + 10.5)
- **10.5. Ferramenta para Controle de Versão**
- O controle de versões em projetos de software já não é mais feito manualmente, com a utilização de diretórios e padrões de nomeação de arquivos, porque esse tipo de controle não é efetivo quando se deseja obter relatórios de versões ou quando se realiza o desenvolvimento de variantes. Além disso, essa forma de controle consome espaço de armazenamento, pois cada versão é armazenada integralmente no diretório.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- (Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10 -- tópico 10.3 + 10.4 + 10.5)
- Assim, o mais adequado para uma empresa de desenvolvimento de software seria o uso de um sistema automatizado de controle de versões. Em 2018, a ferramenta mais largamente usada para este fim é Git, um projeto de código aberto originalmente desenvolvido por Linus Torvalds a partir de 2005.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- (Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10 -- tópico 10.3 + 10.4 + 10.5)
- Git é considerado uma ferramenta de controle de versões distribuída, já que, ao contrário de outras, que têm apenas um repositório para os itens, ela faz com que cada desenvolvedor que esteja trabalhando em um repositório tenha uma cópia completa deste.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

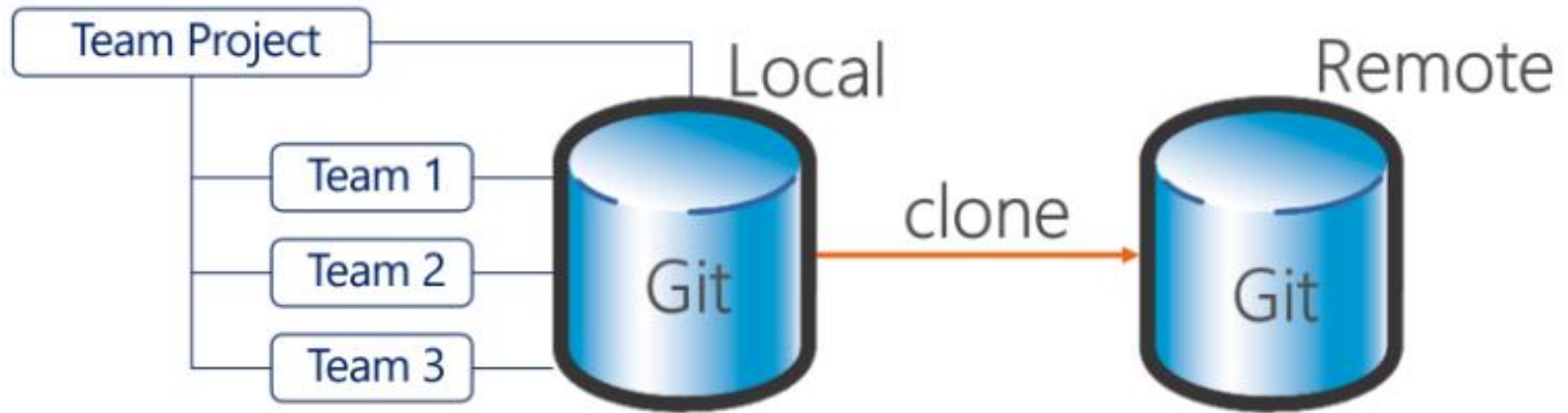
- #Controle de Mudança
- (Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10 -- tópico 10.3 + 10.4 + 10.5)
- A ferramenta permite que se trabalhe simultaneamente em várias ramificações (*branch = ramo*) de um projeto. Por exemplo, um analista poderá estar trabalhando na implementação da versão 2.0 de um item, a qual será futuramente incluída em um *build*, e poderá momentaneamente retomar a versão 1.3 do mesmo item, já incluída em uma *release*, para corrigir um defeito, gerando assim a versão 1.3.1 deste item, e depois voltar a trabalhar na versão 2.0.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- Fonte: <https://github.com>
- O *Git* é um sistema de controle de versão distribuído com ênfase em velocidade. O Git foi inicialmente projetado e desenvolvido por Linus Torvalds para o desenvolvimento do kernel Linux, mas foi adotado por muitos outros projetos.
- Cada diretório de trabalho do Git é um repositório com um histórico completo e habilidade total de acompanhamento das revisões, não dependente de acesso a uma rede ou a um servidor central.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- Fonte: <https://github.com>

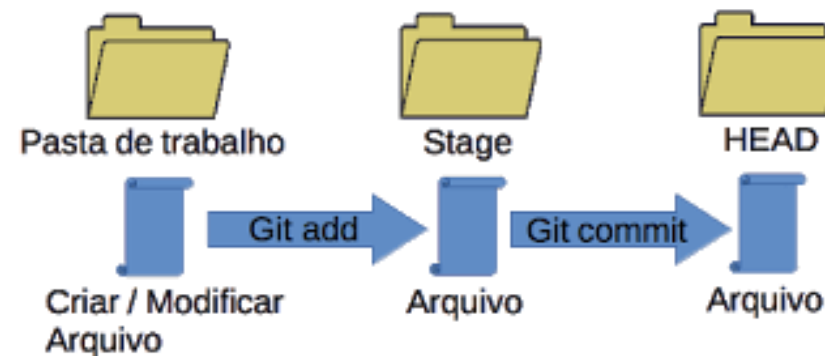


GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- Fonte: <https://github.com>
- Versionar um código fonte (arquivos) -- Criar um novo repositório git
- Para versionar arquivos pelo Git a primeira coisa que é preciso fazer é criar um novo repositório – após criar um diretório em sua máquina e estando dentro do mesmo, execute o comando: *git init*
- Após criá-lo, você poderá verificar que para controle de todo versionamento o git cria um diretório oculto .git

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- Fonte: <https://github.com>
- Funcionamento do Git
- O trabalho de versionamento do Git dentro de um repositório é simples, basicamente todo repositório Git trabalha com uma estrutura de árvores em três níveis:
- *Work Directory*: contém os arquivos vigentes;
- *Stage* ou *index*: funciona como uma área temporária;
- *Head*: aponta para o último *commit* feito;



GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- Fonte: <https://github.com>
- Trabalhando com repositórios remotos
- Existem diversas ferramentas de mercado que nos oferecem um repositório Git remoto, ferramentas como Azure DevOps, Git Hub ou Git Lab. Caso tenha algum repositório remoto em ferramentas como estas, você pode criar uma cópia local executando o comando:
- *git clone <caminhodorepositorio>*
- ou fornecendo já no comando os dados de acesso:
- *git clone usuario@servidor:<caminhadorepositório>*

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- Fonte: <https://github.com>
- Adicionando arquivos e confirmando as alterações
- Durante o versionamento dos arquivos o primeiro passo é adicionar novos arquivos ao nosso *Index (stage)* utilizando para isso o comando:
- `git add <arquivos>`

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- Fonte: <https://github.com>
- Adicionando arquivos e confirmando as alterações
- Você pode criar um arquivo chamado *Codigo.cs* e para adicionar um versionamento utilize o comando *git add **, onde o caractere *** representa todos os arquivos novos deste diretório. Com isso executamos o primeiro passo básico de um fluxo de trabalho com Git. Agora para efetivarmos esta mudança, precisamos confirmar as mesmas incluindo ela enviando o arquivo para o *HEAD* do repositório local através do comando:
- *git commit -m "comentário referente as mudancas"*
- Esta sequência de comandos pode ser executada diversas vezes, com isso teremos nossos arquivos sempre versionados localmente, isso mesmo, localmente, restando ao final de um período ou etapa enviar as alterações aplicadas no *HEAD* para o servidor remoto.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- Fonte: <https://github.com>
- Enviando alterações para o servidor remoto
- Para enviarmos estas alterações para um servidor remoto utilizamos o comando:
- *git push origin <nomedabran>*
- Um ponto a se observar é que o comando *git push* só deve ser utilizado se anteriormente tivéssemos clonado o repositório remoto através do comando *git clone*. Caso isso não tenha sido feito devemos adicionar as alterações através do comando:
- *git remote add origin <enderecoservidor>*

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- Fonte: <https://github.com>
- Trabalhando com Branches (ramos)
- Utilizamos *branches* (“ramos”) para trabalharmos em funcionalidades de forma isolada. Sempre temos uma *branch* principal, a qual o Git coloca o nome de master, ela é criada automaticamente quando criamos o repositório. Depois disso podemos criar diversas *branches* isolando nosso trabalho para que no final possamos mesclar (*merge*) a sua *branch* principal.
- Para criarmos um nova *branch* em nosso repositório local execute o comando:
- `git checkout -b <nomedabranh>`
-

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- Fonte: <https://github.com>
- Trabalhando com Branches (ramos)
- No mesmo momento que criamos uma nova *branch* com o comando *git checkout*, automaticamente passamos a trabalhar nesta nova *branch*. Caso queira retornar a uma outra *branch* criada basta executar o comando:
- *git checkout <nomedabranh>*
- Para excluirmos uma *branch* criada, execute o comando:
- *git branch -d <nomedabranh>*

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- Fonte: <https://github.com>
- Trabalhando com Branches (ramos)
- Lembrando que todas estas alterações estão sendo feitas em seu repositório local, ou seja, a *branch* criada não está disponível para os outros membros do time, para que isso aconteça é necessário enviarmos a *branch* para o repositório remoto através do comando:
- `git push origin <nomedabranh>`

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- Fonte: <https://github.com>
- Atualizando e mesclando branches
- Para atualizar seu repositório local com possíveis mudanças feitas por outros membros da equipe na *branch* remota executamos o comando:
- *git pull origin <nomebranchremota>*
- Com a *branch* local atualizado podemos mesclar alterações entre as *branches*, claro isso se elas possuírem diferenças. Para listarmos estas diferenças executamos o comando:
- *git diff <branchdeorigem> <branchdedestino>*

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- Fonte: <https://github.com>
- Atualizando e mesclando branches
- Considerando que existam diferenças (listadas acima) e claro caso você queira mesclar as duas, vamos acessar a *branch* de destino através do comando *git checkout* e na sequência executamos o comando:
- *git merge <nomebranchorigem>*

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- Fonte: <https://github.com>
- Atualizando e mesclando branches
- Vale lembrar que as alterações são feitas localmente, ou seja, para que o repositório remoto seja atualizado precisamos executar o comando *git push*. Outro ponto importante é que caso o processo de *merge* gere um conflito, a ferramenta solicitará que você resolva manualmente estes conflitos, e após este procedimento você precisará incluir as mudanças geradas pelo conflito através do comando *git add*.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- Fonte: <https://github.com>
- Criando rótulos durante o versionamento
- Durante o processo de produção de um software é importante organizarmos nossas alterações, principalmente em momentos de releases, para isso podemos incluir em nosso repositório Tags, uma espécie de rótulos que facilita na identificação das releases. Para criarmos uma tag execute o comando:
- `git tag <nomedatag> <numerodocommit10digitos>`

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- Fonte: <https://github.com>
- Criando rótulos durante o versionamento
- Importante dizer que o número do *commit* que receberá a tag é representado pelos primeiros dígitos do *commit* (não precisa ser o ID inteiro, você pode incluir somente o início, desde que ele seja único) e caso você precise consultar este ID podemos usar o comando:
- *git log*

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- Fonte: <https://github.com>
- Revertendo alterações em seu repositório local
- Como errar é humano, é importante saber que, caso seja necessário podemos reverter algum arquivo que alteramos localmente pela última versão no servidor remoto, para isso execute o comando:
- *git checkout — <nomedoarquivo>*

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança
- Fonte: <https://github.com>
- Revertendo alterações em seu repositório local
- Caso o problema tenha sido um pouco maior e você queira remover todas as alterações confirmadas em seu repositório local na *branch* principal (master) execute os comandos:
- *git fetch origin*
- *git reset --hard origin/master*

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Controle de Mudança (Rastreabilidade)
- (Fonte: RAUL, Wazlawick - Engenharia de Software - Conceitos e Práticas -- cap. 10 -- tópico 10.2)

Elemento	Rastreabilidade
Casos de Teste	São rastreados pelo seu nome. O nome do Caso de Teste possui o mesmo identificados do Caso de Uso que testa. Exemplo: CSU 0012 é testado pelo CST 0012.
Código-fonte	Cada código possuía a sua classe de mesmo nome desenhada no EA. Exemplo: Para a Classe "usuarioVisitante" existe o código "usuarioVisitante".
Tabela	Análogo ao código fonte, o nome da tabela refletia a classe que ele está vinculado. Para a classe usuário, existe, por exemplo, uma tabela usuário.
Planilha de Gerenciamento de Requisitos e Mudanças, Plano do Projeto, Plano da Iteração, Cronograma e Lista de Riscos	A rastreabilidade é definida na imagem de rastreabilidade de todo o projeto (acima). Alterando um Caso de Uso existe uma análise de impacto na Planilha de Gerenciamento de Requisitos e Mudanças, Plano do Projeto e elementos anexos: Plano da Iteração, Cronograma e Lista de Riscos.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **#Planejamento Estratégico = Diferentes versões de projeto**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software)**
- Alguns projetos precisam de variações específicas, conforme as necessidades específicas de cada cliente, ou criação de um ramo para experimentações no projeto, sem comprometer a linha principal de desenvolvimento.
- O sistema de controle de versão oferece funcionalidades que facilitam a coordenação de ramos diferentes de desenvolvimento em um mesmo projeto. As alterações feitas em um ramo muitas vezes precisam ser mescladas em outro ramo.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **# Planejamento Estratégico = Diferentes versões de projeto**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software)**
- Essa operação é bastante delicada e é facilitada em muito com o sistema de controle de versão, que permite bastante controle e automação no processo.
- Mesmo em caso de uma fusão equivocada, o sistema de controle de versão permite voltar ao estado anterior, o que traz bastante segurança aos desenvolvedores.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **# Planejamento Estratégico = Diferentes versões de projeto**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software)**
- Roger Pressman, em seu livro Software Engineering: A Practitioner's Approach, afirma que a gerência de configuração de software (GCS) é o: conjunto de atividades projetadas para controlar as mudanças pela identificação dos produtos do trabalho que serão alterados, estabelecendo um relacionamento entre eles, definindo o mecanismo para o gerenciamento de diferentes versões destes produtos, controlando as mudanças impostas, e auditando e relatando as mudanças realizadas.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- # Planejamento Estratégico = Sincronização de Mudanças Concorrentes
- (Fonte: RAUL, Wazlawick - Engenharia de Software)
- Como os desenvolvedores trabalham em paralelo e concorrentemente nos mesmos arquivos do projeto, é necessária uma política para ordenar e integrar todas essas alterações, de modo a evitar que um desenvolvedor sobrescreva as alterações de outro desenvolvedor.
- O controle de versão além de rastrear e controlar alterações, também coordena a edição colaborativa e o compartilhamento de dados entre os vários desenvolvedores de uma equipe

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- # Planejamento Estratégico = Auditoria de configuração
- (Fonte: RAUL, Wazlawick - Engenharia de Software)
- Executar Auditoria de Configuração Física (componentes)
- Uma Auditoria de Configuração Física (PCA) identifica os componentes de um produto que serão implantados do Repositório do Projeto. Os passos são:

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- # Planejamento Estratégico = Auditoria de configuração
- (Fonte: RAUL, Wazlawick - Engenharia de Software)
- Identificar a baseline a ser implantada (geralmente é apenas um nome e/ou número, mas também pode ser uma lista completa de todos os componentes e suas respectivas versões).
- Confirmar que todos os artefatos necessários, conforme especificado pelo Caso de Desenvolvimento, estão presentes na baseline.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- # Planejamento Estratégico = Auditoria de configuração
- (Fonte: RAUL, Wazlawick - Engenharia de Software)
- Outros Níveis da Auditoria de Configuração Física
- Algumas organizações usam uma Auditoria de Configuração Física para confirmar a consistência do design e/ou documentação do usuário com o código. O (RUP) Rational Unified Process recomenda que a verificação dessa consistência seja executada como parte da atividade de revisão no decorrer do processo de desenvolvimento.
- No último estágio, as auditorias devem se limitar a verificar os produtos liberados necessários que estão presentes, sem se preocupar em revisar o conteúdo.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- # Planejamento Estratégico = Auditoria de configuração
- (Fonte: RAUL, Wazlawick - Engenharia de Software)
- Executar Auditoria de Configuração Funcional
- Uma Auditoria de Configuração Funcional (FCA) confirma que uma baseline atende aos requisitos estabelecidos para ela. Os passos para a execução dessa auditoria são:
- Preparar um relatório que liste cada requisito estabelecido para a baseline, seu procedimento de teste correspondente e o resultado de teste (aprovado/reprovado) da baseline.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- # Planejamento Estratégico = Auditoria de configuração
- (Fonte: RAUL, Wazlawick - Engenharia de Software)
- Confirmar que cada requisito passou por um ou mais testes e que o resultado de todos esses testes foi 'aprovado'.
- Em Descobertas da Auditoria de Configuração, liste quaisquer requisitos que não tenham passado por procedimentos de teste e os requisitos que estão com teste incompleto ou que foram reprovados.
- Gerar uma lista das CRs estabelecidas para essa baseline. Confirme que cada CR foi fechada. Em Descobertas da Auditoria de Configuração, liste quaisquer CRs que não estão fechadas.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **# Planejamento Estratégico = Auditoria de configuração**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software)**
- Reportar Descobertas
- Se houver alguma discrepância, ela será capturada em Descobertas da Auditoria de Configuração conforme descrito anteriormente. Além disso, os seguintes passos deverão ser executados:
- Identificar ações corretivas. Talvez, isso requeira a entrevista de vários membros da equipe do projeto para que a origem da discrepância e as ações apropriadas sejam identificadas.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **# Planejamento Estratégico = Auditoria de configuração**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software)**
- Para artefatos ausentes, a ação apropriada é geralmente controlar a configuração do artefato ou criar uma CR ou tarefa que produzirá o artefato ausente.
- No caso de requisitos não testados ou reprovados no teste, o requisito pode ser estabelecido para uma baseline posterior ou negociado para ser removido do conjunto de requisitos.
- Para CRs em aberto, a CR pode ser simplesmente fechada, testada ou adiada para uma baseline posterior.
- Para cada ação corretiva, atribua uma responsabilidade e determine uma data de conclusão

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **# Planejamento Estratégico = Plano de Gerência de Riscos**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software)**
- Todo projeto de software deve ter um *plano de gerência de riscos*. Esse plano inclui os vários elementos apresentados neste capítulo. Em resumo, ele deve mostrar:
- Quais são os riscos identificados
- Uma análise qualitativa ou quantitativa de cada risco que indique, por exemplo, a probabilidade de sua ocorrência e de seu impacto sobre o projeto, caso ocorra

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **# Planejamento Estratégico = Plano de Gerência de Riscos**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software)**
- Como a probabilidade de o risco ocorrer pode ser reduzida (plano de redução de probabilidade)
- Como o impacto do risco, caso ocorra, pode ser reduzido (plano de redução de impacto)
- O que fazer se o que era um risco se tornar efetivamente um problema (plano de resposta ao risco)
- Como monitorar os riscos

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- # Planejamento Estratégico = Plano de Gerência de Riscos
- (Fonte: RAUL, Wazlawick - Engenharia de Software)
- Os planos de redução de probabilidade e impacto também são chamados *planos de mitigação de risco*, ou seja, são planos executados de forma preventiva para evitar que o risco ocorra e, se ainda assim ele ocorrer, seu prejuízo seja reduzido.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **# Planejamento Estratégico = Plano de Gerência de Riscos**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software)**
- Uma característica altamente desejada para um planejador, e mesmo para um gerente de projeto, em relação ao risco é a capacidade de visão antecipada. Um bom planejador e um bom gerente são capazes de visualizar com antecedência possíveis situações anômalas que poderiam impedir o bom andamento do projeto. Essa previsão, longe de ser uma atitude pessimista, poderá salvar o projeto no futuro ou pelo menos mantê-lo dentro do cronograma e do orçamento previstos. Além disso, nem todos os riscos precisam preocupar o planejador de projeto. Como será visto mais adiante, apenas os riscos de maior exposição merecerão mais atenção.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **# Planejamento Estratégico = Plano de Gerência de Riscos**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software)**
- Outro princípio importante é a antecipação das atividades de maior risco, como preconizado nos modelos Espiral, RUP e métodos ágeis. A ideia é: se um risco pode inviabilizar um projeto, é melhor que isso seja analisado e descoberto o quanto antes, porque quanto mais tempo passar, maior terá sido o investimento e, por conseguinte, o custo.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- # Planejamento Estratégico = Plano de Gerência de Riscos
- (Fonte: RAUL, Wazlawick - Engenharia de Software)
- Os riscos podem ser classificados em três grupos em relação ao conhecimento que se tem deles:
 - *Conhecidos*: são aqueles já identificados e para os quais a equipe possivelmente estará preparada.
 - *Desconhecidos*: são aqueles que poderiam ter sido descobertos se as medidas de identificação adequadas tivessem sido tomadas.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **# Planejamento Estratégico = Plano de Gerência de Riscos**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software)**
- • *Impossíveis de prever*: são aqueles que não teriam sido descobertos nem mesmo com as melhores técnicas de identificação.
- Assim, o plano de gerência de riscos vai poder tratar apenas o primeiro grupo de riscos. Para minimizar o segundo grupo, boas atividades de identificação de riscos devem ser desenvolvidas. Já o terceiro grupo vai depender da capacidade do gerente de responder de forma organizada a situações totalmente imprevistas.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- # Planejamento Estratégico = Identificação de Riscos
- (Fonte: RAUL, Wazlawick - Engenharia de Software)
- Normalmente, um risco compõe-se de três elementos:
 - Uma *causa*, na forma de uma condição incerta ou desconhecida, mas que pode ocorrer com uma determinada probabilidade. Por exemplo, o uso de uma tecnologia nova que não é dominada pela equipe de desenvolvimento.
 - Um *problema*, que pode ocorrer em função da causa. Por exemplo, a equipe ter dificuldades sérias para implementar os requisitos usando essa tecnologia.
 - Um *efeito* causado pelo problema em um ou mais objetivos do projeto ou iteração. Por exemplo, um atraso no cronograma ou um produto desenvolvido com qualidade inferior à desejada.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **# Planejamento Estratégico = Identificação de Riscos**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software)**
- Assim, um risco é um problema que tem uma causa e pode ocasionar um efeito. Se esse problema ocorrer, estima-se que um dos objetivos do projeto será impactado, seja no tempo, custo, qualidade, seja em outro aspecto. Por exemplo: “Como consequência do uso de um novo hardware (uma exigência definida), erros inesperados de integração do sistema podem ocorrer (um risco incerto), o que levaria ao estouro dos custos do projeto (um efeito sobre o objetivo do orçamento)”.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **# Planejamento Estratégico = Identificação de Riscos**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software)**
- Mais adiante, mostramos que, como a causa do risco está ligada à sua probabilidade de ocorrer, os planos de redução de probabilidade deverão agir na causa. Já o efeito do risco define o seu impacto e assim os planos de redução de impacto deverão agir nos efeitos. Estes dois são os planos de mitigação de risco. Finalmente, o componente “problema” do risco está relacionado com a situação resultante e, assim, os planos de contingência estarão ligados à tentativa de resolver ou absorver o problema, caso ele efetivamente ocorra.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- # Planejamento Estratégico = Identificação de Riscos
- (Fonte: RAUL, Wazlawick - Engenharia de Software)

- Relações entre os componentes do risco e seus planos

Componente do risco	Propriedade do risco	Plano relacionado
Causa	Probabilidade	Mitigação: redução de probabilidade
Efeito	Impacto	Mitigação: redução de impacto
Problema		Plano de contingência

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **# Planejamento Estratégico = Identificação de Riscos**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software)**
- O PMBOK (PMI, 2017), referência em gerenciamento de projetos, define que o risco é uma condição incerta que pode ter tanto um efeito positivo quanto um efeito negativo sobre o projeto. Assim, existiriam também os *riscos positivos* ou *oportunidades*. Mas, certamente, os riscos mais importantes a serem identificados são aqueles que podem prejudicar o projeto.
- Assim como os requisitos de um projeto, os riscos devem ser identificados e priorizados para serem abordados adequadamente.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **# Planejamento Estratégico = Identificação de Riscos**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software)**
- Assim como os requisitos de um projeto, os riscos devem ser identificados e priorizados para serem abordados adequadamente.
- O ideal é que o planejador tenha a sua disposição um catálogo com riscos que ocorreram no passado em projetos semelhantes. É importante que esse histórico nunca se perca, mas, caso não exista tal registro, uma identificação de riscos pode ser feita em uma reunião com a equipe e discussão sobre possíveis incertezas relacionadas com os tópicos mencionados. Sugestões para essa discussão são apresentadas nas subseções a seguir.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- # Planejamento Estratégico = Identificação de Riscos
- (Fonte: RAUL, Wazlawick - Engenharia de Software)
- O Processo Unificado associa diferentes tipos de risco com as diferentes fases do projeto:
 - • *Concepção*: riscos de requisitos e de negócio.
 - • *Elaboração*: riscos de tecnologia e arquitetura de sistema.
 - • *Construção*: riscos de programação e teste de sistema.
 - • *Transição*: riscos de utilização do sistema no ambiente final.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **# Planejamento Estratégico = Identificação de Riscos**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software)**
- A literatura apresenta várias técnicas para identificação de riscos na fase de planejamento de um projeto. Podem-se citar as seguintes:
 - Uso de *checklists* predefinidos com possíveis riscos: tais listas podem ser obtidas tanto na literatura ou na Internet quanto a partir de projetos anteriores executados pela mesma equipe. Uma excelente base para iniciar uma lista desse tipo é o relatório SEI de taxonomia de riscos, apresentado na Seção 8.3.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- # Planejamento Estratégico = Identificação de Riscos
- (Fonte: RAUL, Wazlawick - Engenharia de Software)
- • Reuniões e *brainstormings* com gerente e equipe de projeto com experiência em outros projetos.
- • Análise de cenários e lições aprendidas em projetos anteriores com contexto semelhante.
- A identificação de riscos deve considerar diferentes fontes, tais como tecnologia, pessoas e o próprio projeto. As subseções a seguir discutem mais detalhadamente estas possíveis fontes.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **# Riscos Potenciais = Riscos Tecnológicos**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software)**
- *Os riscos tecnológicos* estão relacionados com todas as incertezas referentes ao modo como a equipe será capaz de lidar com a tecnologia necessária para realizar o projeto. Quanto menos experiência a equipe tiver com essas tecnologias, maiores serão os riscos.
- Projetos que envolvam diferentes sistemas de software e hardware enfrentarão problemas de compatibilidade frequentes. Tornar tais sistemas compatíveis demandará tempo e custo extras ao projeto. Assim, o planejador deve saber se diferentes tecnologias terão de interagir e qual é a experiência da equipe com esse tipo de integração.
- Outro ponto que pode oferecer risco tecnológico a um projeto é a questão da obsolescência. Quão rápido as tecnologias usadas ou produzidas serão suplantadas por outras mais eficientes?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **# Riscos Potenciais = Riscos Relacionados com Pessoas**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software)**
- Há vários tipos de interessados em um projeto. Cada um dos papéis de interessado pode produzir um risco característico, como os descritos a seguir:
 - *Riscos de pessoal*: projetos são executados por pessoas. Então, perder uma pessoa da equipe de forma permanente ou temporária pode ter um grande impacto na medida em que essa pessoa for insubstituível.
 - *Riscos de cliente*: até que ponto o cliente se manterá interessado no projeto? Mudanças políticas ou administrativas poderão afetar o interesse da organização em investir no projeto? Mesmo que a organização mantenha o interesse no projeto, o cliente estará disponível para esclarecer requisitos e realizar testes?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **# Riscos Potenciais = Riscos Relacionados com Pessoas**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software)**
- *Riscos de negócio*: muitas vezes, a empresa até constrói um bom produto no prazo e com o custo definidos, mas mesmo assim o projeto fracassa. Entre outras coisas, a organização pode não ter a habilidade necessária para vender o produto, ou a empresa pode até ter essa habilidade, mas o produto não apresenta efetivamente apelo comercial.
- *Riscos legais*: existem problemas ou possibilidade de litígio? Uso de material protegido por direitos autorais? Necessidade de celebração de contrato com terceiros? Normas e leis específicas em outros estados ou países?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **# Riscos Potenciais = Riscos de Projeto**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software)**
- Os riscos de gerenciamento do projeto envolvem a capacidade da equipe de planejar e seguir o plano dentro do cronograma e do custo previstos. Esse tipo de risco costuma se manifestar das seguintes formas:
 - *Riscos de requisitos*: a equipe, por ser inexperiente, pode não ter sido capaz de identificar corretamente os requisitos do projeto, o que causará problemas ao longo dele. Requisitos poderão ser insuficientes, excessivos ou incorretos. Ou, ainda, os requisitos podem ser naturalmente instáveis, em razão de características do próprio projeto.
 - *Riscos de processo*: o modelo de processo escolhido é adequado às características do projeto? A equipe tem experiência com o processo? O gerente tem experiência em projetos anteriores?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **# Riscos Potenciais = Riscos de Projeto**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software)**
- *Riscos de orçamento*: até que ponto a verba necessária para o projeto está garantida? Os custos foram corretamente previstos em projetos anteriores?
- *Riscos de cronograma*: é possível que os prazos sejam alterados e a ordem em que as funcionalidades devem ser entregues possa mudar? O planejador deve saber em que grau a equipe se mostrou capaz de ater-se ao cronograma em projetos passados.
- A inexistência de qualquer uma dessas informações caracteriza um risco importante ao projeto na medida da sua incerteza.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **# Riscos Potenciais = Checklist de Riscos**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software) – 8.3**
- O método de identificação de riscos do SEI (CARR, KONDA, MONARCH, ULRICH & WALKER, 1993) é baseado em uma taxonomia que contém termos relacionados com o processo de desenvolvimento de software. Para cada item dessa taxonomia, pode ser elaborado um questionário ou *checklist*, a partir de qual riscos podem ser identificados.
- São definidas três grandes classes de risco:
 - Engenharia do produto.
 - Ambiente de desenvolvimento.
 - Restrições externas.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **# Riscos Potenciais = Checklist de Riscos**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software) – 8.3**
- Cada uma dessas categorias possui seus próprios elementos de risco, e cada elemento apresenta suas propriedades, a partir das quais são formuladas perguntas que permitem analisar se o projeto corre ou não algum risco referente a elas.
- Inicialmente, o *checklist* referente à *engenharia do produto*, tem como elementos de risco os requisitos, o design, a codificação e o teste de unidade e a integração, além de outros aspectos específicos de engenharia.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Requisitos: Estabilidade: os requisitos podem mudar durante o desenvolvimento?

Os requisitos são estáveis? Se não, quais os efeitos disso no sistema? (qualidade / funcionalidade / cronograma / integração / design / teste)

As interfaces externas do sistema estão mudando ou vão mudar?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Requisitos: Completeza: estão faltando requisitos ou estão especificados de forma incompleta?

Existem tópicos a serem esclarecidos nas especificações?

Existem requisitos que se sabe que deveriam estar nas especificações, mas não estão? Se sim, é possível obter esses requisitos e colocá-los nas especificações?

O cliente tem expectativas ou requisitos que não estão escritos? Se sim, há forma de capturá-los? As interfaces externas são completamente definidas?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Requisitos: Clareza: os requisitos estão obscuros ou necessitam de interpretação?

Requisitos: Validade: os requisitos vão levar ao produto que o cliente tem em mente?

Você é capaz de entender os requisitos da forma como estão escritos? Se não, há ambiguidades sendo resolvidas satisfatoriamente? Se sim, não há ambiguidades ou problemas de interpretação?

Existem requisitos que podem não especificar exatamente o que o cliente quer? Se sim, como você está resolvendo isso?

Você e o cliente compreendem a mesma coisa a partir dos requisitos? Se sim, existe um processo para determinar isso?

Como você valida os requisitos junto ao cliente? Prototipação? Análise? Simulação?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Requisitos: Exequibilidade: os requisitos são exequíveis de um ponto de vista analítico?

Há requisitos que sejam tecnicamente difíceis de implementar? Se sim, quais são? Se sim, porque eles são difíceis de implementar? Se não, foram feitos estudos de exequibilidade sobre os requisitos? Se sim, qual seu grau de confiança em tais estudos?

Requisitos: Precedentes: os requisitos especificam algo que nunca foi feito antes, ou que a empresa nunca fez antes?

Existe algum requisito do estado da arte? Tecnologias? Métodos? Linguagens? Hardware? Se não, algum destes é novo para a equipe? Se sim, a equipe tem conhecimento suficiente nestas áreas? Se não, há um plano para adquirir conhecimento nestas áreas?

Requisitos: Escala: os requisitos especificam um produto maior, mais complexo ou requerendo mais organização do que a empresa tem experiência?

O tamanho e complexidade do sistema são uma preocupação? Se não, você já fez algo deste tamanho e complexidade antes? O tamanho requer uma organização maior do que o usual para a empresa?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Design: Funcionalidade: existem problemas potenciais para obter os requisitos funcionais?

Design: Dificuldade: o design ou implementação serão difíceis de serem realizados?

Design: Interfaces: as interfaces internas de hardware e software são bem definidas e controladas?

Há algum algoritmo especificado que possa não satisfazer os requisitos? Se não, há algum algoritmo ou design que obtenha os requisitos de forma marginal?

Como você determina a exequibilidade dos algoritmos e design? Prototipação? Modelagem? Análise? Simulação?

Alguma parte do design depende de hipóteses otimistas ou não realistas?

Existe algum requisito que seja difícil de obter um design? Se não, você tem soluções para todos os requisitos? Se sim, quais são os requisitos e por que eles são difíceis?

Há interfaces internas bem definidas? Software para software? Software para hardware?

Há um processo para definir interfaces internas? Se sim, há um processo de controle de mudança para interfaces internas?

Há hardware sendo desenvolvido em paralelo com o software? Se sim, as especificações do hardware estão mudando? Se sim, todas as interfaces com o software já foram definidas?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Design: Performance: existem tempos de resposta ou taxas de transferência rigorosos?

Existem problemas com performance? Taxa de transferência?
Escalonamento de eventos de tempo real assíncronos?
Respostas em tempo real? Tempo para recuperação de falhas (*recovery timeline*)? Tempo de resposta? Tempo de resposta, acesso e número simultâneo de usuários da base de dados?

Foi feita uma análise de performance? Se sim, qual o seu nível de confiança na análise feita? Se sim, você tem um modelo para rastrear a performance através do design e implementação?

Design: Testabilidade: o produto é difícil ou impossível de testar?

O software vai ser fácil de testar?

O design inclui características que facilitam o teste?

Os testadores se envolveram com a análise dos requisitos?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Restrições de hardware: existem restrições apertadas no hardware alvo?

Arquitetura?

Capacidade de memória?

Taxa de transferência?

Resposta em tempo real?

Tempo para recuperação de falhas?

Performance da base de dados?

Funcionalidade?

Confiabilidade?

Disponibilidade?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Software não desenvolvido: há problemas com software usado, mas não desenvolvido pela equipe?

Você está reusando ou fazendo reengenharia em software não desenvolvido pela equipe? Se sim, você prevê algum problema? Documentação? Performance? Funcionalidade? Prazo de entrega? Personalização?

Se estiver usando COTS, há algum problema com este tipo de software? Documentação insuficiente para determinar interfaces, tamanho ou performance? Performance fraca? Requer uma grande parcela de memória ou da base de dados? É difícil de interfacear com o software? Não foi testado sistematicamente? Não está livre de defeitos? Não foi adequadamente mantido? O vendedor demora para responder?

Você prevê algum problema com a integração de atualizações ou revisões de COTS?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Codificação: Exequibilidade: a implementação do design é difícil ou impossível?

Codificação: Teste: os níveis e tempos especificados para os testes de unidade são adequados?

Existem partes do produto não completamente especificadas no design?

Os algoritmos e design são fáceis de implementar?

Você começa o teste de unidade antes de verificar código com respeito ao design?

Testes de unidade suficientes são especificados?

Há tempo suficiente para realizar todo o teste de unidade que você acha necessário?

Serão feitos relaxamentos nos testes de unidades se houver problemas de cronograma?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Codificação: Codificação/Implementação: há problemas com codificação e implementação?

Os designs e especificações estão em um nível adequado para permitir a codificação?

O design muda à medida que o código é escrito?

Há restrições de sistema que tornam o código difícil de ser feito? Tempo? Memória? Armazenamento externo?

A linguagem de programação é adequada para este projeto?

O projeto usa mais de uma linguagem? Se sim, há compatibilidade entre o código produzido pelos diferentes compiladores?

O computador de desenvolvimento é o mesmo onde o sistema vai rodar? Se não, há diferenças relativas à compilação entre os dois computadores?

Se estiver sendo desenvolvido hardware, suas especificações são suficientes para o desenvolvimento do software?

As especificações do hardware mudam à medida que o software é codificado?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Integração: Ambiente: o ambiente de integração e teste é adequado?

Haverá hardware suficiente para uma adequada integração e teste?

Há problemas para desenvolver cenários realísticos e testar dados para demonstrar algum dos requisitos? Tráfego de dados especificado? Resposta em tempo real? Tratamento de eventos assíncronos? Interação multiusuário?

Você é capaz de verificar a performance nas suas instalações? Se sim, isso é suficiente para todos os testes?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Integração: Produção: a definição de interfaces e instalações são inadequadas ou o tempo insuficiente?

O hardware-alvo estará disponível quando necessário?

Critérios de aceitação foram acordados com o cliente para todos os requisitos? Se sim, há um acordo formal?

As interfaces externas foram definidas, documentadas e transformadas em baselines?

Há algum requisito que seja difícil de testar?

Foi especificada integração de produto suficiente?

Foi alocado tempo suficiente para integração e teste do produto?

Se usar COTS, os dados do vendedor serão aceitos na verificação dos requisitos alocados a COTS? Se sim, o contrato é claro neste ponto?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Integração: Sistema: a integração de sistema é descoordenada, a definição de interface é pobre ou as instalações são inadequadas?

Integração de sistema suficiente foi especificada?

Tempo suficiente para integração de sistema e testes foi previsto?

O produto será integrado a um sistema existente? Se sim, haverá um período de transição definido? Se não, como vai-se garantir que o programa funcionará corretamente depois de integrado?

A integração do sistema vai ocorrer nas instalações do cliente?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Atributos: Manutenibilidade: a implementação será difícil de entender e manter?

Atributos: Confiabilidade: os requisitos de confiabilidade ou disponibilidade são difíceis de obter?

A arquitetura, design ou código criam dificuldades de manutenção?

A equipe de manutenção foi envolvida cedo no processo?

A documentação do produto é suficiente para que seja mantido por uma organização externa?

Existem requisitos de confiabilidade?

Existem requisitos de disponibilidade? Se sim, os prazos de recuperação de falhas são um problema?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Atributos: Riscos à segurança (safety): os requisitos relacionados com riscos à segurança são inexecutáveis ou não demonstráveis?

Atributos: Segurança do sistema (security): os requisitos de segurança do sistema são mais rigorosos do que o usual?

Existem requisitos relacionados com riscos à segurança? Se sim, você vê alguma dificuldade em obtê-los?

Será difícil verificar a satisfação destes requisitos?

Existem requisitos de segurança no estado da arte ou sem precedentes?

O sistema deve seguir regulamentos estritos de segurança estabelecidos por agência governamentais (como, por exemplo, Orange Book)?

Você já implementou este nível de segurança antes?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Atributos: Fatores humanos: o sistema será difícil de usar por conta de interface com usuário mal definida?

Você vê alguma dificuldade em satisfazer os requisitos de fatores humanos? Se não, como você garante que vai satisfazer estes requisitos?

Se estiver usando prototipação, é um protótipo do tipo *throw-away*? Se não, está fazendo prototipação evolucionária? Se sim, você tem experiência neste tipo de desenvolvimento? Se sim, há versões preliminares entregáveis? Se sim, isso complica o controle de mudança?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Atributos: Especificações: a documentação é adequada para o design, implementação e teste do sistema?

A especificação dos requisitos do software é adequada para o design do sistema?

A especificação dos requisitos de hardware é adequada para o design e implementação do sistema?

As interfaces externas necessárias foram bem especificadas?

As especificações de teste são adequadas para testar completamente o sistema?

Se já estiver na fase de implementação ou após ela, as especificações de design são adequadas para implementar o sistema? Interfaces internas?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Processo de desenvolvimento: Formalidade: a implementação será difícil de entender e manter?

Há mais de um modelo de desenvolvimento sendo usado? Espiral? Cascata? Incremental? Se sim, a coordenação entre eles é um problema?

Há planos formais e controlados para todas as atividades de desenvolvimento? Análise de requisitos? Design? Codificação? Integração e teste? Instalação? Garantia de qualidade? Gerenciamento de configuração? Se sim, os planos especificam bem o processo? Se sim, os desenvolvedores são familiarizados com os planos?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Processo de desenvolvimento: Adequação: o processo é adequado para o modelo de desenvolvimento?

Processo de desenvolvimento: Controle de processo: o processo de desenvolvimento é executado, monitorado e controlado com métricas? Os locais de desenvolvimento distribuídos são coordenados?

O processo de desenvolvimento é adequado para este produto?

O processo de desenvolvimento é suportado por um conjunto de procedimentos, métodos e ferramentas compatíveis?

Todos seguem o processo de desenvolvimento? Se sim, como isso é garantido?

Você consegue mensurar se o processo de desenvolvimento está atingindo suas metas de qualidade e produtividade?

Se há locais de desenvolvimento distribuídos, há coordenação adequada entre eles?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Processo de desenvolvimento: Familiaridade: os membros do projeto têm experiência no uso do processo? O processo é compreendido por toda a equipe?

Processo de desenvolvimento: Controle de produto: há mecanismos para controlar a mudança no produto?

As pessoas estão confortáveis com o processo de desenvolvimento?

Existe um mecanismo de controle de rastreabilidade de requisitos, que rastreia requisitos desde sua especificação até os casos de teste?

O mecanismo de rastreabilidade é usado na avaliação de impacto de mudanças de requisitos?

Existe um mecanismo formal de controle de mudança? Se sim, ele cobre todas as mudanças nas baselines de requisitos, design, código e documentação?

As mudanças em qualquer nível são mapeadas para cima até o nível de sistema e para baixo até o nível de teste?

Há análise adequada quando novos requisitos são adicionados ao sistema?

Você tem algum meio de rastrear interfaces?

Os planos e procedimentos de teste são atualizados como parte do processo de mudança?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Sistema de desenvolvimento: Capacidade: existe poder de processamento, estações de trabalho, memória e espaço de armazenamento suficientes?

Sistema de desenvolvimento: Adequação: o sistema de desenvolvimento dá suporte a todas as fases, atividades e funções?

Há estações de trabalho e poder de processamento para toda a equipe?

Há capacidade suficiente para fases que se entrelaçam como codificação, integração e teste?

O sistema de desenvolvimento suporta todos os aspectos do projeto? Análise de requisitos? Análise de performance? Design? Codificação? Teste? Documentação? Gerenciamento de configuração? Requisitos e gerenciamento de rastreabilidade?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Sistema de desenvolvimento: Usabilidade: qual a facilidade de uso do sistema de desenvolvimento?

As pessoas avaliam que o sistema de desenvolvimento é fácil de usar?

Existe boa documentação sobre o sistema de desenvolvimento?

Sistema de desenvolvimento: Familiaridade: existe pouca experiência anterior da empresa ou membros da equipe com o sistema de desenvolvimento?

As pessoas já usaram estas ferramentas e métodos antes?

Sistema de desenvolvimento: Confiabilidade: o sistema de desenvolvimento sofre com erros, paralisação e capacidade de backup nativa insuficiente?

O sistema de desenvolvimento é considerado confiável?
Compilador? Ferramentas de desenvolvimento? Hardware?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Sistema de desenvolvimento: Suporte ao sistema: existe suporte ágil de vendedor ou especialista no sistema de desenvolvimento?

Sistema de desenvolvimento: Capacidade de entrega (deliverability): os requisitos de definição e aceitação para a entrega do sistema de desenvolvimento para o cliente não foram incluídos no orçamento?

A equipe foi treinada no uso das ferramentas?

Você tem acesso a especialistas no seu uso?

Os vendedores respondem rapidamente aos problemas?

Você vai entregar o sistema de desenvolvimento ao cliente?

Se sim, recursos, cronograma e orçamento foram alocados para esta entrega?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Processo de gerência: Planejamento: o planejamento é oportuno, líderes técnicos foram incluídos e planos de contingência realizados?

O desenvolvimento é gerenciado de acordo com o plano? Se sim, as pessoas apagam incêndios com frequência?

As pessoas de todos os níveis estão incluídas no planejamento de seu próprio trabalho?

Existem planos de contingência para riscos conhecidos? Se sim, como você determina a hora de ativar estes planos?

Questões de longo prazo foram adequadamente tratadas?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Processo de gerência: Organização de projeto: os papéis e relacionamentos estão claros?

Processo de gerência: Experiência em gerência: os gerentes têm experiência em desenvolvimento de software, gerenciamento de projeto de software, no domínio de aplicação, no processo de desenvolvimento ou em grandes projetos?

A organização do desenvolvimento é efetiva?

As pessoas compreendem os seus papeis e os papeis dos outros?

As pessoas sabem quem tem autoridade para quê?

O projeto tem gerentes experientes? Gerenciamento de projeto de software? Com desenvolvimento de software? Com o processo de desenvolvimento em uso? Com o domínio da aplicação? Com projetos deste tamanho ou complexidade?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Processo de gerência: Interfaces da equipe de desenvolvimento: existem interfaces fracas com o cliente, outros contratados ou gerentes de nível superior?

A gerência comunica problemas para cima e para baixo na linha hierárquica?

Os conflitos com o cliente são documentados e resolvidos no tempo apropriado?

A gerência envolve as pessoas apropriadas nos encontros com o cliente? Líderes técnicos? Desenvolvedores? Analistas?

A gerência trabalha de forma que todas as facções do lado do cliente estejam representadas nas decisões sobre funcionalidade e operação?

É uma boa política apresentar uma visão otimista ao cliente ou gerentes de nível superior?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Métodos de gerência: Monitoramento: as métricas de gerência são definidas e o progresso do desenvolvimento rastreado?

Métodos de gerência: Gerenciamento de pessoal: o pessoal do projeto é treinado e usado adequadamente?

Existem relatórios de status estruturados periódicos? Se sim, as pessoas obtêm respostas aos seus relatórios de status?

A informação apropriada é reportada aos níveis organizacionais corretos?

O progresso é comparado com o plano? Se sim, a gerência tem uma visão clara sobre o que está acontecendo?

O pessoal é treinado nas habilidades requeridas pelo projeto? Se sim, isso é parte do plano do projeto?

Há pessoas alocadas ao projeto que não possuem o perfil necessário?

É fácil para os membros do projeto conseguir ação da gerência?

Os membros em todos os níveis estão conscientes de seu status *versus* plano?

As pessoas sentem que é importante manter o plano?

A gerência consulta as pessoas antes de tomar decisões que afetam o trabalho delas?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Métodos de gerência: Garantia de qualidade: existem procedimentos e recursos adequados para garantir a qualidade do produto?

A função de garantia de qualidade tem pessoal adequado para o projeto?

Existem mecanismos adequados para garantir a qualidade?
Se sim, todas as áreas e fases possuem procedimentos de qualidade? Se sim, as pessoas estão acostumadas a trabalhar com estes procedimentos?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Métodos de gerência: Gerenciamento de configuração: os procedimentos de gerenciamento de configuração, mudança e instalação são adequados?

Há um sistema gerenciador de configuração adequado?

A função de gerenciamento de configuração tem pessoal adequado?

É necessária coordenação com algum sistema instalado? Se sim, há gerenciamento de configuração adequado para o sistema instalado? Se sim, o sistema de gerenciamento de configuração sincroniza o seu trabalho com as mudanças no local?

O sistema vai ser instalado em múltiplos locais? Se sim, o sistema de gerenciamento de configuração permite múltiplos locais?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Ambiente de trabalho: Atitude em relação a qualidade: há falta de orientação relacionada com o trabalho com qualidade?

Ambiente de trabalho: Cooperação: falta espírito de equipe? Os conflitos resultantes requerem intervenção da gerência?

Todos os níveis de pessoal estão orientados para procedimentos de qualidade?

O cronograma costuma ficar no caminho da qualidade?

As pessoas trabalham cooperativamente através das fronteiras funcionais?

As pessoas trabalham cooperativamente na direção de metas comuns?

A intervenção da gerência às vezes é necessária para fazer as pessoas trabalharem juntas?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Ambiente de trabalho: Comunicação: há pouca consciência a respeito de metas ou missão e comunicação pobre de informação técnica entre membros da equipe e gerentes?

Há boa comunicação entre os membros do projeto?
Gerentes? Líderes técnicos? Desenvolvedores? Testadores?
Gerente de configuração? Equipe de qualidade?

Os gerentes são receptivos às comunicações da equipe? Se sim, você se sente à vontade para pedir ajuda ao seu gerente?
Os membros são capazes de comunicar riscos mesmo sem ter uma solução pronta?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Ambiente de trabalho: Moral: há uma atmosfera não produtiva ou não criativa?

As pessoas sentem que não há reconhecimento ou recompensa por trabalho feito acima das expectativas?

Como está a moral no projeto? Se não estiver bem, qual é o principal motivo para a baixa moral?

Há problemas para manter as pessoas que são necessárias?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Recursos: Cronograma: o cronograma é inadequado ou instável?

O cronograma tem sido estável?

O cronograma é realístico? Se sim, o método de estimação é baseado em dados históricos? Se sim, o método de estimação funcionou bem no passado?

Existe alguma coisa para a qual um cronograma realístico não foi preparado? Análises e estudos? Garantia de qualidade? Treinamento? Cursos e treinamento em manutenção? Equipamentos? Sistema de desenvolvimento que vai ser entregue?

Há dependências externas que poderiam impactar o cronograma?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Recursos: Pessoal: a equipe é inexperiente, sem conhecimento de domínio, sem habilidades ou em número insuficiente?

Há alguma área em que esteja faltando pessoal qualificado?
Engenharia de software e métodos de análise de requisitos?
Especialistas em algoritmos? Design e métodos de design?
Linguagens de programação? Métodos de integração e teste?
Confiabilidade? Manutenibilidade? Disponibilidade? Fatores humanos? Gerenciamento de configuração? Garantia de qualidade? Ambiente-alvo? Nível de segurança? COTS? Reúso de software? Sistema operacional? Banco de dados? Domínio de aplicação? Análise de performance? Aplicações de tempo crítico?

Há pessoal adequado para formar a equipe de projeto?

A equipe é estável?

Você tem acesso às pessoas certas quando precisa delas?

Os membros da equipe já implementaram sistemas deste tipo?

O projeto depende de algumas poucas pessoas?

Há problemas para obter pessoal?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Recursos: Orçamento: o orçamento é insuficiente ou instável?

O orçamento é estável?

O orçamento é baseado em uma estimativa realística? Se sim, o método de estimativa é baseado em dados históricos? Se sim, o método funcionou bem no passado?

Funcionalidades ou características do sistema já foram removidas para reduzir custo?

Há alguma coisa para a qual um orçamento adequado não foi alocado? Análises e estudos? Garantia de qualidade? Treinamento? Cursos de manutenção? Equipamento? Sistema de desenvolvimento a ser entregue?

Mudanças no orçamento acompanham mudanças nos requisitos?

Se sim, esta é uma parte padrão do processo de controle de mudança?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Recursos: Instalações: as instalações são adequadas para construir e entregar o produto?

As instalações de desenvolvimento são adequadas?
O ambiente de integração é adequado?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Contrato: Tipo de contrato: o tipo de contrato é uma fonte de risco para o projeto?

Que tipo de contrato foi feito? (custo mais prêmio, preço fixo...). Ele apresenta algum problema?

O contrato representa uma carga para algum aspecto do projeto? Declarações de trabalho? Especificações? Descrições de itens de dados? Partes? Envolvimento excessivo de cliente?

A documentação requerida é uma carga? Quantidade excessiva? Cliente detalhista e exigente? Longo ciclo de aprovações?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Contrato: Restrições: o contrato causa alguma restrição?

Contrato: Dependências: o projeto tem dependência de algum produto ou serviço externo que pode afetar o produto?

Há problemas com direitos de dados?

Contratados associados?

Contratado principal? (se você for um subcontratado)

Subcontratados?

Vendedores ou fornecedores?

Equipamento ou software desenvolvido pelo cliente?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Comunicação externa: Cliente: há problemas com o cliente, como ciclo longo de aprovação de documentos, comunicação fraca ou conhecimento inadequado do domínio?

O ciclo de aprovação do cliente é adequadamente rápido?
Documentação? Revisões de projeto? Revisões formais?

Você sempre segue em frente antes de receber aprovação do cliente?

O cliente entende os aspectos técnicos do sistema?

O cliente entende software?

O cliente interfere com processos ou pessoas?

O gerente trabalha com o cliente para alcançar entendimentos e decisões mútuas em tempo aceitável? Entendimento sobre requisitos? Critérios de teste? Ajustes de cronograma? Interfaces?

Quão efetivos são seus mecanismos para chegar a um acordo com o cliente? Grupos de trabalho (contratuais)? Encontros de intercâmbio técnico (contratuais)?

Todas as facções do cliente são envolvidas para se chegar a acordos? Se sim, há um processo formalmente definido?

A gerência apresenta um quadro realista ou otimista ao cliente?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#8.4 Análise de Riscos

(Fonte: RAUL, Wazlawick - Engenharia de Software)

Uma vez identificados os riscos potenciais, a análise de riscos vai determinar quais são verdadeiramente relevantes para que se gastem tempo e dinheiro com sua prevenção.

- Em geral, a análise de riscos vai tentar determinar a probabilidade de ocorrência e o impacto de cada risco potencial.
- Algumas propriedades de riscos, entre outras, podem ser assim identificadas:

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#8.4 Análise de Riscos

(Fonte: RAUL, Wazlawick - Engenharia de Software)

- *Probabilidade*: é a chance de que o risco realmente se torne um problema. Caso se disponha de séries históricas, a probabilidade pode ser medida quantitativamente. Por exemplo, se 35% dos projetos anteriores sofreram atraso no cronograma, então pode-se esperar que esse risco tenha uma probabilidade de 35% em projetos futuros.
- Porém, na maioria das vezes, poderá ser mais factível considerar apenas valores discretos como probabilidade *alta* (quase certo que vá ocorrer), *média* (tem alguma chance de ocorrer) e *baixa* (pouco provável que ocorra) para um risco.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#8.4 Análise de Riscos

(Fonte: RAUL, Wazlawick - Engenharia de Software)

- *Impacto*: o impacto é a medida do prejuízo que um risco pode trazer ao projeto. O impacto é independente da probabilidade. Pode-se ter riscos de alto impacto com alta ou baixa probabilidade e riscos de baixo impacto com alta ou baixa probabilidade.
- O impacto pode ser mensurado, por exemplo, quando se sabe o valor da multa por dia de atraso na entrega do produto, mas normalmente será mais prático avaliar o impacto como *alto* (pode inviabilizar o projeto), *médio* (pode aumentar significativamente o custo do projeto) ou *baixo* (apenas provoca algum contratempo, sem inviabilizar os objetivos do projeto).

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#8.4 Análise de Riscos

(Fonte: RAUL, Wazlawick - Engenharia de Software)

- *Proximidade*: alguns riscos podem ser de alta probabilidade, mas baixa proximidade, ou seja, podem ocorrer só um futuro distante. Por exemplo, a obsolescência tecnológica é um risco de alto impacto e alta probabilidade, mas possivelmente de baixa ou média proximidade, dependendo da tecnologia.
- Quando a proximidade é considerada propriedade dos riscos, convém também usá-la para o cálculo da exposição do risco.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#8.4 Análise de Riscos

(Fonte: RAUL, Wazlawick - Engenharia de Software)

- *Acoplagem*: a acoplagem define o quanto um risco pode afetar outros riscos. Por exemplo, se os requisitos estiverem mal estabelecidos, outros riscos de projeto podem ter sua probabilidade ou seu impacto aumentado.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#8.4 Análise de Riscos

(Fonte: RAUL, Wazlawick - Engenharia de Software)

- O produto da probabilidade pelo impacto consiste na *exposição* (ou *importância*) do risco. Assim, riscos de maior exposição precisarão ter uma abordagem detalhada no projeto para que sejam tratados. Já os riscos de baixa exposição não necessitam de tanto investimento. Eventualmente, será suficiente manter a equipe ciente de sua existência para tomar providências caso a exposição do risco se altere.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#8.4 Análise de Riscos

(Fonte: RAUL, Wazlawick - Engenharia de Software)

- Deve-se lembrar de que, apesar dessa identificação das propriedades de riscos no momento do planejamento, durante a execução do projeto, a equipe deve ter em mente que todas as propriedades podem mudar com o tempo. Assim, o monitoramento dos riscos também é uma atividade que deverá ser realizada com frequência.
- Quando a probabilidade e o impacto do risco são avaliados qualitativamente, pode-se usar uma tabela, a partir da combinação da probabilidade pelo impacto, chega-se a um valor para a exposição do risco.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#8.4 Análise de Riscos

(Fonte: RAUL, Wazlawick - Engenharia de Software)

Probabilidade				
		Alta	Média	Baixa
Impacto	Alto	Alta exposição	Alta exposição	Média exposição
	Médio	Alta exposição	Média exposição	Baixa exposição
	Baixo	Média exposição	Baixa exposição	Baixa exposição

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#8.4 Análise de Riscos

(Fonte: RAUL, Wazlawick - Engenharia de Software)

- Normalmente, os atributos de riscos são definidos em função da experiência e do conhecimento de especialistas. Mas, se houver um registro de projetos anteriores, pode-se ter uma expectativa mais concreta sobre a ocorrência de alguns riscos. Assim, muitas vezes, o planejador de projeto poderá adicionar à sua análise de riscos um conjunto de dados ou observações que justifiquem sua escolha para os valores de determinadas propriedades dos riscos.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#8.4 Análise de Riscos

(Fonte: RAUL, Wazlawick - Engenharia de Software)

- *Planos de mitigação de riscos* são executados antes que o risco ocorra. Para os riscos de alta exposição, os planos de mitigação são definidos ainda na fase de planejamento do projeto. Para riscos de média exposição, os planos são definidos e guardados para serem aplicados caso essa exposição aumente ao longo do projeto.
- Há dois tipos de planos de mitigação: *plano de redução de probabilidade* e *plano de redução de impacto*. O primeiro age nas causas e o outro age nos efeitos do risco.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#8.5. Planos de Mitigação de Riscos

(Fonte: RAUL, Wazlawick - Engenharia de Software)

Para exemplificar a apresentação de planos de mitigação, será usado um conjunto de riscos identificados e analisados conforme tabela do slide 13. Os riscos cujo código é prefixado com “t” são *riscos tecnológicos*. Os riscos prefixados com “pe” são *riscos de pessoal*. Os riscos prefixados com “pr” são *riscos de projeto* e os riscos prefixados com “l” são *riscos legais*.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

8.5. Planos de Mitigação de Riscos – Tabela 8.6 - (Fonte: RAUL, Wazlawick - Engenharia de Software)

Id	Causa	Risco	Efeito	Prob.	Imp.	Exp.
pr3	Requisitos ainda muito instáveis.	Pode haver mudanças importantes nos requisitos ao longo do desenvolvimento.	Perda de tempo desenvolvendo partes que depois não serão usadas e atrasos no cronograma.	A	A	A
pr2	O tempo de desenvolvimento pode ser longo.	Pode haver concorrentes que lancem produtos antes.	Chegar ao mercado depois da janela de oportunidade.	A	A	A
t8	Necessidade de muitos comandos baseados em gestos.	Gestos muito parecidos podem significar comandos diferentes.	O sistema pode interpretar erroneamente os comandos (desenhos, formas). Usuário pode ter que decorar muitos comandos diferentes.	A	M	A
pe1	Ainda não se sabe se será possível contratar equipe com experiência nas tecnologias.	Necessidade de treinamento.	Atrasos de cronograma e custos com treinamento.	A	M	A
t4	O processo implementado pela ferramenta pode não atender aos desejos do usuário.	O usuário não vai escolher a ferramenta porque usa um processo de desenvolvimento diferente.	Problemas relacionados com a venda. Pode haver necessidade de implementar vários processos, o que vai contra a filosofia inicial da ferramenta. Grandes empresas já têm processo estabelecido e teriam que mudar.	M	A	A
t6	A tecnologia de comando de voz ainda não é bem desenvolvida.	Comandos de voz podem não ser corretamente entendidos.	Usuários frustrados.	A	B	M
t9	Não existem ferramentas CASE com gráficos 3D ou em níveis de profundidade.	Não existe um padrão ou referência para tais interfaces nem estudos de usabilidade.	Necessidade de pesquisar padrões de usabilidade para interfaces 3D em ferramentas CASE.	A	B	M

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

8.5. Planos de Mitigação de Riscos – Tabela 8.6 - (Fonte: RAUL, Wazlawick - Engenharia de Software)

Id	Causa	Risco	Efeito	Prob.	Imp.	Exp.
t3	Não é conhecido um padrão de usabilidade para CASE em touch screen.	Poderá ser desenvolvida uma ferramenta com usabilidade falha.	Problemas com usuário final (desinteresse).	M	M	M
pe1	O projeto será desenvolvido por bolsistas.	Bolsistas não veem o projeto como carreira.	Podem-se perder desenvolvedores ao longo do projeto, necessitando de substituição.	M	M	M
l1	Uso de tecnologia de terceiros.	Pagamento de direitos autorais.	Aumento de custo.	M	M	M
t1	Superfície de toque é tecnologia nova.	Podem ocorrer mudanças nos padrões. Qual o melhor sistema operacional?	Produto obsoleto ou necessidade de desenvolver para vários sistemas operacionais.	B	M	B
t2	Tecnologia nova.	Podem não existir bibliotecas suficientemente adequadas para o desenvolvimento.	Necessidade de desenvolver novas bibliotecas básicas.	B	B	B
t5	O acesso aos recursos avançados será secundário na interface, que prioriza as ações mais elementares.	Pode gerar problemas de usabilidade para usuários mais avançados.	Gerar desinteresse por usuários avançados. É necessária uma boa análise de caso de uso e usabilidade na ferramenta durante seu desenvolvimento.	B	B	B
t7	O código gerado pela ferramenta, por default, não será modificável.	O código pode não ser o mais eficiente possível.	Sistemas gerados pela ferramenta podem ser ineficientes.	B	B	B

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#8.5. Planos de Mitigação de Riscos

(Fonte: RAUL, Wazlawick - Engenharia de Software)

- O projeto trata da produção de uma ferramenta CASE em tecnologia *touch screen* com comandos de voz e interface em realidade virtual.
- Os riscos foram identificados e avaliados em função de sua probabilidade (P) e impacto (I), e a exposição (E), a resultante foi calculada de acordo com a Tabela 8.5, com os valores *baixo* (B), *médio* (M) e *alto* (A). Os riscos aparecem na tabela ordenados de acordo com sua exposição.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#8.5. Planos de Mitigação de Riscos

(Fonte: RAUL, Wazlawick - Engenharia de Software)

- Os riscos de exposição alta tiveram planos de mitigação elaborados de forma sumária e executados. Os riscos de exposição média deveriam ter seus planos apenas elaborados e guardados caso a exposição do risco aumentasse. Os riscos de baixa exposição deveriam apenas continuar sendo monitorados e, caso sua exposição aumentasse para média, deveriam ter seus planos elaborados (e executados, caso a exposição passasse para alta).
- Nas subseções a seguir são discutidos estes planos de mitigação bem como apresentados exemplos de planos simplificados para os riscos indicados na Tabela 8.6.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#8.5.1. Plano de Redução de Probabilidade de Risco

(Fonte: RAUL, Wazlawick - Engenharia de Software)

O plano de redução de probabilidade de risco consiste nas ações identificadas como necessárias para diminuir a probabilidade de que um risco ocorra. Esse tipo de plano deve agir nas *causas* do risco, ou seja, na segunda coluna da Tabela 8.6. Por exemplo, se é bem provável que a equipe tenha dificuldade com uma nova tecnologia a ser usada, podem-se realizar cursos de treinamento para diminuir a probabilidade de isso se tornar um estorvo ao andamento do projeto.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#8.5.1. Plano de Redução de Probabilidade de Risco

(Fonte: RAUL, Wazlawick - Engenharia de Software)

- Riscos classificados como de alta exposição devem ter planos de mitigação e contingência com atividades bem definidas, prazos e responsáveis pela sua execução. É importante que tais atividades sejam incluídas no plano do projeto ou iteração e que suas ações sejam contabilizadas no tempo e no custo do projeto.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#8.5.1. Plano de Redução de Probabilidade de Risco

(Fonte: RAUL, Wazlawick - Engenharia de Software)

- É interessante lembrar que alguns ciclos de vida, como Espiral, Cascata com Redução de Risco, métodos ágeis e o UP, já preveem que atividades de estudo e mitigação de riscos sejam previstas e incorporadas ao projeto.
- No caso de riscos de média exposição, é recomendado apenas elaborar os planos para serem aplicados mais tarde caso seja necessário.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#8.5.1. Plano de Redução de Probabilidade de Risco

(Fonte: RAUL, Wazlawick - Engenharia de Software)

- Riscos de baixa exposição podem apenas ser monitorados para verificar se a probabilidade ou o impacto aumentam ao longo do projeto e, nesse caso, o plano de redução de probabilidade pode ser criado como resposta a essa mudança de estado.
- Considerando o exemplo da Tabela 8.6, os planos simplificados de redução de probabilidade poderiam ser elaborados como mostrado na Tabela 8.7.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

8.5.1. Plano de Redução de Probabilidade de Risco Tabela 8.7 - (Fonte: RAUL, Wazlawick – Eng. de Software)

Id	Causa do risco	Risco	Plano de redução de probabilidade
pr3	Requisitos ainda muito instáveis.	Pode haver mudanças importantes nos requisitos ao longo do desenvolvimento.	Realizar reuniões de eliciação de requisitos. Inspecionar requisitos. Procurar produtos semelhantes na Internet e analisá-los. Planejar o desenvolvimento de protótipos.
pr2	O tempo de desenvolvimento pode ser longo.	Pode haver concorrentes que lancem produtos antes.	Planejar desenvolvimento orientado a cronograma com entregas de versões parciais usáveis em intervalos de dois meses.
t8	Necessidade de muitos comandos baseados em gestos.	Gestos muito parecidos podem significar comandos diferentes.	Pesquisar padrões existentes para comandos baseados em gestos e catalogá-los. Definir hierarquia de comandos e comandos baseados em contextos para reduzir a quantidade de comandos necessários.
pe1	Ainda não se sabe se será possível contratar equipe com experiência nas tecnologias.	Necessidade de treinamento.	Publicar anúncio solicitando currículos para pessoas com as habilidades desejadas.
t4	O processo implementado pela ferramenta pode não atender aos desejos do usuário.	O usuário não vai escolher a ferramenta porque usa um processo de desenvolvimento diferente.	Pesquisar qual o processo mais usado no mercado-alvo. Adaptar a ferramenta para uso com o(s) processo(s) dominante(s).

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

8.5.1. Plano de Redução de Probabilidade de Risco Tabela 8.7 - (Fonte: RAUL, Wazlawick – Eng. de Software)

Id	Causa do risco	Risco	Plano de redução de probabilidade
t6	A tecnologia de comando de voz ainda não é bem desenvolvida.	Comandos de voz podem não ser corretamente entendidos.	Pesquisar aplicativos operados por tecnologia de voz e testá-los. Verificar existência de módulos reusáveis de comando por voz.
t9	Não existem ferramentas CASE com gráficos 3D ou em níveis de profundidade.	Não existe um padrão ou referência para tais interfaces, nem estudos de usabilidade.	Catalogar e estudar ferramentas semelhantes com interfaces 3D ou em níveis.
t3	Não é conhecido um padrão de usabilidade para CASE em touch screen.	Poderá ser desenvolvida uma ferramenta com usabilidade falha.	Catalogar e estudar ferramentas semelhantes já desenvolvidas para superfícies de toque. Estudar normas de usabilidade em geral e normas específicas para ferramentas CASE e para sistemas baseados em superfície de toque.
pe1	O projeto será desenvolvido com bolsistas.	Bolsistas não veem o projeto como carreira.	Verificar o valor de salário de mercado. Verificar a possibilidade de oferecer salários mais atraentes. Verificar a possibilidade de subcontratar desenvolvimento.
l1	Uso de tecnologia de terceiros.	Pagamento de direitos autorais.	Verificar valores e condições de uso de potenciais tecnologias. Verificar existência de soluções livres.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#8.5.1. Plano de Redução de Probabilidade de Risco

(Fonte: RAUL, Wazlawick - Engenharia de Software)

- Deve-se observar que os planos não necessariamente garantem que os riscos vão ser mitigados. Por exemplo, o plano para o risco l1 prevê verificar a existência de soluções livres. Mas não há garantia de que elas existam.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#8.5.1. Plano de Redução de Probabilidade de Risco

(Fonte: RAUL, Wazlawick - Engenharia de Software)

- O *plano de redução de impacto de risco* é definido e aplicado de forma semelhante ao plano de redução de probabilidade, exceto pelo fato de que, nesse caso, as ações devem procurar diminuir o impacto do risco. Assim, tais planos vão procurar diminuir os *efeitos* desse risco, e não suas causas.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#8.5.1. Plano de Redução de Probabilidade de Risco

(Fonte: RAUL, Wazlawick - Engenharia de Software)

- Por exemplo, se determinado membro da equipe é o único a dominar a tecnologia a ser usada no projeto, sua saída pode causar grande impacto no andamento do projeto. Assim, um plano de redução de impacto poderia consistir em treinar outro desenvolvedor nessa tecnologia ou colocá-lo para trabalhar em conjunto com o desenvolvedor crítico para que possa aprender o máximo possível sobre a tecnologia. Assim, no caso da falta do outro, o prejuízo ao projeto não será tão grande.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#8.5.1. Plano de Redução de Probabilidade de Risco

(Fonte: RAUL, Wazlawick - Engenharia de Software)

- Novamente, os riscos de alta exposição determinam a existência de um plano detalhado de redução de impacto com atividades previstas, prazos e responsáveis. Riscos de média exposição poderão receber apenas uma lista de ações a serem efetuadas em caso de necessidade, sem necessariamente identificar responsáveis ou prazos, e os riscos de baixa exposição serão apenas monitorados.
- Considerando o exemplo da Tabela 8.6, os planos de redução de probabilidade poderiam ser elaborados como mostrado na Tabela 8.8.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

8.5.2. Plano de Redução de Impacto de Risco -Tabela 8.8 - (Fonte: RAUL, Wazlawick - Engenharia de Software)

Id	Risco	Efeito	Plano de redução de impacto
pr3	Pode haver mudanças importantes nos requisitos ao longo do desenvolvimento.	Perda de tempo desenvolvendo partes que depois não serão usadas e atrasos no cronograma.	Enfatizar desenvolvimento modular com baixo acoplamento entre módulos. Estabilizar arquitetura-base o quanto antes. Implementar um sistema eficiente de gerenciamento de versões.
pr2	Pode haver concorrentes que lancem produtos antes.	Chegar ao mercado depois da janela de oportunidade.	Manter constante estudo de mercado para garantir que o produto tenha características inovadoras.
t8	Gestos muito parecidos podem significar comandos diferentes.	O sistema pode interpretar erroneamente os comandos (desenhos, formas). Usuário pode ter que decorar muitos comandos diferentes.	Elaborar design de interface alternativo, considerando gestos e alguma forma de eliminar possíveis ambiguidades em gestos. Implementar sistema de ajuda on-line para gestos.
pe1	Necessidade de treinamento.	Atrasos de cronograma e custos com treinamento.	Pesquisar e encomendar bibliografia para treinamento de equipe nas tecnologias necessárias. Prever orçamento para treinamento.
t4	O usuário não vai escolher a ferramenta porque usa um processo de desenvolvimento diferente.	Problemas relacionados com a venda. Pode haver necessidade de implementar vários processos, o que vai contra a filosofia inicial da ferramenta. Grandes empresas já têm processo estabelecido e teriam que mudar.	Verificar se existe possibilidade de definir um metaprocessos adaptável para a ferramenta.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

8.5.2. Plano de Redução de Impacto de Risco -Tabela 8.8 - (Fonte: RAUL, Wazlawick - Engenharia de Software)

Id	Risco	Efeito	Plano de redução de impacto
t6	Comandos de voz podem não ser entendidos corretamente.	Usuários frustrados.	Projetar interfaces alternativas a comandos de voz.
t9	Não existe um padrão ou referência para tais interfaces, nem estudos de usabilidade.	Necessidade de pesquisar padrões de usabilidade para interfaces 3D em ferramentas CASE.	Prever a realização de testes de usabilidade para a ferramenta. Prever ciclos de prototipação de interface.
t3	Poderá ser desenvolvida uma ferramenta com usabilidade falha.	Problemas com usuário final (desinteresse).	Idem ao risco t9.
pe1	Bolsistas não veem o projeto como carreira.	Podem-se perder desenvolvedores ao longo do projeto, necessitando de substituição.	Usar programação em pares e padrões de codificação. Planejar integrações frequentes e posse coletiva de código.
l1	Pagamento de direitos autorais.	Aumento de custo.	Prever custos com direitos autorais no orçamento. Verificar existência de tecnologias livres.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#8.5.1. Plano de Redução de Probabilidade de Risco

(Fonte: RAUL, Wazlawick - Engenharia de Software)

- Os planos de redução de probabilidade e impacto mostrados aqui são apenas sugestões elaboradas a partir de uma rápida reflexão sobre os riscos. Outros planos mais detalhados, ou mesmo em direções diferentes, poderiam ter sido elaborados.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#8.5.1. Plano de Redução de Probabilidade de Risco

(Fonte: RAUL, Wazlawick - Engenharia de Software)

- Por vezes, também pode haver certa ambiguidade entre o plano ser de redução de impacto ou de probabilidade, isto é, pode haver determinadas ações que reduzam tanto a probabilidade quanto o impacto de um risco. Se o planejador de projeto começar a ter problemas em classificar um plano como de um ou outro tipo, será mais interessante que ele crie apenas um conjunto de planos de mitigação de risco, pois sua classificação como redução de probabilidade ou redução de impacto não é tão importante quanto a necessidade de que seja efetivamente planejado e executado, se for o caso.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Plano de Gerência de Riscos
- (Fonte: RAUL, Wazlawick - Engenharia de Software)

-Como criá-lo e executá-lo?

-O que devo fazer primeiro?

-Preciso entender melhor cada etapa!

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **#Plano de Gerência de Riscos**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software)**
- Uma característica altamente desejada para um planejador, e mesmo para um gerente de projeto, em relação ao risco é a capacidade de visão antecipada.
- Um bom planejador e um bom gerente são capazes de visualizar com antecedência possíveis situações anômalas que poderiam impedir o bom andamento do projeto. Essa previsão, longe de ser uma atitude pessimista, poderá salvar o projeto no futuro ou pelo menos mantê-lo dentro do cronograma e do orçamento previstos.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- #Identificação de Riscos
- (Fonte: RAUL, Wazlawick - Engenharia de Software)
- Ações importantes:
 - Reuniões e *brainstormings* com gerente e equipe de projeto;
 - Análise de cenários;
 - Análise em projetos anteriores com contexto semelhante;
 - Identificação de riscos deve considerar diferentes fontes: tais como tecnologia, pessoas e o próprio projeto.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **#Riscos Tecnológicos**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software)**
- *Os riscos tecnológicos* estão relacionados com todas as incertezas referentes ao modo como a equipe será capaz de lidar com a tecnologia necessária para realizar o projeto. Quanto menos maturidade a equipe tiver com essas tecnologias, maiores serão os riscos.
- Projetos que envolvam diferentes sistemas de software e hardware enfrentarão problemas de compatibilidade frequentes. Tornar tais sistemas compatíveis demandará tempo e custo extras ao projeto. Assim, o planejador deve saber se diferentes tecnologias terão de interagir e qual é a maturidade da equipe com esse tipo de integração.
- Outro ponto que pode oferecer risco tecnológico a um projeto é a questão da absorção de inovação tecnológica dos usuários.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **#Riscos Relacionados com Pessoas**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software)**
- *Riscos de pessoal*: projetos são executados por pessoas. Então, perder uma pessoa da equipe de forma permanente ou temporária pode ter um grande impacto na medida em que essa pessoa for “insubstituível”.
- *Riscos de cliente*: até que ponto o cliente se manterá interessado no projeto? Mudanças políticas ou administrativas poderão afetar o interesse da organização em investir no projeto? Mesmo que a organização mantenha o interesse no projeto, o cliente estará disponível para esclarecer requisitos e realizar testes?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **#Riscos Relacionados com a Empresa**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software)**
- *Riscos de negócio*: muitas vezes, a empresa até constrói um bom produto no prazo e com o custo definidos, mas mesmo assim o projeto fracassa. Entre outras coisas, a organização pode não ter a habilidade necessária para vender o produto, ou a empresa pode até ter essa habilidade, mas o produto não apresenta efetivamente apelo comercial.
- *Riscos legais*: existem problemas ou possibilidade de litígio? Uso de material protegido por direitos autorais? Necessidade de celebração de contrato com terceiros? Normas e leis específicas em outros estados ou países?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **#Riscos de Projeto**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software)**
- Os riscos de gerenciamento do projeto envolvem a capacidade da equipe de planejar e seguir o plano dentro do cronograma e do custo previsto. Esse tipo de risco costuma se manifestar das seguintes formas:
 - *Riscos de requisitos*: a equipe, pode não ter sido capaz de identificar corretamente os requisitos do projeto, o que causará problemas ao longo dele. Requisitos poderão ser insuficientes, excessivos ou incorretos. Ou, ainda, os requisitos podem ser naturalmente instáveis, em razão de características do próprio projeto.
 - *Riscos de processo*: o modelo de processo escolhido é adequado às características do projeto? A equipe tem maturidade com o processo? O gerente tem maturidade em projetos anteriores?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **#Riscos de Projeto**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software)**
- • *Riscos de orçamento*: até que ponto a verba necessária para o projeto está garantida? Os custos foram corretamente previstos em projetos anteriores?
- • *Riscos de cronograma*: é possível que os prazos sejam alterados e a ordem em que as funcionalidades devem ser entregues possa mudar? O planejador deve saber em que grau a equipe se mostrou capaz de ater-se ao cronograma em projetos passados.
- A inexistência de qualquer uma dessas informações caracteriza um risco importante ao projeto na medida da sua incerteza.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- **#Riscos de Projeto**
- **(Fonte: RAUL, Wazlawick - Engenharia de Software)**
- O Gerente de Projetos deve preparar e aplicar inúmeras perguntas;
- Para estas perguntas, obrigatoriamente, devemos ter as respostas;
- Então, tenha atenção redobrada para formular tais perguntas e abstrair as respostas.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Diversos – capítulo 8 (8.1 a 8.5)

(Fonte: RAUL, Wazlawick - Engenharia de Software)

Requisitos: Estabilidade: os requisitos podem mudar durante o desenvolvimento?

Requisitos: Completeza: estão faltando requisitos ou estão especificados de forma incompleta?

Requisitos: Clareza: os requisitos estão obscuros ou necessitam de interpretação?

Requisitos: Validade: os requisitos vão levar ao produto que o cliente tem em mente?

Requisitos: Exequibilidade: os requisitos são exequíveis de um ponto de vista analítico?

Requisitos: Precedentes: os requisitos especificam algo que nunca foi feito antes, ou que a empresa nunca fez antes?

Requisitos: Escala: os requisitos especificam um produto maior, mais complexo ou requerendo mais organização do que a empresa tem experiência?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Diversos – capítulo 8 (8.1 a 8.5)

(Fonte: RAUL, Wazlawick - Engenharia de Software)

Design: Funcionalidade: existem problemas potenciais para obter os requisitos funcionais?

Design: Dificuldade: o design ou implementação serão difíceis de serem realizados?

Design: Interfaces: as interfaces internas de hardware e software são bem definidas e controladas?

Design: Performance: existem tempos de resposta ou taxas de transferência rigorosos?

Design: Testabilidade: o produto é difícil ou impossível de testar?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Diversos – capítulo 8 (8.1 a 8.5)

(Fonte: RAUL, Wazlawick - Engenharia de Software)

Restrições de hardware: existem restrições apertadas no hardware alvo?

Software não desenvolvido: há problemas com software usado, mas não desenvolvido pela equipe?

Codificação: Exequibilidade: a implementação do design é difícil ou impossível?

Codificação: Teste: os níveis e tempos especificados para os testes de unidade são adequados?

Codificação: Codificação/Implementação: há problemas com codificação e implementação?

Integração: Ambiente: o ambiente de integração e teste é adequado?

Integração: Produção: a definição de interfaces e instalações são inadequadas ou o tempo insuficiente?

Integração: Sistema: a integração de sistema é descoordenada, a definição de interface é pobre ou as instalações são inadequadas?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Diversos – capítulo 8 (8.1 a 8.5)

(Fonte: RAUL, Wazlawick - Engenharia de Software)

Atributos: Manutenibilidade: a implementação será difícil de entender e manter?

Atributos: Confiabilidade: os requisitos de confiabilidade ou disponibilidade são difíceis de obter?

Atributos: Riscos à segurança (safety): os requisitos relacionados com riscos à segurança são inexecutáveis ou não demonstráveis?

Atributos: Segurança do sistema (security): os requisitos de segurança do sistema são mais rigorosos do que o usual?

Atributos: Fatores humanos: o sistema será difícil de usar por conta de interface com usuário mal definida?

Atributos: Especificações: a documentação é adequada para o design, implementação e teste do sistema?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Diversos – capítulo 8 (8.1 a 8.5)

(Fonte: RAUL, Wazlawick - Engenharia de Software)

Processo de desenvolvimento: Formalidade: a implementação será difícil de entender e manter?

Processo de desenvolvimento: Adequação: o processo é adequado para o modelo de desenvolvimento?

Processo de desenvolvimento: Controle de processo: o processo de desenvolvimento é executado, monitorado e controlado com métricas? Os locais de desenvolvimento distribuídos são coordenados?

Processo de desenvolvimento: Familiaridade: os membros do projeto têm experiência no uso do processo? O processo é compreendido por toda a equipe?

Processo de desenvolvimento: Controle de produto: há mecanismos para controlar a mudança no produto?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Diversos – capítulo 8 (8.1 a 8.5)

(Fonte: RAUL, Wazlawick - Engenharia de Software)

Sistema de desenvolvimento: Capacidade: existe poder de processamento, estações de trabalho, memória e espaço de armazenamento suficientes?

Sistema de desenvolvimento: Adequação: o sistema de desenvolvimento dá suporte a todas as fases, atividades e funções?

Sistema de desenvolvimento: Usabilidade: qual a facilidade de uso do sistema de desenvolvimento?

Sistema de desenvolvimento: Familiaridade: existe pouca experiência anterior da empresa ou membros da equipe com o sistema de desenvolvimento?

Sistema de desenvolvimento: Confiabilidade: o sistema de desenvolvimento sofre com erros, paralisação e capacidade de backup nativa insuficiente?

Sistema de desenvolvimento: Suporte ao sistema: existe suporte ágil de vendedor ou especialista no sistema de desenvolvimento?

Sistema de desenvolvimento: Capacidade de entrega (deliverability): os requisitos de definição e aceitação para a

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Diversos – capítulo 8 (8.1 a 8.5)

(Fonte: RAUL, Wazlawick - Engenharia de Software)

Processo de gerência: Planejamento: o planejamento é oportuno, líderes técnicos foram incluídos e planos de contingência realizados?

Processo de gerência: Organização de projeto: os papéis e relacionamentos estão claros?

Processo de gerência: Experiência em gerência: os gerentes têm experiência em desenvolvimento de software, gerenciamento de projeto de software, no domínio de aplicação, no processo de desenvolvimento ou em grandes projetos?

Processo de gerência: Interfaces da equipe de desenvolvimento: existem interfaces fracas com o cliente, outros contratados ou gerentes de nível superior?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Diversos – capítulo 8 (8.1 a 8.5)

(Fonte: RAUL, Wazlawick - Engenharia de Software)

Métodos de gerência: Monitoramento: as métricas de gerência são definidas e o progresso do desenvolvimento rastreado?

Métodos de gerência: Gerenciamento de pessoal: o pessoal do projeto é treinado e usado adequadamente?

Métodos de gerência: Garantia de qualidade: existem procedimentos e recursos adequados para garantir a qualidade do produto?

Métodos de gerência: Gerenciamento de configuração: os procedimentos de gerenciamento de configuração, mudança e instalação são adequados?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Diversos – capítulo 8 (8.1 a 8.5)

(Fonte: RAUL, Wazlawick - Engenharia de Software)

Ambiente de trabalho: Atitude em relação a qualidade: há falta de orientação relacionada com o trabalho com qualidade?

Ambiente de trabalho: Cooperação: falta espírito de equipe? Os conflitos resultantes requerem intervenção da gerência?

Ambiente de trabalho: Comunicação: há pouca consciência a respeito de metas ou missão e comunicação pobre de informação técnica entre membros da equipe e gerentes?

Ambiente de trabalho: Moral: há uma atmosfera não produtiva ou não criativa?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Diversos – capítulo 8 (8.1 a 8.5)

(Fonte: RAUL, Wazlawick - Engenharia de Software)

Recursos: Cronograma: o cronograma é inadequado ou instável?

Recursos: Pessoal: a equipe é inexperiente, sem conhecimento de domínio, sem habilidades ou em número insuficiente?

Recursos: Orçamento: o orçamento é insuficiente ou instável?

Recursos: Instalações: as instalações são adequadas para construir e entregar o produto?

Contrato: Tipo de contrato: o tipo de contrato é uma fonte de risco para o projeto?

Contrato: Restrições: o contrato causa alguma restrição?

Contrato: Dependências: o projeto tem dependência de algum produto ou serviço externo que pode afetar o produto?

Comunicação externa: Cliente: há problemas com o cliente, como ciclo longo de aprovação de documentos, comunicação fraca ou conhecimento inadequado do domínio?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Diversos – capítulo 8 (8.1 a 8.5)

(Fonte: RAUL, Wazlawick - Engenharia de Software)

Probabilidade: é a chance de que o risco realmente se torne um problema.

Impacto: o impacto é a medida do prejuízo que um risco pode trazer ao projeto.

Escala de Riscos: Alta – ação imediata

Média – ação logo após a resolução dos riscos de nível Alta

Baixa – monitoramento constante

Probabilidade				
		Alta	Média	Baixa
Impacto	Alto	Alta exposição	Alta exposição	Média exposição
	Médio	Alta exposição	Média exposição	Baixa exposição
	Baixo	Média exposição	Baixa exposição	Baixa exposição

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Diversos – capítulo 8 (8.1 a 8.5)

(Fonte: RAUL, Wazlawick - Engenharia de Software)

Planos de mitigação de riscos são executados antes que o risco ocorra.

- Para os riscos de alta exposição, os planos de mitigação são definidos ainda na fase de planejamento do projeto.
- Para riscos de média exposição, os planos são definidos e guardados para serem aplicados caso essa exposição aumente ao longo do projeto.
- Há dois tipos de planos de mitigação: *plano de redução de probabilidade* e *plano de redução de impacto*.
- O primeiro age nas causas e o outro age nos efeitos do risco.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- Tarefa da aula de hoje
 - Preencha as 03 (três) tabelas baseando-se no seu Trabalho de Conclusão de Curso.
 - É obrigatório citar o mínimo de 05 (cinco) riscos e as devidas ações em cada tabela.
 - Submeter em um único arquivo na plataforma AVA (hoje até 23h59)

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Planos de Mitigação de Riscos

(Fonte: RAUL, Wazlawick - Engenharia de Software)

Id	Causa	Risco	Efeito	Prob.	Imp.	Exp.

O plano de redução de probabilidade de risco consiste nas ações identificadas como necessárias para diminuir a probabilidade de que um risco ocorra. Esse tipo de plano deve agir nas *causas* do risco.

Id	Causa do risco	Risco	Plano de redução de probabilidade

O plano de redução de impacto de risco é definido e aplicado de forma semelhante ao plano de redução de probabilidade, exceto pelo fato de que, nesse caso, as ações devem procurar diminuir o impacto do risco.

Id	Risco	Efeito	Plano de redução de impacto

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- A ISO 9001 é uma norma que tem como principal objetivo ajudar o gestor de uma organização de qualquer setor e qualquer tamanho a otimizar seus processos, de forma a torná-los mais ágeis, eliminando gargalos e ineficiências e, desta forma, satisfazendo melhor o cliente.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- Com a aplicação da norma, a melhoria na gestão da organização vai ocorrer a partir dos resultados a seguir:
 - Identificação do contexto no qual a organização está inserida.
 - Com o uso da abordagem de processos é definida uma visão integral da organização.
 - São identificados os riscos que podem prejudicar o andamento dos trabalhos.
 - Os resultados do desempenho e eficácia dos processos são medidos e avaliados.
 - Garante-se que os próprios processos são continuamente melhorados.
 - A satisfação dos clientes é monitorada.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- Para que uma empresa seja certificada pela ISO 9001 ela deve passar por uma auditoria de certificação que é feita por certificadores reconhecidos pelo IAF (International Accreditation Forum). No Brasil, o representante do IAF é o INMETRO (Instituto Nacional de Metrologia, Normalização e Qualidade Industrial).
- O processo de certificação ocorre em três etapas. Na primeira etapa a organização deve, com base nas orientações da norma, implementar os processos e artefatos necessários e recomendados. Na segunda etapa, a organização realiza um processo de auditoria interna para verificar a necessidade de eventuais ajustes. Já na terceira etapa, a organização contrata a auditoria externa do certificador reconhecido que fará uma nova auditoria e, se a organização passar, emitirá o selo de certificação com validade de três anos.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

A tabela apresenta apenas alguns aspectos da norma, já que ela é bastante extensa.

Aspecto	Requisito
Controle do projeto	Todas as atividades relacionadas com o projeto devem ser documentadas.
Controle de documentos	Deve haver procedimentos para controlar a produção, distribuição e atualização de documentos relevantes aos projetos.
Sistema de qualidade	As políticas, regras e processos relacionados com a qualidade devem ser documentados na forma de um manual e deve, também, haver evidências de sua implementação na prática.
Controle de não conformidade	Deve haver mecanismos para evitar que um produto que esteja em não conformidade com os requisitos seja colocado em uso.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

A tabela apresenta apenas alguns aspectos da norma, já que ela é bastante extensa.

Aspecto	Requisito
Registro de qualidade	Devem ser mantidos os registros de qualidade produzidos ao longo do processo.
Auditora interna	Deve haver um sistema de avaliação interna do programa de qualidade.
Identificação e rastreabilidade do produto	Todo produto deve ser passível de identificação ao longo de todo o processo de produção e mesmo depois de colocado em uso.
Ação corretiva	Deve haver mecanismos que permitam não apenas corrigir as não conformidades, mas também descobrir suas causas e, se possível, evitar que ocorram novamente.
Responsabilidade da direção	A administração deve indicar um responsável pelo sistema de qualidade da organização. Entre suas atribuições estão: garantir que a política de qualidade seja definida, documentada, comunicada, implementada e atualizada.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- A ISO 9000-3 (KEHOE & JARVIS, 1996), é compreendida como uma das primeiras tentativas de melhorar o processo de produção de software, consistindo em um guia para aplicação da ISO 9001 à indústria de software. A ISO 9000-3 apresentava padrões para gerenciamento e garantia de qualidade aplicáveis a companhias de qualquer tamanho.
- Um dos problemas com a antiga ISO 9000-3 é que ela não tratava a melhoria contínua do processo, apenas indicava os processos que as empresas deveriam ter e manter.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- A ISO 9000-3 relaciona-se com um conjunto de normas ISO que dizem respeito aos aspectos de qualidade de processo de software. Ela considera que o processo de produção de software é variado, podendo ser dirigido por diferentes modelos, mas ao mesmo tempo considera que determinadas fases ou disciplinas existirão com maior ou menor ênfase ou, ainda, com diferentes formas de organização em qualquer processo de produção.
- Não existe certificação em ISO 9000-3. Como ela é um guia para aplicação da ISO 9001 na indústria de software, então a certificação é em ISO 9001.
- A norma contém dois tipos de cláusulas: requisitos e orientações. Essas cláusulas são relacionadas com aspectos sistêmicos, corretivos, de recursos, de gerenciamento e de realização.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- Requisitos são as coisas que a empresa deve fazer para ser certificada pela ISO 9001.
- Já as orientações na 9000-3 são específicas à indústria de software.
- Em relação às orientações, elas ainda se dividem em dois grupos: recomendações e sugestões. Recomendações são as coisas que a empresa deve fazer para satisfazer os requisitos da 9001. Já as sugestões são coisas que a empresa poderia fazer para ficar mais confortavelmente dentro dos limites dos requisitos da 9001.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- Outro conjunto de normas que foram muito utilizadas até grande parte delas ser suplantada em 2015 é a família ISO/IEC 15504. Ela foi criada como uma complementação para a ISO/IEC 12207 (definição de processos do ciclo de vida de desenvolvimento de software) e tinha como objetivo orientar a avaliação e a auto avaliação da capacidade de empresas em processos e, a partir dessa avaliação, permitir a melhoria dos processos. A partir de 2015 esse conjunto de normas passou a ser apoiada pela família ISO 330xx.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

A família 330xx (normas 33001, 33003, 33020 e 33061) pode ser utilizada para avaliar a qualidade dos processos realizados na organização, ou seja, as propriedades de processos tais como segurança, eficiência, eficácia, confidencialidade, integridade e sustentabilidade. Além disso, ela também pode ser usada para avaliar a capacidade de processos, assim como era feito com a ISO 15504, ou seja, classificando cada processo da organização em um dos seis níveis estabelecidos.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

Nível	Nome	Descrição
0	Incompleto	Este nível representa uma falha geral em se ater aos objetivos de um processo, ou ausência de processo. Não há produtos e saídas facilmente identificáveis para o processo sendo avaliado. Pode ser atribuído quando o processo não é implementado, ou quando é implementado, mas falha em atingir seus objetivos.
1	Realizado	Neste nível, o propósito do processo geralmente é obtido, mas não necessariamente de forma planejada ou rastreável.
2	Gerenciado	Neste nível, os projetos entregam produtos com qualidade aceitável dentro dos prazos e orçamento definidos. A execução dos projetos de acordo com a definição dos processos é realizada e rastreável.
3	Estabelecido	Neste nível a própria gerência dos projetos deve ser realizada de acordo com um processo estabelecido, no qual bons princípios de engenharia de software são empregados. Este nível, ao contrário do anterior, necessita de um gerenciamento planejado e utilizando um processo-padrão.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

Nível	Nome	Descrição
4	Previsível	Neste nível, os projetos são realizados de forma consistente dentro de limites de controle. Medidas de performance detalhadas são coletadas e analisadas, o que leva a uma compreensão quantitativa da capacidade do processo e a uma melhor habilidade de prever performances futuras. Neste caso, a performance é gerenciada de forma objetiva e a qualidade do trabalho é quantitativamente conhecida.
5	Inovando	Neste nível, a realização do processo é otimizada para satisfazer necessidades correntes e futuras do negócio, e os processos repetidamente satisfazem estas necessidades. Metas quantitativas de eficiência e efetividade para processos são estabelecidas. O monitoramento contínuo e eficaz permite a melhoria contínua do processo a partir da análise de resultados. Administrar um processo envolve inovação, com a incorporação constante de novas ideias e tecnologias, bem como a modificação de processos ineficientes ou não efetivos.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

O modelo de avaliação estabelecido na ISO 33001 se estrutura em duas dimensões, uma dimensão de processos, que indica quais os processos avaliados e uma dimensão de capacidade que indica a avaliação que a organização recebeu (notas de 0 a 5) em cada processo.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- A dimensão de processos é dividida em dois grandes grupos:
 - Processos de ciclo de vida de sistema
 - AGR – Processos de acordo
 - ORG – Processos organizacionais de viabilização de projeto
 - PRO – Processos de projeto
 - ENG – Processos técnicos

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- Processos de ciclo de vida de sistema

Categoria	Processo	Subprocesso
AGR – Processos de acordo	AGR.1 Aquisição	AGR.1A Preparação de aquisição
		AGR.1B Seleção de fornecedor
		AGR1.C Monitoramento de acordo
		AGR1.D Aceitação de adquirente
	AGR.2 Fornecimento	AGR.2A Licitação de fornecedores
		AGR.2B Fechamento de contrato
	AGR.3 Gerenciamento de mudança de contrato	AGR.2C Entrega e suporte de produto ou serviço

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- Processos de ciclo de vida de sistema

Categoria	Processo	Subprocesso
ORG – Processos organizacionais de viabilização de projeto	ORG.1 Gerenciamento de modelo de ciclo de vida	ORG.1A Estabelecimento de processo
		ORG.1B Avaliação de processo
		ORG.1C Melhoria de processo
	ORG.2 Gerenciamento de infraestrutura	
	ORG.3 Gerenciamento de portfólio de projeto	
	ORG.4 Gerenciamento de recursos humanos	ORG.4A Desenvolvimento de habilidades
		ORG.4B Aquisição e provimento de habilidades
		ORG.4C Gerenciamento de conhecimento
	ORG.5 Gerenciamento de qualidade	
	ORG.6 Alinhamento organizacional	
	ORG.7 Gerenciamento organizacional	

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- Processos de ciclo de vida de sistema

Categoria	Processo	Subprocesso
PRO – Processos de projeto	PRO.1 Planejamento de projeto	
	PRO.2 Avaliação e controle de projeto	
	PRO.3 Gerenciamento de decisão	
	PRO.4 Gerenciamento de risco	
	PRO.5 Gerenciamento de configuração	
	PRO.6 Gerenciamento de informação	
	PRO.7 Medição	

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- Processos de ciclo de vida de sistema

Categoria	Processo	Subprocesso
ENG – Processos técnicos	ENG.1 Definição de requisitos de interessado	
	ENG.2 Análise de requisitos de sistema	
	ENG.3 Design arquitetural do sistema	
	ENG.4 Implementação de software	
	ENG.5 Integração de sistema	
	ENG.6 Teste de qualificação do sistema	
	ENG.7 Instalação de software	
	ENG.8 Suporte à aceitação de software	
	ENG.9 Operação de software	ENG9.A Uso operacional
		ENG9.B Suporte ao usuário
	ENG.10 Manutenção de software	
	ENG.11 Eliminação de software	

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1).

- A dimensão de processos é dividida em dois grandes grupos:
 - Processos de ciclo de vida de software
 - DEV – Processos de implementação de software
 - SUP – Processos de suporte de software
 - REU – Processos de reuso de software

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- Processos de ciclo de vida de software

Categoria	Processo
DEV – Processos de implementação de software	DEV.1 Análise de requisitos de software
	DEV.2 Design arquitetural de software
	DEV.3 Design detalhado de software
	DEV.4 Construção de software
	DEV.5 Integração de software
	DEV.6 Teste de qualificação de software

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- Processos de ciclo de vida de software

Categoria	Processo
SUP – Processos de suporte de software	SUP.1 Gerenciamento de documentação de software
	SUP.2 Gerenciamento de configuração de software
	SUP.3 Garantia de qualidade de software
	SUP.4 Verificação de software
	SUP.5 Validação de software
	SUP.6 Revisão de software
	SUP.7 Auditoria de software
	SUP.8 Resolução de problemas com software

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- Processos de ciclo de vida de software

REU – Processos de reuso de software	REU.1 Engenharia de domínio
	REU.2 Gerenciamento de ativos reusáveis
	REU.3 Gerenciamento de programas reusáveis

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- Processos suplementares da ISO 33061

Categoria	Processo
QNT – Processos de quantificação	QNT.1 Melhoria de processo quantitativa
	QNT.2 Gerenciamento de desempenho quantitativo
SUP – Processos de suporte de software	SUP.9 Gerenciamento de solicitações de mudança em software
SPL – Processos de entrega de produto	SPL.1D Entrega de produto (<i>release</i>)
	SPL.1E Suporte de aceitação de produto

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- Atributos dos níveis de processo de acordo com a ISO 33020

Nível	Atributo	Resultados esperados
1 – Realizado	PA 1.1 Desempenho de processo	O processo obtém as saídas desejadas.
2 – Gerenciado	PA 2.1 Gerenciamento de desempenho	Objetivos de desempenho de processo são identificados.
		O desempenho do processo é planejado.
		O desempenho do processo é monitorado.
		O desempenho é ajustado para atender aos planos.
		Responsabilidades e autoridade para realizar o processo são definidas, atribuídas e comunicadas.
		O pessoal que realiza o processo está preparado para executar suas responsabilidades.
		Recursos e informação para realizar o processo são identificados, disponibilizados, alocados e usados.
		Interfaces entre as partes envolvidas são gerenciadas para garantir efetiva comunicação e clara atribuição de responsabilidade.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- Atributos dos níveis de processo de acordo com a ISO 33020

Nível	Atributo	Resultados esperados
2 – Gerenciado	PA 2.2 Gerenciamento do produto do trabalho	Requisitos para o produto do trabalho do processo são definidos.
		Requisitos para a documentação e controle do produto do trabalho do processo são definidos.
		O produto do trabalho é claramente identificado, documentado e controlado.
		O produto do trabalho é revisado de acordo com o planejado e é ajustado se necessário para atender aos requisitos.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- **Atributos dos níveis de processo de acordo com a ISO 33020**

3 – Estabelecido	PA 3.1 – Definição de processo	Um processo-padrão, incluindo diretrizes especializadas é definido e mantido, descrevendo os elementos fundamentais que devem ser incorporados a um processo definido.
		A sequência e interação do processo-padrão com outros processos é determinada.
		Competências requeridas e papéis para realizar o processo são identificados como parte do processo-padrão.
		Infraestrutura requerida e ambiente de trabalho para realizar o processo são identificados como parte do processo-padrão.
		Métodos e métricas adequadas para monitoramento da efetividade e adequação do processo são determinados.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- **Atributos dos níveis de processo de acordo com a ISO 33020**

3 – Estabelecido	PA 3.2 – Implantação de processo	Um processo definido é implantado baseado em um processo apropriadamente selecionado e/ou personalizado.
		Papéis requeridos, responsabilidades e autoridade para realizar os processos definidos são atribuídos e comunicados.
		As pessoas que realizam o processo são competentes com base em educação, treinamento e experiência apropriadas.
		Os recursos e informações necessários para realizar os processos definidos são disponibilizados, alocados e usados.
		A infraestrutura e ambiente de trabalho requeridos para realizar os processos definidos são disponibilizados gerenciados e mantidos.
		Dados apropriados são coletados e analisados como base para o entendimento do comportamento do processo para demonstrar a adequação e efetividade do processo e para avaliar onde a melhoria contínua do processo pode ser feita

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- **Atributos dos níveis de processo de acordo com a ISO 33020**

4 - Previsível	PA 4.1 – Análise quantitativa	O processo está alinhado com metas de negócio quantitativas.
		As necessidades de informação de processo em apoio aos objetivos de negócio quantitativos definidos são estabelecidas.
		Objetivos de mensuração do processo são derivados das necessidades de informação do processo.
		Relações mensuráveis entre elementos de processo que contribuem para o desempenho do processo são identificados.
		Objetivos quantitativos para desempenho de processo no suporte a metas de negócio relevantes são estabelecidos.
		Medições e frequência de medições apropriadas são identificadas e definidas alinhadas aos objetivos de medição de processo e objetivos quantitativos de desempenho de processo.
		Resultados de medições são coletados, validados e reportados com o objetivo de monitorar a extensão na qual os objetivos quantitativos do processo foram obtidos.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- **Atributos dos níveis de processo de acordo com a ISO 33020**

4 - Previsível	4.2 Controle quantitativo	Técnicas para analisar os dados coletados são selecionadas.
		Possíveis causas de variação em processos são determinadas a partir da análise dos dados coletados.
		Distribuições que caracterizam o desempenho do processo são estabelecidas.
		Ações corretivas são tomadas para tratar possíveis causas de variação.
		Distribuições distintas são estabelecidas (se necessário) para analisar o processo sob a influência das possíveis causas de variação.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- **Atributos dos níveis de processo de acordo com a ISO 33020**

5 - Inovando	PA 5.1 – Inovação de processo	Objetivos de inovação de processo são definidos para suportar as metas relevantes de processo.
		Dados apropriados são analisados para identificar oportunidades de inovação.
		Oportunidades de inovação derivadas de novas tecnologias e conceitos de processo são identificadas.
		Uma estratégia de implementação é estabelecida para obter os objetivos de inovação de processo.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- **Atributos dos níveis de processo de acordo com a ISO 33020**

5 - Inovando	PA 5.2 – Implementação de inovação de processo	O impacto de todas as mudanças propostas é avaliado contra os objetivos do processo definido e do processo-padrão.
		A implementação de todas as mudanças acordadas é gerenciada para garantir que qualquer ruptura no desempenho do processo seja entendida e tratada.
		A efetividade da mudança de processo com base no desempenho atual é avaliada contra os requisitos de produto definidos e objetivos do processo.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- A norma 33061 apresenta um guia para avaliação dos processos de uma empresa. Uma avaliação pode ser feita, basicamente, com dois objetivos:
 - Determinar capacidade: uma organização que deseja terceirizar a produção de software pode querer saber ou avaliar a capacidade de potenciais fornecedores em diferentes áreas de processo.
 - Melhoria de processo: uma organização que desenvolve software pode querer melhorar seus próprios processos. A norma possibilita avaliar o estado atual dos processos da empresa e, após as atividades de melhoria, avaliar se houve avanço.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- O modelo de avaliação usado inclui a dimensão de processos, conforme apresentado nas tabelas. O processo de avaliação definido na norma inclui as seguintes atividades:
 - Iniciar a avaliação por parte do interessado.
 - Selecionar o avaliador e sua equipe.
 - Planejar a avaliação, selecionando os processos que serão avaliados de acordo com o modelo e a demanda do interessado.
 - Reunião de pré-avaliação.
 - Coletar dados.
 - Validar dados.
 - Atribuir nível de capacidade aos processos.
 - Relatar os resultados da avaliação.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- O avaliador pode coletar dados de diversas maneiras, incluindo entrevistas, análise de dados estatísticos e registros de qualidade. Usualmente, a falta de registros confiáveis não é um fator positivo para a atribuição de níveis de capacidade em processos. Porém, mesmo quando os registros existem, o avaliador deve validá-los, isto é, certificar-se de que são corretos. Isso, usualmente, pode ser feito através das entrevistas.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- E na prática?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- E na prática?
- A partir de sua maturidade e das diretrizes de avaliação, o avaliador vai, então, atribuir um nível de obtenção na escala de quatro ou de seis valores a cada um dos atributos de processo, iniciando dos atributos dos níveis mais baixos e subindo. À medida que todos os atributos de um nível são avaliados com F, o avaliador pode subir mais um nível.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- E na prática?
- No momento em que encontrar atributos com nota inferior a F ele para e atribui o nível de capacidade determinado pelos atributos. Se a nota obtida nos dois atributos do nível em que o avaliador parou for igual ou superior a L, então este é o nível de capacidade do processo. Porém, se a nota obtida em pelo menos um dos atributos do nível for P ou N, então a capacidade do processo ainda está no nível imediatamente inferior.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- E na prática?
- Para que uma organização tenha um determinado processo avaliado em um nível n, é necessário que ela obtenha pelo menos escala L (ou L-, na escala com seis valores) nos atributos do nível n e escala F nos atributos de todos os níveis anteriores a n.
- O nível 0, incompleto, não tem atributos. Ele corresponde ao estado inicial de qualquer empresa que nunca tenha implementado processos sistemáticos. O nível 1, realizado, tem apenas um atributo. Já os demais níveis têm dois atributos cada.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- E na prática?
- Esta tabela apresenta, assim, os atributos de cada um dos níveis da ISO 33020.
- Escala de obtenção de atributos com quatro valores

Mnemônico	Significado	Intervalo de obtenção
N = 0	Não obtido	0 a 15%
P = 1	Parcialmente obtido	16 a 50%
L = 2	Amplamente obtido	51 a 80%
F = 3	Totalmente obtido	81 a 100%

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- E na prática?
- Esta tabela apresenta, assim, os atributos de cada um dos níveis da ISO 33020.
- Escala de obtenção de atributos com seis valores

Mnemônico	Significado	Intervalo de obtenção
N = 0	Não obtido	0 a 15%
P- = 1	Parcialmente obtido (fraco)	16 a 32%
P+ = 1	Parcialmente obtido (forte)	33 a 50%
L- = 2	Amplamente obtido (fraco)	51 a 67%
L+ = 2	Amplamente obtido (forte)	68 a 80%
F = 3	Totalmente obtido	81 a 100%

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- E na prática? - Exemplo de avaliação

	Processos			
Atributos	AGR.1	AGR.2	...	REU.3
PA3.2	L			
PA3.1	L			P
PA2.2	F			F
PA2.1	F			F
PA1.1	F	P		F
Nível atribuído	3	0	...	2

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- E na prática? - Resultado da avaliação
- O processo AGR.1 foi avaliado no nível 3 porque possui L nos atributos do nível 3 e F nos atributos dos níveis 1 e 2.
- O processo AGR.2 foi avaliado no nível 0 porque não atingiu sequer L nos atributos do nível 1.
- O processo REU.3 atingiu nível 2 porque, embora tenha F nos atributos dos níveis 1 e 2, não atingiu pelo menos L no primeiro atributo avaliado do nível 3.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- Aula prática
- Um Projeto possui 'N' processos → colocá-los na horizontal
- A ISO 33020 aponta para aproximadamente 60 atributos (com os respectivos resultados esperados) → colocá-los na vertical
- A ISO 33020 aponta para duas escalas de pontuação → utilizaremos a escala de 04 itens

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- E na prática?
- Classificando a qualidade do processo: Esclarecendo a classificação →
- O processo AGR.1 foi avaliado no nível 3 porque possui L nos atributos do nível 3 e F nos atributos dos níveis 1 e 2.

Atributos (ISO)	AGR.1		TABELA CLASSIFICAÇÃO			Mnemônico	Significado	Intervalo de obtenção
PA3.2	L	2	N	0		N = 0	Não obtido	0 a 15%
PA3.1	L	2	P	1 - 1.5		P = 1	Parcialmente obtido	16 a 50%
PA2.2	F	3	L	1.6 - 2.5		L = 2	Amplamente obtido	51 a 80%
PA2.1	F	3	F	2.6 - 3.0		F = 3	Totalmente obtido	81 a 100%
PA1.1	F	3						
	5	13						
Nível atribuído	3	2,6						

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Processos - Normas ISO - (Fonte: RAUL, Wazlawick – Eng. de Software – 12.1)

- E na prática?
- Classificando a qualidade do processo: Esclarecendo a classificação →
- O processo REU.3 atingiu nível 2 porque, embora tenha F nos atributos dos níveis 1 e 2, não atingiu pelo menos L no primeiro atributo avaliado do nível 3.

Atributos (ISO)	REU.3		TABELA CLASSIFICAÇÃO			Mnemônico	Significado	Intervalo de obtenção
PA3.2			N	0		N = 0	Não obtido	0 a 15%
PA3.1	P	1	P	1 - 1.5		P = 1	Parcialmente obtido	16 a 50%
PA2.2	F	3	L	1.6 - 2.5		L = 2	Amplamente obtido	51 a 80%
PA2.1	F	3	F	2.6 - 3.0		F = 3	Totalmente obtido	81 a 100%
PA1.1	F	3						
	4	10						
Nível atribuído	2	2,5						

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

- Tarefa:
- https://unifacedu-my.sharepoint.com/:b:/g/personal/carloslucas_unifacedu_br/EaUAB4uRIItDsh6U4ZbC7HMBOQwZEqoBH-6lilSjowH98Q?e=JWbwhR

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Gerenciamento de Configuração / Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

Erro, Defeito e Falha

- Inicialmente, convém definir alguns termos que, caso contrário, poderiam ser considerados sinônimos, mas que na literatura de teste têm significados bastante precisos:
- • Um erro (error) é uma diferença detectada entre o resultado de uma computação e o resultado correto ou esperado.
- • Um defeito (fault) é uma linha de código, bloco ou conjunto de dados incorretos que provocam um erro observado.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Gerenciamento de Configuração / Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

Erro, Defeito e Falha

- Uma falha (failure) é um não funcionamento do software, possivelmente provocada por um defeito, mas com outras causas possíveis.
- Um engano (mistake), ou erro humano, é a ação que produz ou produziu um defeito no software.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Gerenciamento de Configuração / Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- Cabe observar aqui que o termo fault (defeito) algumas vezes é traduzido como falha, mas a falha em si (failure) é a observação de que o software não funciona adequadamente. Existem falhas que são provocadas por defeitos no software, mas outras que são provocadas por dados incorretos ou problemas tecnológicos (como falha de leitura, segurança ou comunicação).
- Assim, nem todas as falhas são provocadas por defeitos. A área de teste de software ocupa-se principalmente das falhas provocadas por defeitos para que os defeitos sejam corrigidos e, assim, essas falhas nunca mais ocorram.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Gerenciamento de Configuração / Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- Já as falhas provocadas por causas externas ao software costumam ser assunto da área de tolerância a falhas. Os requisitos suplementares de tolerância a falhas de um sistema é que vão estabelecer, por exemplo, como o software se comporta no caso de uma falha de comunicação, armazenamento, segurança etc.
- Note que o requisito vai especificar o comportamento do software em uma situação de falha provocada externamente. Assim, caso o software se comporte de acordo com sua especificação e a falha ocorra, pode-se dizer que ela não é provocada por um defeito no software e que, pelo menos em relação a essa situação, ele parece estar livre de defeito.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Gerenciamento de Configuração / Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- No parágrafo anterior evitou-se afirmar categoricamente que o software está livre de defeito, porque, de fato, tal afirmação dificilmente poderá ser feita para sistemas que não sejam triviais. A maioria dos sistemas de médio e grande porte pode conter defeitos ocultos, porque a quantidade de testes necessária para garantir que estejam livres de defeitos, ou seja, o teste exaustivo, pode ser impraticável, a não ser que estes sistemas já tivessem sido definidos desde o seu início com boas técnicas de teste automatizado.
- Porém, o erro humano na construção dos casos de teste pode estar presente mesmo assim, impedindo defeitos de serem encontrados.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Gerenciamento de Configuração / Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- Assim, a área de teste de software ocupa-se em definir conjuntos finitos e exequíveis de teste que, mesmo não garantindo que o software esteja livre de defeitos, consigam localizar os mais prováveis, permitindo assim sua eliminação.
- Uma máxima da área de teste de software afirma que o teste não consegue provar que o software está livre de defeitos. Ele apenas consegue provar que o software possui algum defeito.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Gerenciamento de Configuração / Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

Verificação, Validação e Teste

- Outra distinção que convém ser feita é entre verificação, validação e teste:
- • Verificação: consiste em analisar o software para ver se ele está sendo construído de acordo com o que foi especificado.
- • Validação: consiste em analisar o software construído para ver se ele atende às verdadeiras necessidades dos interessados.
- • Teste: é uma atividade que permite realizar a verificação e a validação do software.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Gerenciamento de Configuração / Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- Assim, a pergunta-chave para a validação é “Estamos fazendo a coisa certa?”, enquanto a pergunta-chave para a verificação é “Estamos fazendo a coisa do jeito certo?”.
- No caso da validação, trata-se de saber se os requisitos incorporados ao software refletem efetivamente as necessidades dos interessados (cliente, usuário etc.). Isso normalmente é o objetivo dos testes em nível de aceitação ou homologação.
- Já no caso da verificação, trata-se de saber se o produto criado atende aos requisitos que foram especificados da forma mais adequada possível, ou seja, se está livre de defeitos e possui outras características de qualidade já definidas. Isso, normalmente é o objetivo dos testes em nível de unidade, integração e sistema.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Gerenciamento de Configuração / Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

Teste e Depuração

- Enquanto a atividade de teste consiste em executar sistematicamente o software para encontrar erros desconhecidos, a depuração é a atividade que consiste em buscar a causa do erro, ou seja, o defeito oculto que o está causando.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Gerenciamento de Configuração / Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

Teste e Depuração

- O fato de se saber que o software não funciona não significa que necessariamente se saiba qual é (ou quais são) a(s) linha(s) de código que provoca(m) esse erro. A depuração pode ser uma atividade dispendiosa. Por isso, os processos mais modernos recomendam que a integração dos sistemas seja feita de forma incremental e que sejam integradas pequenas partes de código de cada vez, pois, se um sistema funcionava antes da integração e passou a falhar depois dela, o defeito provavelmente está nos componentes que acabaram de ser integrados, e não nos componentes que já funcionavam.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Gerenciamento de Configuração / Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

Stubs e Drivers

- Frequentemente, partes do software precisam ser testadas de modo isolado, mas em geral elas se comunicam com outras partes.
- Quando um componente A que vai ser testado chama operações de outro componente B que ainda não foi implementado, pode-se criar uma implementação simplificada de B, chamada stub, que será utilizada no lugar de B. Por exemplo, essa implementação simplificada pode ser uma função que, em vez de realizar um cálculo, retorna um valor predeterminado, mas que será útil para testar o componente A.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

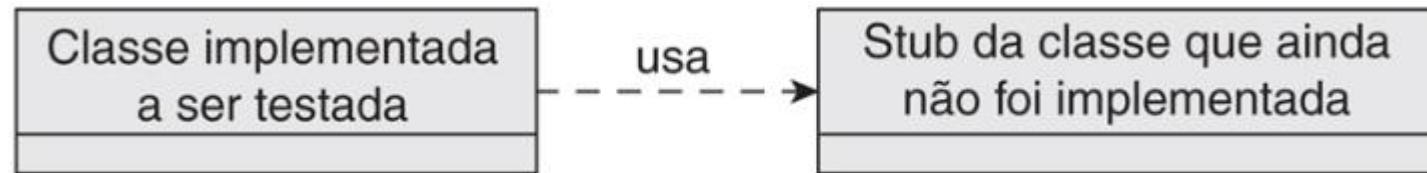
#Gerenciamento de Configuração / Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

Suponha que A é uma classe que precisa usar um gerador de números primos B. A implementação completa de B capaz de gerar o n-ésimo número primo através da função pode ainda não ter sido feita. Mas, possivelmente, para testar A, não será necessário gerar mais do que alguns poucos números primos, por exemplo, os cinco primeiros. Então B poderia ser substituído por uma função stub que gerasse apenas os cinco primeiros números primos.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Gerenciamento de Configuração / Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

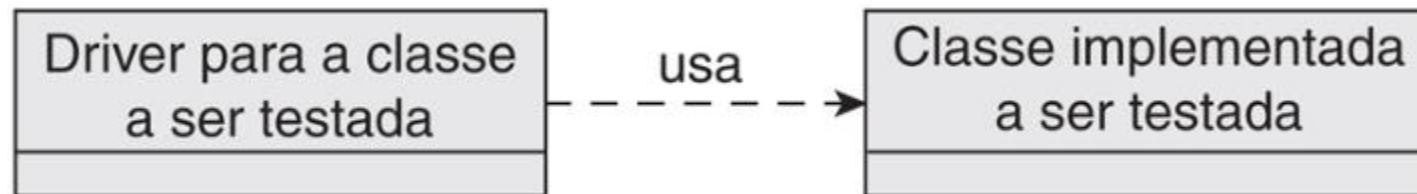
Assim, o stub é implementado com uma simples estrutura de seleção que é definida apenas para os valores de n variando de 1 a 5. Os comandos “assert” estão ali para demonstrar que a função realmente retorna os valores corretos dentro do intervalo definido. Dessa forma, a classe A pode ser testada sem que B tenha sido efetivamente implementada. A Figura representa esquematicamente a relação entre a classe a ser testada e o stub da classe dependente dela.



GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Gerenciamento de Configuração / Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

Por outro lado, muitas vezes é o módulo B que já está implementado, mas o módulo A que chama as funções de B ainda não foi implementado. Nesse caso, deverá ser implementada uma simulação do módulo A, denominada driver. Essa simulação deverá chamar as funções do módulo B e, de preferência, executar todos os casos de teste necessários para testar B, de acordo com a técnica de teste adotada. A Figura mostra esquematicamente a relação entre o driver e a classe a ser testada.



GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Gerenciamento de Configuração / Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- A diferença entre o stub e o driver reside na direção da relação de dependência entre os componentes. Se o componente testado depende (ou seja, chama operações) do componente simulado, então o componente simulado é um stub. Inversamente, se o componente simulado é que chama as operações do componente testado, então o componente simulado é um driver. Assim, um stub e um driver são implementações simplificadas ou simuladas de componentes de sistema, construídas especificamente para atividades de teste.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Gerenciamento de Configuração / Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- Stubs são, normalmente, um código do tipo throw-away, porque depois da implementação real da classe que eles representam, eles não são mais necessários.
- Por outro lado, os drivers são patrimônio de teste. Eles devem ser guardados para a execução de futuros testes de regressão. Isso porque os drivers terão sido construídos de acordo com técnicas especiais para testar sistematicamente as combinações de entradas mais importantes para as classes ou módulos aos quais eles estão associados.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Gerenciamento de Configuração / Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

Fixtures

- Fixtures são variáveis ou objetos criados especificamente para serem usados nas atividades de teste automatizado. Em geral, cada teste será executado em três etapas:
- 1. Preparar o cenário. É o momento em que as fixtures são criadas.
- 2. Executar o teste. Consiste na chamada das funções que vão ser testadas passando as fixtures adequadas em cada caso e verificando se o resultado é o esperado.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Gerenciamento de Configuração / Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

Fixtures

- 3. Limpar o ambiente. Todas as fixtures devem ser destruídas após o teste.
- A criação de fixtures é uma atividade que frequentemente gera código repetido, pois em muitos casos as mesmas fixtures são usadas em mais de um teste.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- Os objetivos dos testes podem variar bastante e abrangem desde verificar se as funções mais básicas do software estão bem implementadas até validar os requisitos junto ao cliente. A classificação de testes mais empregada considera que existem testes de funcionalidade (unidade, integração, sistema, ciclo de negócio, aceitação, operação etc.), e testes suplementares (*performance*, segurança, tolerância a falhas etc.).

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- Deve-se tomar cuidado para não confundir teste de funcionalidade (um nível de teste) com teste funcional (uma técnica de teste); o primeiro termo se refere ao objetivo do teste e o segundo à forma como ele é executado.
- Os testes de funcionalidade têm como objetivo verificar e validar se as funções implementadas estão corretas nos seus diversos níveis.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Unidade**
- Os *testes de unidade* são os mais básicos e costumam consistir em verificar se um componente individual do software (*unidade*) foi implementado corretamente. Esse componente pode ser um método ou procedimento, uma classe completa ou, ainda, um pacote de funções ou classes de tamanho pequeno a moderado. Em geral, essa unidade está isolada do sistema do qual faz parte.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Unidade**
- Os testes de unidade costumam ser realizados pelo próprio programador, e não pela equipe de teste. A técnica de desenvolvimento orientado a testes (TDD), recomenda inclusive que, antes de o programador desenvolver uma unidade de software, ele deve desenvolver o seu *driver* de teste, que, conforme foi visto, é um programa que obtém ou gera um conjunto de *fixtures* que serão usadas para testar sistematicamente o componente que ainda vai ser desenvolvido.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Unidade**
- O teste de unidade pode se valer bem da técnica de teste estrutural, que vai garantir pelo menos que todos os comandos e decisões do componente implementado tenham sido exercitados para verificar se eles têm defeitos. Mas quando se adota TDD é necessário iniciar, pelo menos, com o teste funcional, que verifica o componente relativo à sua especificação. Usualmente, testes estruturais são realizados apenas em código legado ou suspeito, ou ainda código no qual não se tenha muita confiança de que os testes funcionais tenham tido suficiente cobertura de códigos.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Integração**
- Os *testes de integração* são feitos quando unidades (classes, por exemplo) estão prontas, são testadas isoladamente e precisam ser integradas em um *build* para gerar uma nova versão de um sistema. Entre as estratégias de integração, em geral são citadas as seguintes:

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Integração**
- • *Integração big bang*: consiste em construir as diferentes classes ou componentes separadamente e depois integrar tudo junto no final. É uma técnica não incremental, utilizada no ciclo de vida Cascata com Subprojetos. Tem como vantagens o alto grau de paralelismo que se pode obter durante o desenvolvimento e o fato de não precisar de *drivers* e *stubs* durante a integração (embora eles sejam necessários nos testes de unidade), mas como desvantagem cita-se o fato de não ser incremental (portanto, inadequada para o Processo Unificado e métodos ágeis). Além disso, a integração de muitos componentes ao mesmo tempo pode dificultar bastante a localização dos defeitos, pois eles poderão estar em qualquer um dos componentes.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Integração**
- • *Integração bottom-up*: consiste em integrar inicialmente os módulos de mais baixo nível, ou seja, aqueles que não dependem de nenhum outro, e depois ir integrando os módulos de nível imediatamente mais alto. Assim, um módulo só é integrado quando todos os módulos dos quais ele depende já foram integrados e testados. Dessa forma, não é necessário escrever *stubs*, mas em compensação as funcionalidades de mais alto nível do sistema serão testadas tardiamente, quando os módulos de nível superior forem finalmente integrados.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Integração**
- *Integração top-down*: consiste em integrar inicialmente os módulos de nível mais alto, deixando os mais básicos para o fim. A vantagem está em verificar inicialmente os comportamentos mais importantes do sistema em que repousam as maiores decisões. No entanto, como desvantagem, há o fato de que muitos *stubs* são necessários. O teste, para ser efetivo, precisa de bons *stubs*, caso contrário, na integração dos módulos de nível mais baixo poderão ocorrer problemas inesperados.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Integração**
- *Integração sanduíche*: consiste em integrar os módulos de nível mais alto da forma *top-down* e os de nível mais baixo da forma *bottom-up*. Essa técnica reduz um pouco os problemas das duas estratégias anteriores, mas seu planejamento é mais complexo.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Integração**
- • *Integração incremental*: consiste em integrar módulos à medida em que vão ficando prontos, independentemente de serem de nível mais alto, mais baixo ou até intermediário. O planejamento do desenvolvimento dos módulos é que vai determinar se a integração ocorrerá com características mais top-down, bottom-up ou sanduíche.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- A principal desvantagem da maioria das técnicas (exceto a *big bang*) é o fato de que as classes ou componentes individuais que vão ser testados na integração necessitam comunicar-se com outros componentes ou classes que ainda não foram testados ou sequer escritos. Nesse caso, as interfaces que ainda não existem são supridas pelos *stubs*, que são implementações incompletas e simplistas e que se tornam descartáveis depois que o código real é produzido.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- O problema com *stubs*, portanto, é que se perde tempo desenvolvendo software que não vai ser efetivamente entregue. Além disso, nem sempre é possível saber se a simulação produzida pelo *stub* será suficientemente adequada para os testes.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- No caso do desenvolvimento iterativo, especialmente no caso dos métodos ágeis, nem sempre se sabe *a priori* em que ordem os componentes serão integrados, porque vários desenvolvedores estarão trabalhando em paralelo em componentes de diferentes níveis da hierarquia de dependência. Assim, a cada integração deverá ser avaliado se o componente já possui no sistema os componentes dos quais ele depende e os componentes que dependem dele. Na falta destes, usam-se *drivers* ou *stubs*, como nos testes de unidade.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- É necessário ainda estar atento às versões dos componentes do sistema, de forma que a integração sempre deixe claro quais versões de quais componentes foram efetivamente testadas juntas. Assim, os testes de integração nada mais serão do que os testes de unidade executados no contexto da aplicação completa e não de forma isolada. Um bom sistema de controle de versões poderá identificar quais testes devem ser realizados sempre que um novo componente for integrado ao *build*. Para isso, ele deverá usar as relações de dependência entre itens de configuração.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Sistema**
- O *teste de sistema* visa verificar se a versão corrente do sistema permite executar processos ou casos de uso completos do ponto de vista do usuário que executa uma série de operações de sistema em uma interface e é capaz de obter os resultados esperados.
- Assim, o teste de sistema pode ser encarado como o teste de execução dos fluxos de um caso de uso expandido. Se cada uma das operações de sistema (passos do caso de uso) já estiver testada e integrada corretamente, então deve-se verificar se o fluxo principal do caso de uso pode ser executado corretamente, obtendo os resultados desejados, bem como os fluxos alternativos (WAZLAWICK, 2015).

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Sistema**
- Normalmente, o teste de sistema é realizado com o uso da técnica funcional, ou seja, não se examina a estrutura interna do código, apenas a forma como o sistema se comporta em relação a sua especificação.
- Considera-se que o teste de sistema somente é executado em uma versão (*build*) do sistema em que todas as unidades e os componentes integrados já foram testados. Caso muitos erros sejam encontrados durante o teste de sistema, o usual é abortar o processo e refazer, ou mesmo replanejar, os testes de unidade e integração, para que uma versão suficientemente estável do sistema seja produzida e possa ser testada em nível de sistema.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Aceitação**
- O *teste de aceitação* ou *homologação* costuma ser realizado utilizando-se a interface final do sistema. Ele pode ser planejado e executado exatamente como o teste de sistema, mas a diferença é que é realizado pelo usuário final ou cliente, e não pela equipe de desenvolvimento.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Aceitação**
- Em outras palavras, enquanto o teste de sistema faz a *verificação* do sistema, o teste de aceitação faz sua *validação*. O teste de aceitação tem como objetivo principal, portanto, a validação dos requisitos do software, e não apenas a verificação de defeitos em relação às especificações. Ao final do teste de aceitação, o cliente poderá aprovar a versão do sistema testada ou solicitar modificações.
- O teste de aceitação pode ter mais duas variantes:

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- • *Teste alfa*: esse teste é efetuado pelo cliente ou seu representante de forma livre, sem o planejamento e a formalidade do teste de sistema. O usuário vai utilizar o sistema e suas funções livremente e, por isso, esse teste também é chamado *teste de aceitação informal*, em oposição ao *teste de aceitação formal*, que deveria seguir o mesmo planejamento utilizado pelo teste de sistema. O teste alfa é realizado no ambiente de desenvolvimento, ou seja, o cliente vai até o local de desenvolvimento para realizar o teste.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- • *Teste beta*: esse teste é ainda menos controlado pela equipe de desenvolvimento. Nele, versões operacionais do software são disponibilizadas para vários usuários, que, sem acompanhamento direto nem controle da empresa desenvolvedora, vão explorar o sistema e suas funcionalidades. Normalmente, versões beta de sistemas expiram após um período predeterminado, quando então os usuários são convidados a fazer uma avaliação do sistema. O teste beta é realizado fora do ambiente de desenvolvimento, sem controle direto da equipe desenvolvedora.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- Os sistemas feitos sob medida (*tailored*) devem preferencialmente ser testados pelo teste de aceitação formal, pois após o teste haverá uma aceitação formal de que o sistema está correto.
- Já os sistemas de prateleira (*off-the-shelf*) são normalmente testados por testes de aceitação alfa e beta.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Ciclo de Negócio**
- O *teste de ciclo de negócio* é uma abordagem possível tanto no teste de sistema quanto no teste de aceitação formal, e consiste em testar uma sequência de casos de uso que correspondem a um possível ciclo de negócio da empresa.
- Assim, em vez de testar os casos de uso isoladamente, o analista ou cliente vai testá-los no contexto de um ciclo de negócio. Pode-se, por exemplo, testar a sequência de compra de um item do fornecedor, seu registro de chegada em estoque, sua venda, entrega e respectivo pagamento. São vários casos de uso relacionados no tempo, pois representam o ciclo de vida em um item com relação à empresa.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Ciclo de Negócio**
- Os testes de ciclo de negócio podem ser planejados a partir de especificações de modelo de negócio (se existirem).
- Essas especificações costumam ser feitas com diagramas de atividade UML (WAZLAWICK, 2015) ou BPMN (Business Process Modeling and Notation).

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Regressão**
- O *teste de regressão* é executado sempre que um sistema em operação sofre manutenção. O problema é que a correção de um defeito no software, ou a modificação de alguma de suas funções, pode ter gerado novos defeitos. Nesse caso, devem ser executados novamente todos os testes de integração e/ou testes de sistema sobre as partes afetadas.
- No caso de manutenção adaptativa, pode ser necessário executar testes de aceitação, com participação do cliente. Mas, no caso da manutenção corretiva, tais testes não são necessários, visto que a funcionalidade do software, já aprovada pelo cliente, não muda.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Regressão**
- O teste de regressão tem esse nome porque, se ao se aplicarem testes a uma nova versão na qual versões anteriores passaram e ela não passar, então considera-se que o sistema *regrediu*.
- Como é um teste relativamente demorado e aplicado diversas vezes ao longo do tempo de vida do software, o ideal é que o teste de regressão seja sempre automatizado.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Operação**
- O *teste de operação* pode ser uma variante do teste de sistema, aceitação, ciclo de negócio ou regressão que é executado no *ambiente final de implantação* do sistema. Esse tipo de teste é muito importante, especialmente em sistemas críticos, pois problemas de compatibilidade ou convivência entre sistemas, às vezes, podem gerar erros completamente inesperados.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Operação**
- Imagine, por exemplo, um sistema que passou em todos os testes no ambiente de desenvolvimento, inclusive pelos testes de aceitação com a presença do cliente. Agora esse sistema será instalado em uma máquina específica nas dependências da organização cliente. Ele é carregado com dados reais de sistemas legados e colocado para funcionar. Em poucas horas, percebe-se que o sistema começa a travar e rapidamente se torna não operacional. O que aconteceu? É a famosa síndrome de “na minha máquina funcionou”.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Operação**
- Ocorre que a máquina na empresa cliente poderá ter uma versão ligeiramente diferente do sistema operacional, diferente capacidade de memória, processamento ou periféricos, ou ainda, poderá ter instalados na mesma máquina outros produtos de software como antivírus e *firewall*, que podem comprometer de forma inesperada o funcionamento da aplicação que estava homologada no ambiente de desenvolvimento.
- Por outro lado, quando se trata de um sistema a ser instalado em múltiplas máquinas, como por exemplo, sistemas de prateleira. É imprescindível que sejam realizados os testes beta, que neste caso também funcionam como teste de operação.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Testes Suplementares**
- Ao contrário dos testes de funcionalidade, que verificam e validam as funções do sistema, ou seja, os processos que podem ser efetivamente realizados, os *testes suplementares* verificam características normalmente associadas aos requisitos suplementares, como *performance*, interface, tolerância a falhas etc. Uma lista não exaustiva de testes suplementares, incluindo apenas os mais comumente realizados, é apresentada nas subseções a seguir.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Interface com Usuário**
- O *teste de interface com usuário* tem como objetivo verificar se a interface permite realizar, com eficácia e eficiência, as atividades previstas nos casos de uso. Mesmo que as funções estejam corretamente implementadas, o que já teria sido verificado no teste de sistema, isso não quer dizer que a interface também esteja. Então, em geral, é necessário testar a interface de forma objetiva e específica.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Interface com Usuário**
- O teste de interface com usuário pode ainda verificar a conformidade das interfaces com normas ergonômicas que se apliquem, como a NBR ISO 9241-11:2018, por exemplo.
- No Processo Unificado e outros processos baseados em casos de uso, a base para a realização de testes de interface consistirá também nos casos de uso, de forma análoga ao que acontece no teste de sistema. Porém, agora, em vez de simplesmente verificar se o sistema permite a execução das funcionalidades e a obtenção dos resultados corretos, o teste deverá verificar se a interface está organizada da melhor forma possível para atender ao usuário com eficiência.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Interface com Usuário**
- Outra vantagem do emprego de casos de uso como base para o teste de interface com usuário reside no fato de que eles apresentam a estrutura de fluxo principal de uma interação do usuário com o sistema. Assim, a interface projetada deverá procurar otimizar a sequência de ações do usuário nesse fluxo principal. Ao mesmo tempo, as ações necessárias para realizar os fluxos alternativos deverão estar disponíveis no momento em que forem necessárias, ou seja, quando o usuário estiver executando os passos do caso de uso que poderiam provocar esta ou aquela exceção.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Interface com Usuário**
- Em outras palavras, a sequência de operações do caso de uso deve aparecer claramente na interface em uma ordem lógica, evitando-se, por exemplo, que o usuário precise focar sua atenção em diferentes áreas da interface para seguir uma simples sequência lógica. Algumas interfaces exigem que o usuário faça muitos movimentos de braço para levar o *mouse* para cima e para baixo, para a direita e para a esquerda, enquanto executa uma atividade sequencial, o que deveria ser evitado.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Interface com Usuário**
- Da mesma forma, se ocorrerem exceções que precisarem ser tratadas pelo usuário, a mensagem deverá aparecer em local visível e próximo do foco de atenção desse usuário (por vezes, sistemas colocam mensagens de erro em locais que não são facilmente vistos por ele), assim como os controles de interface necessários para iniciar as ações corretivas também deverão estar próximos à mensagem e ao foco de atenção do usuário sempre que possível (e não três ou quatro páginas adiante, que precisem ser roladas).

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Performance (Carga, Estresse e Resistência)**
- Com o sistema tendo ou não requisitos de desempenho, o *teste de performance* pode ser importante, especialmente nas operações que serão realizadas com grande frequência ou de forma iterativa. O teste consiste em executar a operação e mensurar seu tempo, avaliando se está dentro dos padrões definidos. O teste de performance também pode ser usado para avaliar a confiabilidade e a estabilidade de um sistema. Existem, assim, pelo menos três tipos de teste de performance:

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Performance (Carga, Estresse e Resistência)**
- • *Teste de carga*: é a forma mais simples de teste de performance. Normalmente, o teste de carga é feito para determinada quantidade de dados ou transações que deveriam ser típicos para um sistema e avalia o comportamento do sistema em termos de tempo para esses dados ou transações. Dessa forma, pode-se verificar se o sistema atende aos requisitos de performance estabelecidos e também se existem gargalos de performance para serem tratados.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Performance (Carga, Estresse e Resistência)**
- • *Teste de estresse*: é um caso extremo de teste de carga. Procura-se levar o sistema acima do limite máximo de funcionamento esperado para verificar como ele se comporta. Esse tipo de teste é feito para verificar se o sistema é suficientemente robusto em situações anormais de carga de trabalho. Ele também ajuda a verificar quais seriam os problemas encontrados caso a carga do sistema ficasse acima do limite máximo estabelecido.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Performance (Carga, Estresse e Resistência)**
- • *Teste de resistência*: é feito para verificar se o sistema consegue manter suas características de performance durante um longo período de tempo com uma carga nominal de trabalho. Os testes de resistência devem verificar, basicamente, o uso da memória ao longo do tempo para garantir que não existam perdas acumulativas de memória em função de lixo não recolhido (vazamento de memória), e também se não existe degradação de performance após um substancial período de tempo em que o sistema opera com carga nominal.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Performance (Carga, Estresse e Resistência)**
- Pode-se verificar também, no caso de sistemas concorrentes, o que acontece quando um número de usuários próximo ou acima do máximo gerenciável tenta utilizar o sistema. Algumas vezes, efeitos colaterais indesejados podem surgir nessas situações, por isso esses testes são considerados altamente desejáveis.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Segurança**
- Os testes de segurança costumam ser considerados parte da área de segurança computacional.
- Basicamente, os seis tipos de segurança devem ser testados quando os requisitos de sistema assim o exigirem:
- • *Integridade de informação*: é uma forma de garantir ao receptor que a informação que ele recebeu é correta e completa.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Segurança**
- • *Autenticação*: é a garantia de que um usuário realmente é quem ele diz ser e que os documentos, programas e sites realmente são os esperados.
- • *Autorização*: é o processo realizado para verificar se alguma pessoa ou sistema pode ou não acessar determinada informação ou sistema.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Segurança**
- • *Confidencialidade*: é a garantia de que pessoas que não têm direito à informação não poderão obtê-la.
- • *Disponibilidade*: é a garantia de que pessoas que têm direito à informação conseguirão obtê-la quando necessário.
- • *Não repúdio*: é uma forma de garantir que o emissor ou o receptor de uma mensagem não possam alegar, posteriormente, não ter enviado ou não ter recebido a mensagem.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Segurança**
- Pode-se dizer que os testes de confidencialidade e disponibilidade se complementam mutuamente, já que, se apenas a confidencialidade fosse necessária, sabe-se que a melhor forma de impedir que pessoas não autorizadas tenham acesso à informação é destruindo a informação. Porém, se isso for feito, não haverá mais disponibilidade para as pessoas que têm direito à informação poderem acessá-la.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Recuperação de Falha**
- Quando um sistema tem requisitos suplementares referentes à tolerância ou à recuperação de falhas, eles devem ser testados separadamente. Basicamente, busca-se verificar se o sistema de fato atende aos requisitos especificados relacionados com essa questão.
- Normalmente, o *teste de recuperação de falha* trata de situações referentes a:

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Recuperação de Falha**
 - Queda de energia no cliente ou no servidor.
 - Discos corrompidos.
 - Problemas de comunicação.
 - Quaisquer outras condições que possam provocar a terminação anormal do programa ou a interrupção temporária de seu funcionamento.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Recuperação de Falha**
- Assim, o teste de recuperação de falha, também chamado *teste de robustez*, basicamente consiste em verificar a capacidade de o sistema se tornar novamente operacional após a ocorrência de alguma falha. Pode haver a exigência, na forma de requisito, de que o sistema faça isso automaticamente, mas também é possível que um sistema que precise ser reativado manualmente seja submetido ao teste de falha. Nos dois casos, pode-se avaliar o tempo necessário para colocar o sistema novamente em operação, se ocorre algum tipo de corrupção nos dados, quantas ações do usuário ocorridas antes da falha são perdidas e devem ser novamente executadas etc.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste de Instalação**
- No *teste de instalação*, basicamente, busca-se verificar se o software não entra em conflito com outros sistemas eventualmente instalados em uma máquina, bem como se todas as informações e produtos para instalação estão disponíveis para os usuários instaladores.
- O teste de instalação também é associado com o teste de compatibilidade, através do qual se busca verificar se o sistema é compatível com diferentes sistemas operacionais, fabricantes de máquinas, *browsers* etc.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste Estrutural**
- Em relação às técnicas de teste, existem duas grandes famílias:
 - *Testes estruturais*: testes executados com conhecimento do código implementado, ou seja, que testam a estrutura do programa em si (apresentados nesta seção).
 - *Testes funcionais*: testes executados sobre as entradas e saídas do programa sem que se tenha necessariamente conhecimento do seu código-fonte.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste Estrutural**
- Essas duas famílias de testes incluem várias técnicas, algumas das quais são definidas a seguir. Este capítulo apresenta as principais entre estas técnicas e exemplos completos de casos de teste elaborados com elas. Para um maior aprofundamento em relação à área de teste de software pode-se ainda consultar o livro de Delamaro, Maldonado e Jino (2007).
- Os primeiros testes aos quais um sistema é normalmente submetido são os testes de unidade, que têm como objetivo verificar se as funções mais simples do sistema estão corretamente implementadas. Em geral, esse tipo de teste pode ser conduzido como um teste do tipo estrutural, pois o que se deseja é analisar exhaustivamente a estrutura interna do código implementado.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste Estrutural**
- Um dos problemas com os testes de programas é que é impossível definir um procedimento algorítmico que certifique que um programa qualquer está livre de defeitos. Esse problema é computacionalmente *indecidível* (LUCCHESI, SIMON, SIMON & KOWALTOWSKI, 1979). Assim, de certa forma, os testes precisam ser feitos por amostragem.
- Então, o teste estrutural não vai testar *todas* as possibilidades de funcionamento de um programa, mas as mais representativas. Em especial, o que se busca com o teste estrutural é exercitar todos os comandos do programa e todas as condições pelo menos uma vez para cada um de seus valores possíveis (verdadeiro ou falso).

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste Estrutural**
- Uma primeira técnica de teste estrutural pode ser definida assim: cada comando de seleção ou repetição deve ser testado em pelo menos duas situações: quando a condição de seleção ou repetição é *verdadeira* e quando a condição é *falsa*. No caso dos comandos de repetição deve-se testar como o programa se comporta quando a repetição não é realizada nenhuma vez (uma frequente fonte de defeitos em código) e quando ela é realizada pelo menos uma vez.
- Assim, o teste estrutural é capaz de detectar uma quantidade substancial de possíveis erros pela garantia de ter executado pelo menos uma vez todos os comandos e condições do programa. As subseções a seguir apresentam mais detalhes sobre esta técnica.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Complexidade Ciclomática**
- Uma medida de complexidade de programa bastante utilizada é a *complexidade ciclomática*, a qual pode ser definida, de forma simplificada, assim: se n é o número de estruturas de seleção e repetição no programa, então a complexidade ciclomática do programa é $n + 1$. Assim, a complexidade ciclomática mínima é 1, e ela é atribuída a qualquer programa que tenha apenas comandos sequenciais, sem nenhuma decisão ou repetição. Porém, quando estes comandos existem, a quantidade deles é somada ao “1” atribuído ao programa. Por exemplo, um programa com dois IF terá complexidade ciclomática igual a 3.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Complexidade Ciclomática**
- • Estruturas de seleção no estilo IF: 1 ponto cada.
- • Estruturas de seleção no estilo IF-ELSE: 1 ponto cada. Observa-se assim, que o comando “ELSE” não conta, pois ele em si não é um comando de decisão.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Complexidade Ciclomática**
- Estruturas de seleção no estilo CASE, SWITCH, ELSEIF ou ELIF: 1 ponto para cada opção do CASE ou SWITCH, ou para cada ELSEIF ou ELIF. Observe que o comando CASE ou SWITCH isoladamente não contam, mas apenas as opções definidas por eles. Por outro lado, quando se usa ELSEIF ou ELIF, deve-se contar o IF inicial e mais um ponto para cada ELSEIF ou ELIF que o seguir. Assim como o ELSE ao final de uma sequência de ELIFs não conta, também o OTHERWISE ou DEFAULT ao final de uma estrutura CASE ou SWITCH não conta.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Complexidade Ciclomática**
- • Estruturas de repetição no estilo FOR: 1 ponto cada.
- • Estruturas de repetição no estilo REPEAT-UNTIL ou DO-WHILE: 1 ponto cada.
- • Operadores lógicos binários como OR ou AND na condição de qualquer das estruturas acima: acrescenta-se 1 ponto para cada OR ou AND (ou qualquer outro operador lógico binário, se a linguagem suportar, como XOR, NAND, NOR ou IMPLIES).

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Complexidade Ciclomática**
- • Operador lógico unário NOT: não conta.
- • Chamada de sub-rotina (inclusive recursiva): não conta.
- • Estruturas de seleção e repetição em sub-rotinas ou programas chamados: não contam.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Complexidade Ciclomática**
- Assume-se, em geral, que programas com complexidade ciclomática menor ou igual a 10 podem ser simples e fáceis de testar; com complexidade ciclomática de 11 a 20, são de médio risco em relação ao teste; de 21 a 50, são de alto risco; e que programas com complexidade acima de 50 não são testáveis. Porém, deve-se tomar cuidado com a interpretação do termo “programa”. Um programa, nesse sentido, é um bloco de código único. Se ele fizer chamadas a outros blocos de código, as estruturas desses outros blocos não serão contabilizadas na complexidade ciclomática do programa em questão. Ou seja, a complexidade é contada para uma função ou método individual e não é afetada por outras funções ou métodos chamados por ele.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Grafo de Fluxo**
- O *grafo de fluxo* de um programa é obtido colocando-se todos os comandos em nós e os fluxos de controle em arestas. Comandos em sequência podem ser colocados em um único nó, e estruturas de seleção e repetição devem ser representadas, em regra, através de nós distintos com arestas que indiquem a decisão e a repetição, quando for o caso.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Caminhos Independentes**
- O valor da complexidade ciclomática indica número *máximo* de execuções *necessárias* para exercitar todos os comandos do programa. Assim, se um programa tem complexidade ciclomática 4, por exemplo, isso significa que no máximo quatro casos de teste exercitarão todas as condições e todas as arestas. Esse valor é um máximo porque alguns caminhos de execução deste fluxo podem ser impossíveis de testar.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Caminhos Independentes**
- Assim, uma abordagem mais adequada para o teste seria exercitar não apenas todos os nós, mas todas as *arestas* do grafo de fluxo. Isso é feito pela determinação dos *caminhos independentes* do grafo, que são possíveis navegações do início ao fim do grafo.
- O conjunto dos caminhos independentes pode ser definido através do seguinte algoritmo:

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Caminhos Independentes**
- 1. Inicialize o conjunto dos caminhos independentes com um caminho qualquer do início ao fim do grafo, preferencialmente o caminho mais curto, ou seja, o que passe pela menor quantidade de arestas possível.
- 2. Enquanto for possível, adicione ao conjunto dos caminhos independentes outros caminhos que passem por pelo menos uma aresta na qual nenhum dos caminhos anteriores passou antes. Aqui também deve-se dar preferência a caminhos possíveis que minimizem a quantidade de novas arestas adicionadas de cada vez.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Limitações**
- O teste estrutural costuma ser aplicado para verificar defeitos no código, mas esse tipo de teste não é capaz de identificar, por exemplo, se o programa foi bem especificado. Ou seja, o teste vai verificar se o programa se comporta de acordo com a especificação dada, mas não necessariamente se ele se comporta de acordo com a especificação *esperada*. Então, essa técnica não pode ser convenientemente usada para validar sistemas, apenas para verificar defeitos.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes (Níveis de Teste de Funcionalidade) - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Limitações**
- Outra limitação conhecida do teste estrutural é o fato de que funcionalidades ausentes não são testadas, pois a técnica preconiza apenas o teste daquilo que existe no programa. Ou seja, se algum comando *deveria* ter sido incluído no código, mas não foi, o teste estrutural não será capaz de identificar isso.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Testes de Fundamentos**
 - Erro, Defeito e Falha
 - Verificação, Validação e Teste
 - Teste e Depuração
 - *Stubs e Drivers*
 - Fixtures

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Testes de Funcionalidades**
 - Teste de Unidade
 - Teste de Integração
 - Teste de Sistema
 - Teste de Aceitação
 - Teste de Ciclo de Negócio
 - Teste de Regressão
 - Teste de Operação

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Testes Suplementares**
 - Teste de Interface com Usuário
 - Teste de Performance (Carga, Estresse e Resistência)
 - Teste de Segurança
 - Teste de Recuperação de Falha
 - Teste de Instalação

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste Estrutural**
 - Complexidade Ciclomática
 - Grafo de Fluxo
 - Caminhos Independentes
 - Casos de Teste
 - Múltiplas Condições e Caminhos Impossíveis
 - Limitações

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste Funcional**
 - Particionamento de Equivalência
 - Análise de Valor Limítrofe
 - Teste Funcional em Nível de Sistema
- **TDD – Desenvolvimento Orientado por Testes**
- **Medição em Teste**
- **Depuração**

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste Funcional**
- Para descrever sobre o teste funcional, é preciso recordar a ação do teste estrutural que é adequado quando se pretende verificar a estrutura de um programa.
- Mas, em várias situações, a necessidade consiste em verificar a funcionalidade de um programa independentemente de sua estrutura interna. Um programa pode ter uma especificação ou comportamento esperado, em geral, elaborado em um contrato de operação de sistema, na forma de pré e pós-condições, e o que se deseja saber é se ele efetivamente cumpre esse contrato ou especificação.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste Funcional**
- O *teste funcional* considera apenas os valores de entrada de um programa e suas saídas, conforme uma especificação. Como não se pode, usualmente, testar todos os valores possíveis de entrada para um programa, uma amostragem adequada deve ser feita. Basicamente duas técnicas determinam quais devem ser estes valores: *classes de equivalência* e *análise de valor limítrofe*.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Particionamento de Equivalência**
- Um dos princípios do teste funcional é a identificação de situações equivalentes. Por exemplo, se um programa aceita um conjunto de dados (normalidade) e rejeita outro conjunto (exceção), pode-se dizer que existem duas *classes de equivalência* para os dados de entrada do programa: a dos dados aceitos e a dos dados rejeitados. Pode ser impossível testar todos os elementos de cada classe de equivalência, até porque essas classes podem ter infinitos elementos. Então, o *particionamento de equivalência* vai determinar que *pelo menos* um elemento de cada classe de equivalência seja testado.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Particionamento de Equivalência**
- Classicamente, a técnica de particionamento de equivalência considera a divisão das entradas possíveis da seguinte forma :
 - Se as entradas válidas são especificadas como um *intervalo de valores* (por exemplo, de 10 a 20), então são definidas uma classe válida, de 10 a 20, e duas classes inválidas: menor do que 10 e maior do que 20.
 - Se as entradas válidas são especificadas como uma *quantidade de valores* (por exemplo, uma lista com 5 elementos), então são definidas uma classe válida (lista com 5 elementos) e duas inválidas (lista com menos de 5 elementos e lista com mais de 5 elementos).

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Particionamento de Equivalência**
- Classicamente, a técnica de particionamento de equivalência considera a divisão das entradas possíveis da seguinte forma (MYERS, SANDLER, BADGETT & THOMAS, 2004):
 - Se as entradas válidas são especificadas como um *conjunto de valores aceitáveis* que podem ser tratados de forma diferente (por exemplo, as *strings* “masculino” e “feminino”), então é definida uma classe válida para cada uma das formas de tratamento e uma classe inválida para outros valores quaisquer.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Particionamento de Equivalência**
- Classicamente, a técnica de particionamento de equivalência considera a divisão das entradas possíveis da seguinte forma (MYERS, SANDLER, BADGETT & THOMAS, 2004):
 - Se as entradas válidas são especificadas como uma condição do tipo “deve ser de tal forma” (por exemplo, uma restrição sobre os dados de entrada como “o ISBN do livro não deve constar na base de dados”), então devem ser definidas uma classe válida (livros cujo ISBN não consta na base de dados) e uma inválida (livros cujo ISBN já consta na base de dados).

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Particionamento de Equivalência**
- As classes válidas devem ser definidas não só em termos de restrições sobre as entradas, mas também em função dos resultados a serem produzidos. Se a operação a ser testada puder ter dois comportamentos possíveis em função do valor de um dos parâmetros, vão existir duas classes distintas de valores válidos para aquele parâmetro, um para cada comportamento possível.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Análise de Valor Limítrofe**
- Um dos ditados em teste de software diz que os *bugs* (tradução literal: “insetos”) costumam se esconder nas *frestas*. Em função dessa realidade, a técnica de partição de equivalência normalmente é usada em conjunto com o critério de análise de valor limítrofe.
- A *análise de valor limítrofe* consiste em considerar não apenas um valor qualquer para teste dentro de uma classe de equivalência, mas um ou mais valores fronteiros com outras classes de equivalência quando isso puder ser determinado.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Análise de Valor Limítrofe**
- A análise de valor limítrofe sugere que possíveis erros de lógica do programa não vão ocorrer em pontos arbitrários dentro desses intervalos, mas nos pontos em que um intervalo se encontra com outro. Isso acontece, por exemplo, quando um programador usa, em alguma parte do programa que envolva repetição ou seleção, “>” em vez de “>=”, ou quando ele usa n em vez de $n-1$.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Análise de Valor Limítrofe**
- Assim, se existir um erro no programa para alguma dessas classes, é muito mais provável que ele seja capturado dessa forma do que se fosse selecionado um valor qualquer dentro de cada um desses três intervalos.
- Mas a técnica não diz que *apenas* estes valores devem ser testados. Ela diz que eles devem *ser obrigatoriamente* testados, mas outros valores dentro das classes de equivalência podem também ser testados.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste Funcional em Nível de Sistema**
- O teste de sistema, consiste em testar a interface do sistema como se fosse um usuário tentando realizar os fluxos de atividades requeridos. Possivelmente uma das melhores formas de documentar estes fluxos atualmente, sejam os *casos de uso* (WAZLAWICK, 2015). O teste funcional é adequado para este nível de teste porque ele será feito sem que o código em si seja conhecido. A ideia é executar todos os fluxos possíveis do caso de uso e obter os resultados desejados em cada situação.
- Um caso de uso possui sempre um *fluxo principal*, também chamado *caminho feliz*, que é a sequência de ações do usuário e respostas do sistema onde nada dá errado. Por exemplo, ir a um caixa automático, se identificar, solicitar uma quantia em dinheiro, receber essa quantia e ir embora.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste Funcional em Nível de Sistema**
- Mas existem fluxos alternativos que são definidos para tratar situações de exceção como, por exemplo, quando o usuário erra a senha, solicita uma quantia além do seu saldo ou além do saldo da máquina ou fora do horário de operação da máquina. Estes são chamados *fluxos de exceção* e normalmente são sequências de ações que iniciam em algum ponto do fluxo principal e possivelmente voltam a ele depois.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste Funcional em Nível de Sistema**
- Há ainda as *variantes*, que são fluxos alternativos que não correspondem necessariamente a situações de erro, mas simplesmente a variações do comportamento do usuário. Ele poderia, por exemplo, se identificar usando biometria em vez de senha ou poderia ainda enviar o produto ao endereço cadastrado, inserir um novo endereço para envio. Esses fluxos alternativos também sairiam em algum ponto do fluxo principal e possivelmente retornariam a ele depois.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Teste Funcional em Nível de Sistema**
- Diferentemente do que acontece com o teste funcional em nível de unidade, onde temos testes de sucesso e de falha, no caso de testes de sistema, o objetivo é sempre chegar ao final do caso de uso passando apenas pelo fluxo principal, ou ao menos iniciando nele, passando por um fluxo de exceção ou variante e retornando ao fluxo principal. Assim, não há, usualmente, testes de falha, mas testes que avaliam como o sistema se comporta quando ocorrem exceções e variantes e como ele se recupera no caso das exceções.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Desenvolvimento Orientado por Testes**
- 1. Primeiramente o desenvolvedor, que recebe a especificação de uma nova funcionalidade a ser implementada, deve programar um conjunto de testes automatizados para testar o código que ainda não existe.
- 2. Esse conjunto de testes deve ser executado e falhar. Isso é feito para mostrar que os testes efetivamente têm algum poder de verificação e não terão sucesso a não ser que o código específico seja desenvolvido. Se o código passar em algum desses testes, é possível que ou o teste não seja efetivo, ou que a característica a ser implementada já exista no sistema.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Desenvolvimento Orientado por Testes**
- 3. Em seguida, o código deve ser desenvolvido da forma mais simplificada possível, isto é, apenas para passar nos testes.
- 4. Depois que o código passar em todos os testes, ele pode ser refatorado, para atender a padrões de qualidade interna, e testado novamente até que passe em todos os testes novamente.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Desenvolvimento Orientado por Testes**
- Um dos pontos-chave nesse processo é que, depois que os testes foram estabelecidos, o programador não deve implementar nenhuma funcionalidade além daquelas para as quais os testes foram criados. Isso evita que os programadores percam tempo criando estruturas que efetivamente não eram necessárias desde o início.
- Para escrever um conjunto de testes efetivo, é necessário que o programador primeiro compreenda perfeitamente os requisitos e a especificação do componente que vai desenvolver. Nesse sentido, os contratos de operação de sistema (pós e condições) e o roteiro de teste funcional correspondente podem ajudar muito.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Desenvolvimento Orientado por Testes**
- Um desenvolvimento extra pode ser atingido quando se consegue automatizar os testes de aceitação do usuário.
- Essa abordagem, conhecida como ATDD (Acceptance Test-Driven Development) (KOSKELA, 2007), disponibiliza aos desenvolvedores a garantia de que os requisitos especificados estão sendo atendidos, mesmo que o usuário não esteja presente todo o tempo.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Medição em Teste**
- Uma informação importante a ser relatada por equipes de teste é o *status* da atividade de teste (CROWTHER, 2012), que deve ser feito de forma precisa e compreensiva.
- Então, nesse caso, aplica-se também o conceito de métrica para avaliação objetiva de *status*. Pode-se falar em dois grandes conjuntos de métricas de teste:

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Medição em Teste**
- • *Métricas do processo de teste*: as medidas relacionadas reportam a quantidade de testes requeridos, planejados, executados etc., mas não o estado do produto em si.
- • *Métricas de teste do produto*: as medidas relacionadas reportam o estado do produto em relação à atividade de teste, por exemplo, quantos defeitos foram encontrados, quantos estão em revisão, quantos foram resolvidos etc.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Medição em Teste**
- As seguintes medições relacionadas com as métricas de processo de teste poderiam ser apresentadas durante a preparação e a execução dos testes:
 - Número de testes previstos (sua necessidade já foi identificada).
 - Número de testes planejados (casos de teste definidos).
 - Número de testes executados, incluindo: (a) testes que falharam e (b) testes que foram aprovados.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Medição em Teste**
- Essas métricas podem ser comparadas ou relativizadas com outras métricas usualmente tomadas sobre o processo, como o tempo investido nas atividades, o número de pontos de função etc.
- Alguns exemplos de métricas de teste relativas ao produto são:
 - Número de defeitos descobertos.
 - Número de defeitos corrigidos.
 - Distribuição dos defeitos por gravidade.
 - Distribuição dos defeitos por classe ou pacote.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Medição em Teste**
- As medições relacionadas com essas métricas podem ser apresentadas em gráficos que permitem visualizar o desempenho das equipes de desenvolvimento e testes em que se espera que o número de defeitos descobertos seja relativamente constante no tempo e que as atividades de correção atinjam seus objetivos também de forma constante, até que o produto esteja suficientemente livre de defeitos para ser implantado .

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Depuração**
- *Depuração* é o processo de remover defeitos de um programa. Normalmente se inicia com atividades de teste que identificam o problema, ou a partir de inspeções de código, ou ainda a partir de relatos de usuários. No último caso, a equipe deve, no início do processo de depuração, tentar reproduzir o defeito relatado pelo usuário, o que nem sempre é possível ou fácil.
- A depuração é uma atividade artesanal em que a quantidade e a variedade de casos dificultam a elaboração de um processo padrão. Porém, a adoção de boas técnicas de engenharia de software, como integrações frequentes, controle de versões, desenvolvimento orientado a testes etc., podem facilitar bastante o processo de descoberta e correção de defeitos.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes - (Fonte: RAUL, Wazlawick – Eng. de Software – 13)

- **Depuração**
- A tarefa de depuração pode ser tão simples quanto corrigir uma *string* com erro ortográfico ou tão complexa a ponto de exigir um trabalho de pesquisa, coleta de dados e formulação de hipóteses digna dos melhores detetives.
- De forma geral, ferramentas, conhecidas como *debuggers*, ajudam no trabalho de depuração, pois permitem executar programas passo a passo, observando o valor das variáveis, parar, retornar etc.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **Visão geral**
- A **Implementação** realiza o desenho de um sistema em termos de diversos tipos de componentes de código fonte e código binário, conforme as tecnologias escolhidas. Classes e outros elementos de desenho interno são transformados, em unidades de implementação, geralmente constituídas de arquivos de código fonte. Essas unidades são verificadas de diversas maneiras, como compilações, análises estáticas, revisões informais, inspeções, testes de unidade e testes de integração. Métodos mais recentes de implementação partem dos testes de unidade, sendo as unidades programadas até conseguirem passar nesses testes; testes de regressão são então realizados para garantir que as unidades já implementadas continuem funcionando.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- As unidades são integradas entre si, com unidades resultantes de ciclos anteriores e componentes reutilizados, adquiridos de terceiros, obtidos de uma biblioteca da organização, ou reaproveitados de projetos anteriores. Quando é completada a implementação de novas funções do produto, tem-se uma nova liberação, em um processo de desenvolvimento iterativo.
- Consideramos também que a **Implementação** inclui as atividades cujo objetivo é gerar a documentação de uso do produto. Essa documentação inclui manuais de usuários, ajuda on-line, sítios Web integrados ao produto, material de treinamento, material para demonstrações e outros recursos.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **Artefatos**
- Durante a implementação, os implementadores consultam o **Modelo da solução**, que deve ter sido detalhado em nível suficiente. Nesse modelo, a parte mais ligada à implementação é a **Visão lógica**, que representa, em nível superior de abstração, as classes a serem implementadas, além de registrar as decisões de desenho que influenciam a implementação. Indiretamente, a implementação é também dirigida pela **Visão de uso** e pelo **Plano das liberações**. Seria possível representar fielmente o código implementado por uma **Visão de implementação**; pelas razões discutidas no capítulo sobre **Desenho**, essa visão não será adotada aqui.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Outros artefatos usados são os componentes produzidos anteriormente e reutilizados, que incluem arquivos executáveis, arquivos de componentes, bibliotecas, tabelas de bancos de dados e documentos. São usados também componentes de testes, como os controladores (*drivers*), e, possivelmente, dependendo dos tipos de teste de unidade, outros componentes conferentes e inspetores, definidos no capítulo sobre **Testes**, e cotos (*stubs*), que substituem provisoriamente unidades que ainda não estão disponíveis.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **TÉCNICAS**
- **2.1 Visão geral**
- O núcleo das atividades de implementação é formado pelas técnicas básicas que formam o ciclo de programação: desenho detalhado, codificação, testes de unidade e integração. Todas elas, por sua vez, usam conceitos de modularização. Em qualquer produto não trivial, a implementação é complementada pela confecção da documentação de usuário.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **TÉCNICAS**
- **2.1 Visão geral**
- A programação começa com o **Desenho detalhado**, que completa o desenho interno no nível necessário para guiar a codificação. O resultado do detalhamento é verificado, geralmente por meio de revisão informal feita pelos autores, possivelmente com a ajuda de outros programadores. Nessa revisão, são conferidos os detalhes de interfaces, lógica e máquinas de estado das operações.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- A tradução do desenho detalhado no código de uma ou mais linguagens de programação é feita na **Codificação**. As unidades de código fonte produzidas podem ser submetidas a uma revisão informal de código, para eliminar os defeitos que são introduzidos por erros de digitação ou de uso da linguagem. Essa revisão também deve ser feita pelos autores, possivelmente com a ajuda de outros programadores.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- As unidades de código fonte são transformadas em código objeto por meio da compilação. As revisões informais podem ser substituídas pela prática da **programação em pares**, na qual cada unidade é produzida por um par de programadores, que se revezam nas funções de digitar e verificar em tempo real o que está sendo programado.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- O desenho detalhado e o código são verificados pela execução dos **Testes de unidade**. Esses testes formam o principal tipo de testes de desenvolvimento, e usam componentes de testes de unidade, que devem também ter sido desenhados, revistos, codificados e compilados. Embora bem menos formais que os testes de sistema, os resultados desses testes devem pelo menos ser registrados em relatórios simplificados, produzidos de forma automatizada.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- No método de **Implementação dirigida por testes**, a confecção dos testes de unidade precede a codificação, ou, pelo menos, é feita simultaneamente com ela. Na literatura de processos ágeis, esse método é frequentemente chamado de “desenvolvimento dirigido por testes”; usamos aqui o termo “implementação dirigida por testes”, para contrastá-lo com “desenvolvimento dirigido por casos de uso”, indicando que aquele método de implementação é um dos aspectos desse método de desenvolvimento.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- A **Integração**, finalmente, liga as unidades implementadas com os componentes construídos em ciclos anteriores ou reutilizados. O código executável resultante é submetido aos testes de integração, que incluem regressões com os testes de sistema de liberações anteriores, para confirmar que não foram introduzidos efeitos colaterais indesejáveis. Nos métodos mais modernos de programação, adotados nos processos ágeis, a integração é feita de forma contínua, usando-se ferramentas que, além da compilação e ligação, automatizam a gestão de configurações e a realização dos testes de unidade e integração.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- A **Documentação de usuário** tem como principal insumo os resultados do desenho externo, enquanto as atividades acima partem do desenho interno. Em certos tipos de aplicativos, como os sistemas baseados em Web, essa atividade não gera documentos separados, e sim gabaritos para a geração de páginas que são misturadas aos relatórios produzidos, de acordo com a tecnologia empregada. Nesse caso, pode ser mais apropriado tratá-la como parte da atividade de **Criação de conteúdo**, da disciplina de **Engenharia de sistemas**.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Desenho detalhado
- 2.2.1 Visão geral
- O desenho detalhado começa por verificar os pré-requisitos para a implementação das unidades, como as definições contidas no **Modelo da solução**, principalmente na **Visão lógica**. É possível, no entanto, que a maior parte dessa visão só seja completada após a implementação. Nesse caso, após serem usadas as técnicas de implementação dirigida por testes, do refatoramento e da integração contínua, o desenho emerge do código; durante a implementação; o desenho detalhado fica inicialmente apenas implícito no código, e a atualização do **Modelo da solução**, se for requerida, vem depois. Nessa estratégia, o desenho detalhado é feito consultando-se diretamente a **Visão de análise do Modelo do problema**.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- O desenho detalhado, além disso, tem que:
- •decidir como serão testadas as unidades desenhadas, para que o desenho produzido seja testável, e desenhar os respectivos testes de unidade;
- •analisar os aspectos de eficiência, decidindo se as unidades são críticas do ponto de vista dos requisitos de desempenho ou se esses aspectos podem ser tratados depois, em uma etapa de otimização do produto;
- •pesquisar, analisar e desenhar os algoritmos e as estruturas de dados que deverão ser usados;

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- definir detalhes do comportamento, usando recursos como precondições, pós-condições, asserções, lógica combinatória, máquinas de estados e scripts em pseudolinguagem;
- •definir outros aspectos importantes das unidades, como construção de instâncias, destruição de instâncias (caso não seja automatizada pela linguagem de programação, ou, mesmo nesse caso, se requerer tratamento específico, como o método finalize() dos objetos Java), listas de parâmetros, valores de retorno, classes genéricas ou parametrizadas, tratamento de exceções e uso de anotações;
- •refatorar sempre que apareçam oportunidades para melhorar o desenho com isso;
- •produzir, caso necessários, diagramas da **Visão de implementação**;

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- no caso de classes persistentes, acertar os detalhes de tratamento da persistência, possivelmente usando uma **Visão de dados**;
- •definir o desenho físico estático, atribuindo elementos lógicos a componentes na **Visão de componentes**, caso seja necessária alguma modificação ou acréscimo, em relação ao padrão de tratamento de componentes do ambiente de desenvolvimento;
- •no caso de sistemas distribuídos, completar o desenho físico dinâmico, definindo processos concorrentes e atribuindo unidades a nodos do sistema, na **Visão de implantação**;
- •revisar o desenho detalhado, informalmente ou por meio de programação em pares.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- O desenho detalhado utiliza as práticas recomendadas de **modularização**, que tratam da divisão de sistemas e produtos em **módulos**. Esses são definidos como *unidades de programa discretas e identificáveis em relação à compilação e à combinação com outras unidades e carga, ou partes logicamente separáveis de um programa*, segundo o glossário do IEEE; ou como *unidades de armazenamento e manipulação de software*, segundo a UML. Módulos contêm rotinas, que são *subprogramas chamados por outros programas ou subprogramas*.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Em relação a questões de eficiência, que são a primeira preocupação de muitos programadores, deve-se observar que geralmente a eficiência resulta mais de bom desenho do que da codificação. Questões como o tratamento de estruturas de dados e da persistência, o uso de algoritmos apropriados e a atribuição de processos e unidades aos nodos, em sistemas distribuídos, podem facilmente dominar os aspectos de desempenho. Glass [Glass03] observa que os compiladores otimizantes podem produzir código com 90% da eficiência de código codificado manualmente em linguagem de montagem (*assembler*), ou mesmo superá-la, dada a complexidade de algumas arquiteturas atuais. Além disso, geralmente há compromissos entre otimização de tempo e espaço, que devem ser resolvidos pelo desenho lógico ou detalhado.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- O desenho detalhado utiliza as práticas recomendadas de **modularização**, que tratam da divisão de sistemas e produtos em **módulos**. Esses são definidos como *unidades de programa discretas e identificáveis em relação à compilação e à combinação com outras unidades e carga, ou partes logicamente separáveis de um programa*, segundo o glossário do IEEE; ou como *unidades de armazenamento e manipulação de software*, segundo a UML. Módulos contêm rotinas, que são *subprogramas chamados por outros programas ou subprogramas*.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **Módulos**
- *Níveis de modularização*
- Módulos são constituídos por coleções de rotinas, dados e tipos correlatos, implementados por meio de **componentes** cuja **interface** com o restante de um sistema é bem definida. As interfaces dos componentes de um módulo podem também ser consideradas as interfaces do módulo. Um módulo pode conter outros módulos, formando uma hierarquia. O topo da hierarquia é um sistema inteiro, que pode ser subdividido em vários níveis de módulos, como subsistemas, vários níveis de pacotes, e, finalmente as classes, em tecnologias orientadas a objetos.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Geralmente, trata-se como módulo de nível mais baixo, em cada linguagem de programação, a divisão que é fisicamente representada por um arquivo de código fonte. No caso da linguagem Java, por exemplo, é tipicamente uma classe. Nessa linguagem, entretanto, o conceito de módulo é um pouco mais complexo, pois uma classe pode conter classes internas, inclusive anônimas; interfaces, no sentido dado pela UML, também podem ser representadas por unidades de código fonte Java.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Se a linguagem usada para a implementação não suportar de forma direta um conceito que possa representar módulos, estes devem ser realizados através de convenções de codificação que permitam distinguir entre nomes importados, exportados e internos. Por exemplo, nomes de estruturas de dados e sub-rotinas podem ter um prefixo comum, que indica que pertencem à mesma “classe”, do ponto de vista conceitual.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

.Hierarquia de unidades

Sistema	1. (IEEE) Coleção de componentes organizados para realizar uma função ou conjunto de funções. 2. (UML) Coleção de unidades conectadas, organizadas para cumprir um objetivo.
Subsistema	(IEEE) Um sistema subordinado ou secundário dentro de um sistema maior.
Pacote	(UML) Mecanismo para organização de elementos em grupos, que estabelece a quem pertencem os elementos, provendo nomes únicos para referência a esses elementos.
Classe	(UML) Descritor para um conjunto de objetos que partilham os mesmos atributos, operações, relacionamentos e comportamento.
Módulo	1. (IEEE) Unidade de programa discreta e identificável em relação à compilação e à combinação com outras unidades e carga. 2. (IEEE) Parte logicamente separável de um programa. 3. (UML) Unidade de armazenamento e manipulação de software.
Componente	1. (PMBOK) Uma parte constituinte, um elemento ou parte de um todo complexo. 2. (IEEE) Uma das partes que constituem um produto ou sistema. 3. (UML) Parte modular de um desenho de sistema, que encapsula a implementação atrás de um conjunto de interfaces.
Interface	1. (IEEE) Uma fronteira partilhada através da qual é passada informação. 2. (IEEE) Um componente que liga dois ou mais componentes, com o propósito de passar informação entre eles. 3. (UML) A declaração de um conjunto coerente de características e obrigações públicas, que serve de contrato entre produtores e consumidores de serviços.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- *Organização física dos módulos*
- É recomendável que haja correlação forte entre as estruturas lógica e física. Isso vale principalmente se a linguagem não suportar explicitamente a definição de módulos; nesse caso, a estrutura física pode ser usada para simular a estrutura lógica. Módulos são implementados por meio de componentes (de código fonte ou objeto). O acesso aos recursos dos componentes deve ser feito apenas através de interfaces explicitadas no desenho do sistema e documentadas no desenho e no código.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- O detalhe do mapeamento entre estruturas lógica e física é dependente de linguagem. Em Java, por exemplo, tipicamente cada classe constitui um arquivo, embora, como foi dito anteriormente, arquivos também possam representar interfaces ou agrupamentos muito coesos de classes. Pacotes são mapeados em pastas do sistema de arquivos, constituindo os subsistemas as pastas de mais alto nível.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Em outras linguagens, como linguagens de script, pode haver módulos de nível mais baixo que não são classes (por exemplo, podem ser scripts, bibliotecas ou páginas). Ainda assim, cada módulo de nível mais baixo deve, de preferência, residir em somente um arquivo. Se um módulo tiver de ser partido em arquivos, por motivos de tamanho, deve constituir uma pasta separada. Por outro lado, pode ser conveniente colocar em um único arquivo um pequeno grupo de módulos pouco complexos, coletivamente responsáveis pela execução de um mecanismo.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- *Critérios de modularização*
- Módulos separados devem ser usados sempre que haja necessidade de isolar partes menos estáveis ou mais complexas de um produto. Isso aumenta a robustez do desenho: limitam-se os efeitos colaterais de mudanças e defeitos, tornando o produto mais extensível e de manutenção mais fácil. Devem ser isoladas em módulos, por exemplo, partes cuja implementação ou desenho apresentam dificuldades especiais, como uso de tecnologia nova ou necessidade de interfaces com plataformas complexas. As partes dependentes de aspectos específicos de uma plataforma devem ser isoladas, o que aumenta a portabilidade.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Em qualquer nível, um módulo deve exibir as seguintes propriedades fundamentais:
- **alta coesão** — oferecer um grupo de serviços logicamente correlatos;
- **baixo acoplamento** — oferecer ao mundo exterior uma interface pequena e bem-definida.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- No desenho detalhado, é particularmente importante isolar as áreas propensas a mudanças, visando a reduzir os impactos de possíveis variações de requisitos ou de tecnologia. Tipicamente, essas áreas possuem uma ou mais das seguintes características:
 - dependências de hardware;
 - interfaces de usuário e com outros sistemas;
 - uso de recursos de linguagem ou plataforma que estão fora do padrão;
 - desenho particularmente difícil;

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- implementação particularmente difícil;
- manipulação de variáveis de status frequentemente atualizadas, como variáveis que indicam o estado de atualização de uma estrutura de dados;
- dependências de tamanho de dados, como declarações de constantes que definem tamanhos de estruturas de dados;
- dependências de regras de negócio, como lógica dependente de requisitos legais ou de políticas da organização.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- *Classes*
- Além de levar em conta os critérios anteriores, o desenho detalhado de classes de boa qualidade deve responder às seguintes questões:
- Como os objetos devem ser construídos?
- Como os objetos devem ser destruídos (ou que cuidados especiais devem ser possivelmente tomados, se a destruição for automatizada)?
- Como funcionam exatamente a atribuição, cópia e comparação dos objetos, especialmente em relação ao que acontece com os objetos internos a eles? Como se distingue a cópia profunda (objetos internos copiados) da cópia rasa (objetos internos apenas referenciados)?
- Quais os valores legais para as variáveis (atributos) dos objetos, e, portanto, quais os tipos apropriados?
- Quais variáveis (atributos) devem ser de classe (estáticos) e quais de instância?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Que métodos são necessários para implementar cada operação modelada?
- Que tipos de verificação de precondições, tratamento de exceções e mecanismos de programação defensiva devem ser feitos nos métodos?
- Que métodos devem ser abstratos (obrigatoriamente definidos por subclasses)?
- Que métodos devem ser finais (não redefiníveis por subclasses)?
- Como deve funcionar a passagem de parâmetros e valores de retorno, com possíveis conversões de tipos?
- Que classes têm que tipo de acesso às variáveis e aos métodos de outras classes?
- Que classes devem ser parametrizadas (genéricas)? Que tipos devem ter esses parâmetros? Que métodos devem ser parametrizados?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Rotinas
- *Visão geral*
- O acesso às interfaces de um módulo de nível mais baixo deve ser feito apenas através de **rotinas**, isto é, subprogramas que são chamados por outros programas ou subprogramas (métodos, em linguagens orientadas a objetos). Dados (atributos, variáveis ou campos) devem ser, de preferência, privados em relação ao módulo. Dependendo da linguagem, o nome “rotina” deve ser interpretado como sub-rotina, função, procedimento, método ou membro de função, podendo retornar ou não um valor.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Rotinas
- *Visão geral*
- Em linguagens orientadas a objetos, as classes correspondem aos módulos de nível mais baixo, e rotinas correspondem aos métodos. De um ponto de vista conceitualmente mais rigoroso, uma **operação** (especificação de uma consulta ou transformação que um objeto pode ser chamado a executar) é implementada por meio de um ou mais **métodos** (implementação de uma operação, que especifica o algoritmo ou procedimento que produz os resultados da operação).

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- *Critérios para criação de rotinas*
- A criação de novas rotinas durante o desenho detalhado deve obedecer a critérios bem-definidos. O critério mais importante é sempre a redução da complexidade dos programas, substituindo-se uma série de detalhes por uma abstração adequada. Alguns aspectos específicos de aplicação desse princípio incluem:
 - isolar operações e expressões complexas, que são mais propensas a erros;
 - aumentar a legibilidade do código, diminuindo o número de linhas de cada rotina;
 - ocultar a ordem de execução de sequências, colocando em rotinas separadas código cuja ordem de execução é independente;
 - ocultar detalhes de implementação de estruturas de dados;

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- •ocultar acessos a dados globais (se a linguagem os permitir);
- •ocultar código não portátil;
- •evitar duplicação de código, agrupando em uma única rotina sequências similares de código;
- •melhorar o desempenho, agrupando código otimizável;
- •agrupar código que pode diferir entre variantes de uma família de programas;
- •centralizar pontos de controle, como sequências de acesso a mecanismos persistentes ou dispositivos de hardware;
- •promover a reutilização de código.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- *Coesão*
- Os níveis de coesão de uma rotina do nível mais alto para o mais baixo. Todas as rotinas devem exibir coesão pelo menos comunicacional. A coesão temporal pode ser usada ocasionalmente, e os níveis inferiores de coesão devem ser evitados.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

Níveis de coesão de rotinas

NÍVEL DE COESÃO	DEFINIÇÃO
Funcional	Rotina executa uma única tarefa (simples).
Sequencial	Rotina executa uma sequência de tarefas correlatas e encadeadas.
Comunicacional	Rotina executa sequência de tarefas que usa um conjunto comum de dados.
Temporal	Rotina executa um conjunto de tarefas coincidentes no tempo (por exemplo, iniciação).
Procedimental	Rotina executa uma sequência de tarefas ordenadas mas não correlatas nem encadeadas.
Lógico	Rotina executa uma entre várias tarefas, de acordo com o valor de uma variável de estado (if ... else if ... else, switch).
Coincidente	Rotina executa tarefas não correlatas

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- *Acoplamento*
- O acoplamento se refere à maneira de comunicação entre as rotinas. O acoplamento é um bom compromisso entre as seguintes características: baixa cardinalidade, baixa intimidade, alta visibilidade e alta flexibilidade. Compromissos de engenharia são necessários porque há contradições entre as características. Por exemplo, agrupar em estruturas os dados comunicados tende a melhorar a cardinalidade e a piorar a visibilidade e a flexibilidade.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

Características de comunicação entre rotinas

CARACTERÍSTICA DE COMUNICAÇÃO	DEFINIÇÃO
Cardinalidade	Número de objetos comunicados entre duas rotinas (por exemplo, número de parâmetros).
Intimidade	Grau de controle de acesso (por exemplo, as seguintes formas de comunicação apresentam graus crescentes de intimidade: listas de parâmetros, atributos de classe privados, atributos de classe públicos, variáveis globais e arquivos ou bancos de dados).
Visibilidade	Grau de visibilidade dos dados passados (por exemplo, dados passados como parâmetros separados são mais visíveis do que quando agrupados em uma estrutura).
Flexibilidade	Grau de liberdade que a rotina chamadora tem na montagem dos dados para a rotina chamada (por exemplo, a necessidade de montar uma estrutura complexa acarreta baixa flexibilidade).

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

Níveis de acoplamento entre rotinas

NÍVEL DE ACOPLAMENTO	DEFINIÇÃO
Acoplamento de dados simples	Todos os dados são passados como parâmetros não estruturados.
Acoplamento de estrutura de dados	Usam parâmetros que são estruturas de dados.
Acoplamento de controle	Uma rotina diz à outra o que fazer, passando, por exemplo, uma variável de estado.
Acoplamento de dados globais	As rotinas fazem uso dos mesmos dados globais.
Acoplamento patológico	Uma rotina usa código interno ou altera dados locais de outra.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Rotinas não devem ser divididas artificialmente, apenas para limitar seu tamanho. Por outro lado, deve-se procurar transformar subsequências genéricas de código em rotinas separadas. Se uma subsequência for comum a vários métodos de uma classe, deve-se procurar transformá-la em método privado dessa classe. Rotinas acima de 150-200 linhas de código devem exibir coesão funcional ou sequencial.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Devem-se usar nomes que descrevam de maneira eficaz o que a rotina faz. Normalmente, os nomes de rotinas devem ser verbos com significados fortes. Em linguagens não baseadas em objetos, o nome deve incluir também o objeto da rotina; em linguagens orientadas a objetos, a referência ao objeto (para métodos de instância) ou à classe (para métodos estáticos) será feita pelo respectivo nome (por exemplo, ImprimirRelatorio *versus* relatório.imprimir). Rotinas cujo foco é o valor retornado podem usar uma descrição do valor de retorno (por exemplo, ImpressoraPronta ou impressora.pronta).

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- O nome deve ser tão longo quanto necessário para descrever completamente o que a rotina faz. Na maioria dos casos, ficará tipicamente entre 15 a 20 caracteres. Operações semelhantes devem corresponder a nomes semelhantes (por exemplo, obter, para operações de consulta). Maiores detalhes sobre os nomes são determinados pelas convenções de desenho detalhado e codificação.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- *Parâmetros das rotinas*
- Devem-se usar listas de parâmetros que promovam o entendimento eficaz do uso da rotina. Isso significa obedecer às seguintes diretrizes:
 - Casar os tipos dos parâmetros formais e reais, mesmo que a linguagem não o exija.
 - Colocar os parâmetros na seguinte ordem: parâmetros de entrada (apenas consultados), parâmetros modificados e parâmetros de saída. Objetos ou estruturas cujos atributos ou itens são modificados devem ser considerados parâmetros modificados; consideração semelhante vale para parâmetros de saída.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Colocar parâmetros de status ou códigos de erro em último lugar. Por outro lado, devem ser consideradas práticas usuais da linguagem: por exemplo, existe, em C, a prática de usar o valor de retorno para esse fim. Se a linguagem tiver recursos para tratamento de exceções, como Java, esse mecanismo é preferível.
- Usar a mesma ordem para parâmetros similares em rotinas similares.
- Só passar parâmetros que venham realmente a ser usados. Dependendo das características da linguagem, pode haver exceções; em Java, por exemplo, é o caso dos métodos abstratos.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- •Não usar os parâmetros como variáveis de trabalho, mesmo que a linguagem o permita.
- •Documentar suposições a respeito dos parâmetros (precondições e pós-condições), principalmente se essas não forem verificadas por código.
- •Limitar o número de parâmetros a cerca de sete. Essa é uma aplicação da regra de Miller: na memória humana de curto prazo só cabem sete itens, mais ou menos dois.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- •Passar apenas as partes de uma estrutura que são usadas pela rotina chamada. Por outro lado, pode ser adequado passar toda a estrutura quando esta representar um tipo abstrato de dados. Tipicamente, é o que acontece com a passagem de objetos. Se o objeto como um todo interessar à rotina chamada, ele deve ser passado como parâmetro; se o interesse for apenas em um atributo desse objeto, deve ser passado o valor de retorno de um método de consulta ao atributo em questão.
- •Não utilizar suposições sobre o mecanismo de passagem de parâmetros utilizado pelo compilador. Por exemplo, não usar efeitos colaterais que dependam da ordem de passagem dos parâmetros. Isso torna o código mais portátil e estável.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- *Contratos*
- O contrato de cada operação implementada por um método é uma especificação precisa dos resultados produzidos pela operação, dado um conjunto de entradas conforme com requisitos especificados.
Na **programação por contrato**, o desenho detalhado e a codificação de cada método são desenvolvidos para honrar um contrato especificado. Um contrato pode conter os seguintes elementos;
- •tipos e valores aceitáveis das entradas;
- •tipos e valores esperados dos resultados;
- •precondições a que os valores de entrada devem satisfazer, antes da execução da operação;

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- •pós-condições a que os resultados devem satisfazer, depois da execução da operação;
- •invariantes da classe, isto é, condições que devem permanecer válidas antes e depois da execução da operação;
- •possíveis exceções;
- •possíveis efeitos colaterais;
- •em alguns casos, garantias de desempenho.
- Esses elementos devem ser verificados por asserções ou testes de unidade. Em uma classe especializada, métodos redefinidos podem enfraquecer as precondições dos métodos da classe de base, mas não fortalecê-las; e podem fortalecer as pós-condições e invariantes, mas não enfraquecê-las.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **Técnicas específicas**
- *Lógica combinatória*
- Operações complexas podem dar origem a rotinas com um grande número de caminhos lógicos possíveis, dependendo de condições a que os parâmetros de entrada venham a satisfazer. Mesmo que não haja muitos caminhos possíveis, os resultados podem depender de combinações complexas dos valores das entradas, e nem sempre é possível evitar as combinações inválidas por meio de verificações das precondições no início da rotina.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **Técnicas específicas**
- Se for esse o caso, técnicas baseadas em lógica combinatória, como as tabelas de decisão e os mapas de Karnaugh, devem ser usadas para determinar a lógica que será implementada pela rotina. Essas técnicas também ajudam a minimizar o número de condições que devem ser avaliadas, reduzindo os pontos de introdução de defeitos e, possivelmente, melhorando o desempenho.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- *Lógica sequencial*
- Rotinas mais complexas podem também seguir caminhos diferentes, ou mesmo rotinas mais simples podem causar resultados diferentes, dependendo do estado em que o objeto a que pertencem se encontre. Valores das entradas ou mesmo rotinas inteiras que são válidas em um estado podem ser inválidas em outros; uma entrada válida pode causar uma transição inválida, levando para um estado que não era esperado, possivelmente causando uma exceção ou mesmo um estado inválido. Estados inválidos podem estar associados a dados corrompidos, situação que pode causar erros; em alguns casos, sem possibilidades de recuperação.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- *Lógica sequencial*
- Máquinas de estado podem ser usadas no desenho detalhado de classes cujo comportamento possa ser caracterizado por estados e transições. Esse é, frequentemente, o caso de sistemas embutidos e de tempo real. Nessas classes, o estado é geralmente caracterizado por um atributo ou conjunto de atributos, implementados pelas variáveis de estado. Os eventos que podem causar transições são associados a métodos tipicamente públicos, e os efeitos o são a métodos tipicamente privados.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- *Pseudolinguagem*
- Os scripts em pseudolinguagem podem também ser úteis no desenho detalhado, para representar lógica mais complexa. Nos casos mais comuns, scripts de texto são suficientes, sendo anotados nos modelos dentro dos campos de documentação dos métodos, ou na forma de comentários ou restrições. Os diagramas de atividade podem trazer alguma vantagem no caso de scripts complexos, principalmente se houver tratamento de paralelismo interno à rotina.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Os scripts em pseudolinguagem podem facilitar as revisões do desenho detalhado e reduzir o esforço adicional de inspeção de implementação. Eles dão suporte ao refinamento iterativo, permitindo aumentar o detalhe do desenho em passadas consecutivas, facilitando mudanças do desenho detalhado, antes que este tenha sido comprometido em código. Um bom script pode ser depois aproveitado na forma de comentários do código.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- As seguintes diretrizes devem ser usadas para a escrita de scripts em pseudolinguagem:
- •usar frases da linguagem natural que descrevam com precisão operações específicas;¹
- •evitar detalhes de sintaxe da linguagem alvo, pois isso abaixa o nível de abstração e torna a pseudolinguagem desnecessariamente restrita;
- •descrever a intenção do desenho, e não o mecanismo de implementação na linguagem alvo;
- •descer em nível de detalhe suficiente para tornar a tradução para a linguagem alvo quase imediata para um programador proficiente.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- *Programação defensiva*
- O desenho detalhado de um módulo deve incluir diversos aspectos de prevenção de defeitos. O módulo deve ser desenhado e codificado considerando-se sempre a possibilidade de receber dados errados, através de suas interfaces. A prevenção de defeitos é baseada no conceito de programação defensiva.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Um exemplo de programação defensiva é o uso de asserções. Uma **asserção** é uma expressão lógica que especifica um estado que deve existir, ou um conjunto de condições a que as variáveis de um programa devem satisfazer durante certos pontos da execução do programa. Asserções testam, em determinados pontos de código, a validade de determinadas hipóteses. Por exemplo, o desenho de uma rotina pode conter as precondições que devem ser válidas ao se entrar e as pós-condições que devem ser válidas imediatamente após a saída da rotina.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- A programação defensiva inclui também as seguintes medidas:
- •verificar os valores de todos os dados de fontes externas;
- •verificar os valores de todos os parâmetros de entrada, sempre que razoável;
- •tratar de forma consistente os erros em parâmetros;
- •usar mecanismos de tratamento de exceções.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Normalmente, os mecanismos de depuração (asserções, mensagens de depuração etc.) devem ser retirados em versões de operação e podem ser novamente introduzidos em situações de manutenção. Deve-se desativar código que trata de erros menores ou provoca a parada do sistema e deixar ativo código que trata de erros importantes ou permite a degradação gradual do sistema. Nesses casos, as mensagens devem indicar que se trata de um erro interno e recomendar o contato com o suporte técnico.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Existem as seguintes maneiras de administrar a retirada e a inserção desses mecanismos, em ordem decrescente de conveniência:
 - controle por um sistema de gestão de configurações;
 - uso de pré-processadores e compilação condicional;
 - uso de cotos (*stubs*) de depuração.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **Geração de código**
- Algumas ferramentas de modelagem orientada a objetos oferecem o recurso de **geração de código**. Através desse recurso, o modelo detalhado de desenho é traduzido diretamente em um esqueleto do código fonte. Esse recurso acelera a produção de código e ajuda a garantir a consistência entre código e modelo, principalmente no que diz respeito às interfaces entre unidades. A tarefa de implementação com auxílio de geração automática de código é chamada de **engenharia direta** (*forward engineering*).

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **Geração de código**
- A ferramenta gera, de modo controlável pelo desenvolvedor, elementos não explícitos no desenho, como operações para acesso aos atributos (*getters* e *setters*), e trata das pontas de associação, com geração das coleções Java apropriadas. Marcadores especiais inseridos em anotações (no exemplo, *@generated*) indicam que partes do código devem ser geradas novamente, em caso de uma segunda geração.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- A **engenharia reversa** (*reverse engineering*) transforma do código para o modelo. Graças a esse recurso, as alterações de variáveis e métodos, feitas diretamente no código gerado, podem, por sua vez, ser refletidas em atributos e operações do modelo, ajudando a manter a consistência. A engenharia reversa é útil também para incorporar ao modelo elementos que são criados mais facilmente no ambiente de implementação, como as classes representativas das interfaces de usuário, ou simplesmente quando se usa o método YAGNI, em que o código é desenvolvido primeiro e o modelo gerado depois, principalmente para documentar as decisões de desenho. Muitas das classes de desenho interno modeladas neste livro foram obtidas dessa maneira.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Com esse tipo de ferramentas, a implementação é feita de forma conveniente através de muitas iterações entre o ambiente de modelagem e o ambiente de implementação. Sem o uso de ferramentas, é praticamente impossível manter-se a consistência entre modelo e código, dada a quantidade de detalhes que deve ser cuidada em uma implementação robusta e bem-documentada. A combinação das engenharias direta e reversa forma a **engenharia iterativa** (*round-trip engineering*).

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **Revisão do desenho detalhado**
- O desenho detalhado deve ser cuidadosamente verificado, principalmente quanto aos seguintes pontos:
- •adequação da tradução dos mecanismos de desenho para os mecanismos da linguagem de programação, verificando-se, por exemplo, se relacionamentos, heranças e interfaces (no sentido da UML) foram implementados da maneira mais adequada;
- •correção das interfaces das unidades desenhadas, verificando-se se as assinaturas das operações estão corretas e documentadas corretamente;

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **Revisão do desenho detalhado**
- •correção das máquinas de estados, verificando se elas não possuem estados inatingíveis nem armadilhas (ciclos de estados sem saída) e se os estados e transições são completos e ortogonais (dividem todas as situações possíveis em casos sem interseção);
- •correção da lógica do pseudocódigo, verificando-se se as condições são logicamente corretas e otimizadas e se as malhas estão corretas, executando o número correto de vezes e mantendo os invariantes supostos em seu desenho.
- Um padrão de desenho detalhado e codificação deve incluir listas de conferência da implementação, que cobrem vários aspectos do desenho detalhado de classes e métodos.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

TAREFA - QUADRO DE TESTES

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

Contador:	001
Localização:	Tela de abertura NFE.
Criticidade:	Alta
Objeto de Teste:	Verificar funcionamento da tela de abertura emissão nfe.
Caso de Teste:	Testar o funcionamento do botão "Emissão NF-e e campo fiscal"
Pré - Condição:	1.Operação devidamente cadastrada.
Procedimento:	1. Clicar no campo "Emissão de NF-e" 2. Digite o código da operação ou clique F3 para pesquisar, depois de identificar a operação desejada. Pressione ENTER. 2.1 Digite um carácter, por exemplo, letra "A", no campo operação. 3. Digite o código da empresa ou pressione F3 para pesquisar, depois de selecionada a empresa pressione ENTER. 4. Digite o código do emitente ou pressione F3 para pesquisar, depois de identificado o emitente, pressione ENTER. 5. Clicar no botão "Salvar".
Resultado Esperado:	1. O sistema deverá abrir a tela de abertura para NF-e. 2. O sistema deverá passar para campo EMPRESA carregar dados. 2.1 O Sistema deverá mostrar mensagem de erro. 3. O sistema deverá passar para campo EMITENTE carregar dados. 4. O sistema deverá autenticar o emitente selecionado. 5 . O sistema deverá abrir a tela para emissão da nota fiscal eletrônica.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes – Documentação para teste manual

TAREFA - QUADRO DE TESTES

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Qualidade de Testes – Documentação para teste manual

Contador:	001
Localização:	Tela de abertura NFE.
Criticidade:	Alta
Objeto de Teste:	Verificar funcionamento da tela de abertura emissão nfe.
Caso de Teste:	Testar o funcionamento do botão “Emissão NF-e e campo fiscal”
Pré - Condição:	1.Operação devidamente cadastrada.
Procedimento:	1. Clicar no campo “Emissão de NF-e” 2. Digite o código da operação ou clique F3 para pesquisar, depois de identificar a operação desejada. Pressione ENTER. 2.1 Digite um carácter, por exemplo, letra “A”, no campo operação. 3. Digite o código da empresa ou pressione F3 para pesquisar, depois de selecionada a empresa pressione ENTER. 4. Digite o código do emitente ou pressione F3 para pesquisar, depois de identificado o emitente, pressione ENTER. 5. Clicar no botão “Salvar”.
Resultado Esperado:	1. O sistema deverá abrir a tela de abertura para NF-e. 2. O sistema deverá passar para campo EMPRESA carregar dados. 2.1 O Sistema deverá mostrar mensagem de erro. 3. O sistema deverá passar para campo EMITENTE carregar dados. 4. O sistema deverá autenticar o emitente selecionado. 5 . O sistema deverá abrir a tela para emissão da nota fiscal eletrônica.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **2.3 Codificação**
- **2.3.1 Visão geral**
- A codificação de uma unidade visa a transformar os elementos do desenho detalhado nas construções correspondentes da linguagem de programação adotada. A documentação, os comentários de modelagem e scripts de pseudocódigo são transformados em comentários inseridos no código. O código derivado é inserido abaixo de cada comentário derivado do pseudocódigo, ou substitui esse comentário, caso esse código seja considerado autoexplicativo.
- Uma vez escrito, o código deve ser revisto e compilado. A programação em pares é uma forma de fazer essa revisão simultaneamente com a escrita.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **2.3.2 Revisão do código**
- *Verificações*
- O código deve ser cuidadosamente verificado pelos autores, ou, melhor ainda, por um desenvolvedor diferente do autor, mesmo que haja posteriormente uma inspeção formal da implementação. Os pontos que devem ser verificados incluem:
 - adequação da tradução do desenho detalhado para o código, verificando-se, por exemplo, se relacionamentos e heranças foram implementados da maneira mais adequada;
 - correção da digitação de código não gerado automaticamente;
 - correção sintática (existem defeitos de sintaxe que os compiladores não detectam);
 - correção lógica, que inclui a consistência com as decisões de um desenho detalhado;
 - obediência ao padrão de codificação;
 - qualidade e consistência dos comentários.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- O padrão para desenho detalhado e codificação inclui listas de conferência da implementação, que cobrem vários aspectos. Em caso de geração automática, muitos dos aspectos do código estarão corretos por construção, desde que a ferramenta de geração de código tenha sido configurada de forma adequada. Existem dados que mostram que a revisão do código é mais eficaz, em termos de remoção de defeitos, se feita antes da compilação [Humphrey95]. Como, nos ambientes modernos de desenvolvimento, a compilação é feita de forma quase contínua, isso sugere que a revisão do código também seja feita continuamente durante a codificação, possivelmente por meio da programação em pares.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- *Programação em pares*
- A verificação do código pode ser feita em tempo real, usando-se o conceito da **programação em pares**: todo código é escrito conjuntamente por dois programadores na mesma estação de trabalho, chamados por alguns de **piloto** (o que codifica) e **navegador** (o que confere, avalia e tenta sugerir melhorias). Os papéis de piloto e navegador são trocados de vez em quando. Naturalmente, esse conceito pode ser usado também para o desenho detalhado e os testes de unidade.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- A programação em pares costuma ser vista com desconfiança pelos gerentes, por crerem que divide por dois a produtividade. Como observou Martin Fowler, isso só seria verdade se o esforço de digitação fosse a parte mais significativa do esforço de implementação. Nesse caso, naturalmente, a maior parte da codificação seria automatizável. Além disso, a redução de defeitos trazida pela programação em pares poderia até resultar em aumento da produtividade. Resultados relatados na literatura algumas vezes sustentam essa alegação e outras não (por exemplo, [Williams+00] e [Parrish+04]).

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- El Emam [El Emam01] apresenta resultados de experimentos em que foi mostrado que nas inspeções mais eficazes os inspetores pesquisam os defeitos de forma independente, de maneira a reduzir a influência mútua. Isso seria um argumento contra a programação em pares, do ponto de vista de potência de identificação de defeitos (não de outros aspectos benéficos, como a partilha de conhecimento).

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Vários aspectos práticos têm que ser considerados. Um problema é a formação dos pares. Devem ter o mesmo nível de experiência? Se sim, que fazer com os novatos? Se não, um novato será capaz de revisar em tempo real o trabalho de um programador experiente? Com que frequência piloto e navegador devem trocar de papéis? Com que frequência os pares devem ser reagrupados? Que fazer se o número de programadores de uma equipe for ímpar?

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- As questões abrangem aspectos de ambiente físico de trabalho, como uso de mobília apropriada, e até certas questões culturais podem interferir com o trabalho em duplas. Para responder tanto às questões de impacto na produtividade e qualidade quanto a essas questões práticas, cada organização tem que realizar seus próprios experimentos.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **2.3.3 Compilação**
- A compilação é uma tarefa quase completamente automatizada. Entretanto, o desenvolvedor deve verificar inicialmente se o compilador está configurado com as opções corretas e se os arquivos incluídos estão disponíveis nas versões corretas. A integração do ambiente de desenvolvimento com uma ferramenta de gestão de configurações ajuda a eliminar esse tipo de problemas.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Após a compilação, todos os defeitos de compilação encontrados devem ser removidos, inclusive aqueles que são classificados apenas como advertências. Se os codificadores forem experientes e a revisão do código for bem feita, os erros de compilação serão raros. Por isso, a ocorrência de um número significativo de erros de compilação costuma indicar problemas mais sérios de codificação. Nesse caso, o código deve ser novamente revisto, e possivelmente até refeito. Como mesmo os melhores compiladores deixam passar uma quantidade significativa de defeitos de codificação ([Humphrey95], [Humphrey99]), uma compilação com muitos erros provavelmente significa que existem também alguns defeitos não detectados pelo compilador.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Nos ambientes de desenvolvimento mais recentes, como o Eclipse, os compiladores podem ser integrados com muitas outras ferramentas, como ferramentas de modelagem, testes e gestão de configurações. Algumas importantes ferramentas auxiliares dos compiladores são: **formatadores**, que colocam, total ou parcialmente, o código dentro de um padrão configurado de formato; e **analísadores estáticos**, que verificam se o código obedece a um conjunto de regras padronizadas de codificação. Boa parte do trabalho de colocar o código dentro dos padrões pode ser automatizada ou pelo menos conferida com o uso dessas ferramentas.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Recursos adicionais dos compiladores incluem:
- •suporte a operações de refatoramento, como rebatizar pacotes, classes, variáveis e métodos, mover variáveis e métodos entre classes, mover classes entre pacotes, reorganizar hierarquias de especialização, alterar listas de parâmetros e extrair interfaces;
- •suporte à geração de comentários;
- •geração de métodos rotineiros, como construtores e métodos de consulta e atribuição;
- •substituição, endentação e formatação em geral;
- •localização de referências e declarações;
- •execução com várias configurações, inclusive em modo de depuração.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **2.4 Testes de unidade**
- **2.4.1 Visão geral**
- O teste de unidade é um tipo de teste de desenvolvimento (feito pelos desenvolvedores), que exercita detalhadamente uma unidade de código (por exemplo, uma classe ou grupo de classes correlatas). Em processos tradicionais, os testes de unidade são executados após a confecção do código. Mesmo nesse caso, se forem bem planejados e executados, os testes de unidade poderão detectar cerca de 70% dos defeitos que seriam encontrados pelos usuários [McConnell96].

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Entretanto, os testes de unidade adquirem papel central na metodologia de implementação dirigida por testes, popularizada pelo método XP e adotada, entre outros, no processo *Praxis*. Nessa metodologia, os testes são criados primeiro, exercitando o contrato de cada operação implementada pelos métodos. Em seguida, o código dos métodos é escrito, para cumprir os contratos e, portanto, passar nos testes de unidade. Uma unidade só é considerada implementada quando passou em todos os seus testes de unidade. Além disso, no máximo até a integração, os demais testes de desenvolvimento e de sistema devem ser repetidos como testes de regressão, para garantir que não houve introdução de efeitos colaterais adversos.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Como os testes de unidade são escritos antes do código, eles não têm conhecimento do que o código faz internamente ao método, pelo menos conceitualmente. Por isso, são chamados de **testes de caixa-cinza**, pois conhecem as interfaces entre módulos, mas não o interior dos módulos. Idealmente, deveriam ser escritos testes de unidade para todos os métodos de todas as classes, com a possível exceção de métodos triviais, como os de consulta. Na prática, escolhem-se as classes mais importantes, como as que servem de fachada de um pacote, e dentro dessas os métodos com processamento mais significativo, principalmente métodos públicos, que sejam considerados suficientes para exercitar os contratos importantes da classe.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- •As **estruturas de dados** locais devem ser verificadas para se determinar se os dados armazenados temporariamente mantêm sua integridade durante todos os passos da execução do módulo sob teste.
- •As **condições de limite** devem ser testadas para se assegurar de que a unidade sob teste opera adequadamente nos limites estabelecidos, incluindo, entre outros aspectos, número de instâncias em entradas e limites das entradas e saídas válidas.
- •Os **caminhos independentes** devem ser verificados, de tal forma que todas as instruções da unidade sob teste sejam executadas pelo menos uma vez.
- •Os **caminhos de tratamento de erros** devem ser verificados.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Os testes de unidade para a camada de fronteira formam um caso especial. Eles são de automação mais difícil, pelas razões discutidas adiante, e a implementação da camada de fronteira permite a execução de testes manuais. Nesse caso, os testes de unidade podem ser apenas manuais, usando-se tanto os testes informais mais convenientes quanto os previstos nos scripts de testes manuais, produzidos no desenho dos testes.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **2.4.2 Técnicas específicas**
- *Testes básicos*
- Os testes de unidade básicos devem verificar se as pós-condições e as invariantes são cumpridas, dados conjuntos de entrada significativos, que satisfaçam às precondições. Para escolher os conjuntos de entrada significativos, podem ser usadas as mesmas técnicas de partição de equivalência e de análise de valor limite usadas para os testes funcionais (tratados no capítulo sobre **Testes**). Possivelmente, devem ser conferidos não apenas os valores de saída dos métodos como o estado das estruturas de dados locais à classe, após a execução do método. Se esta afetar objetos persistentes, o estado dos dados persistentes também deve ser conferido.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- O tratamento correto das exceções também deve ser verificado, fornecendo-se conjuntos de entradas capazes de causar cada uma das exceções suportadas e conferindo-se os resultados desses tratamentos. Outro grupo importante de testes de unidade é formado por aqueles que testam o **ciclo de vida** dos objetos. Por exemplo, para objetos persistentes, devem-se testar todas as operações de CRUD (criação, recuperação, alteração e exclusão), com parâmetros válidos e inválidos, quando aplicáveis. Para objetos que representam sessões ou transações, devem ser exercitadas versões vazias (abertas e fechadas sem nada fazer), mínimas (apenas com processamento trivial), completas (que percorrem um ciclo completo, em todas as etapas) e com anormalidades em um ou mais pontos do ciclo.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Nos testes de unidades tradicionais, que usavam técnicas de caixa-branca, procurava-se exercitar o máximo de caminhos independentes dentro do código, inclusive os resultantes de bifurcações lógicas e de tratamentos de exceções. Mesmo com a programação por contratos as técnicas de testes de caixa-branca encontradas na literatura podem ser úteis, se as especificações de módulos ou rotinas tiverem formas procedimentais, como máquinas de estado, diagramas de atividades ou algoritmos em pseudocódigo. [Binder00] oferece um tratamento detalhado dessas técnicas, com destaque para o tratamento de máquinas de estado e de hierarquias de especialização. Estas hierarquias complicam significativamente os testes de caminhos, pois é difícil determinar, em uma hierarquia com várias classes de profundidade e vários métodos redefinidos em subclasses, exatamente quais versões dos métodos serão invocadas em tempo de execução.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Geralmente é impossível exercitar esses caminhos de forma exaustiva, o que implica escolher um subconjunto de caminhos considerados importantes e significativos. Nos testes de caixa-cinza, os caminhos seguidos no código, em princípio, não são considerados no desenho do teste, mas a determinação da cobertura dos caminhos pelos testes continua sendo importante. Se o código tem caminhos que não são exercitados pelo teste, isso significa ou que os testes não estão cobrindo todas as alternativas dos contratos ou que o código tem caminhos que não correspondem às necessidades do contrato, o que pode representar tanto erros de lógica quanto código supérfluo.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- *Testes de condições*
- Os testes de condições têm por objetivo detectar erros nas condições lógicas que devem ser avaliadas pela unidade sob teste. Casos de teste são desenhados para que as combinações dos valores das entradas levem as expressões dessas condições a assumirem o máximo possível de combinações de valores. Técnicas de lógica combinatória, como os mapas de Karnaugh e as tabelas de decisão, devem ser usadas para determinar como os possíveis conjuntos de entradas afetam as condições. Caso o número dessas combinações seja muito grande, é preciso verificar as combinações mais relevantes, para manter sob controle o número de casos de teste.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- São bases para o desenho desses testes as condições contidas nas expressões do contrato e na lógica do pseudocódigo, assim como aquelas presentes nas máquinas de estado (por exemplo, nas guardas das transições). Binder [Binder00] mostra em detalhes como os mapas de Karnaugh, as tabelas de decisão e os diagramas de causa e efeito podem ser usados para gerar um conjunto de testes de condição com cobertura adequada.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- *Testes de coleções*
- Os testes de coleções verificam o tratamento de coleções pela unidade sob teste. Esse tipo de teste funciona também como teste de condições de limite da unidade sob teste: um defeito comum nas malhas que percorrem uma coleção é errar, para cima ou para baixo, o número de elementos do conjunto.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Estender essa abordagem a arranjos multidimensionais geralmente não é viável, pois o número de casos de teste possíveis pode ser proibitivo. Para reduzir o número de casos de teste, pode-se iniciar o teste tratando como arranjo unidimensional a dimensão mais interna. Nesse ponto, devem-se escolher entidades de teste com apenas um elemento em cada outra dimensão. Em seguida, partindo da dimensão mais interna para a mais externa, nessa ordem, deve-se testar cada uma delas, de tal forma que:

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Técnicas similares podem ou não ser aplicáveis a outros tipos de coleções, dependendo da estrutura delas. Por exemplo, em uma coleção estruturada como árvores, devem ser criados casos de teste para árvores representativas dos dados esperados, assim como para casos extremos, como árvores vazias, árvores com um único elemento, árvores que só contêm folhas, árvores muito largas e rasas e árvores muito estreitas e profundas.
- Métodos que iteram sobre duas ou mais coleções podem usar as técnicas para uma coleção, se as iterações forem independentes entre si. Por outro lado, se a cardinalidade das coleções processadas posteriormente depender do resultado do processamento das primeiras coleções, pode-se adaptar as técnicas para coleções multidimensionais.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **2.4.3Ambientes de testes de unidade**
- *Componentes para testes de unidade*
- A execução com sucesso de cada caso de teste só pode ser realizada ao se concluir a codificação da parte de unidade sob teste que é acionada por esse caso. Entretanto, dependendo da ordem de integração, pode ser necessário realizar o teste de uma unidade sem que estejam codificadas unidades que a chamam ou que por ela são chamadas. Dessa forma, pode ser necessário desenhar e implementar componentes de teste, como os **controladores**.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Os controladores (*drivers*) realizam chamadas à unidade sob teste, possivelmente enviando parâmetros e exibindo resultados. Os cotos (*stubs*) substituem as unidades subordinadas à unidade sob teste. O controlador recebe parâmetros dos casos de teste e envia resultados para o exibidor. Entre o controlador, a unidade sob teste e os cotos os dados podem fluir em ambas as direções.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Teste eficaz da unidade pode requerer o desenvolvimento de cotos sofisticados. Nesse caso, pode ser preferível ajustar a ordem de integração (por exemplo, no **Plano das liberações**) para usar a integração *bottom-up* (vide adiante) e fazer o teste já com as unidades subordinadas definitivas.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- O desenvolvimento de componentes de teste representa um *overhead* para o projeto, já que eles não fazem parte do produto final. Os componentes de teste são simplificados quando as unidades apresentam menor acoplamento entre si e quando a ordem de integração foi bem escolhida. Entretanto, é comum que o código dos componentes de teste chegue à ordem de grandeza do volume do código útil. Por isso, componentes de teste devem ser mantidos e reutilizados sempre que possível.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- *Plataformas para testes de unidade*
- *A plataforma JUnit*
- A confecção de testes de unidade é muito facilitada pelo uso de plataformas apropriadas, das quais é mais popular a família *xUnit* de ferramentas de código livre, originalmente desenhada por Kent Beck para a linguagem *Smalltalk*. Disponível para muitas linguagens de programação, a versão mais usada com Java é o *JUnit*, embora existam outras ferramentas populares, como a *TestNG*. A plataforma *JUnit* tem muitas extensões para tratamento de relatórios, cotos etc. Ela é usada como base da arquitetura de testes de unidade da plataforma *Praxis*, e, portanto, nos exemplos deste livro.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Essa plataforma sofreu nos anos mais recentes uma evolução significativa, que passou a permitir testes com ordem controlada. Aqui, ilustramos a utilização tradicional, de ordem não controlada, e a utilização com os recursos mais recentes, semelhantes aos usados para testes de sistema, mas empregados aqui para testes de unidade.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- *Arquitetura de testes de unidade*
- *Visão geral*
- Testes de unidade de boa cobertura podem representar grande volume de código, da mesma ordem de grandeza que o código da aplicação propriamente dito. Esse volume significa também oportunidades para introdução de defeitos, que, em um código de testes, são ainda mais críticos, pois podem mascarar defeitos do código da aplicação. Por isso, é conveniente dispor de uma arquitetura de testes de unidade que permita alto grau de reutilização, além de ser consistente com a metodologia escolhida para a integração.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- *Testes da camada de fronteira*
- A rigor, os testes da camada de fronteira não são propriamente testes de unidade, pois exercitam, na realidade, a colaboração que realiza um caso de uso, e não uma classe isolada. Esses testes não são realmente necessários para a implementação dirigida por testes, pois a camada de fronteira pode ser testada manualmente, e as regressões correspondentes podem ser realizadas com os respectivos testes de sistema. Isso não trará problemas, se for respeitado o princípio de que a camada de fronteira apenas cuida de aspectos de apresentação, e toda a lógica é executada da camada de controle para baixo.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Por essas razões, no *Praxis* os testes da camada de fronteira não são normalmente automatizados. Em ambientes distribuídos, como as aplicações Web, os testes de sistema exercitam o lado cliente, agindo, por exemplo, sobre os navegadores (*browsers*). Nesse caso, poderiam ser úteis testes que exercitem apenas o lado servidor, usando, por exemplo, a plataforma *HttpUnit*.⁵ Entretanto, no caso do *Praxis*, mesmo para aplicativos Web essa opção não foi considerada útil o suficiente.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **2.5 Integração + 2.5.1 Visão Geral**
- Na **Integração**, as unidades codificadas e verificadas por testes de unidade são integradas com outras unidades verificadas e implementadas na iteração corrente e com o conjunto implementado nas iterações anteriores, verificando-se o resultado com testes de integração.
- Numa **construção de integração** (*build*), os arquivos de código objeto (*.class*, em Java) das unidades recém-implementadas são ligados com os arquivos de código objeto já existentes para formar um conjunto executável, que, dependendo da linguagem e do ambiente, pode incluir várias formas de arquivos executáveis (por exemplo, *.exe*, *.jar* ou *.war*), arquivos de dados e de configuração e bibliotecas de componentes.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- O conjunto é submetido aos **testes de integração**.
- Os testes de integração devem incluir a execução dos testes manuais da função, produzidos durante o desenho dos testes, principalmente se os testes da camada de fronteira não tiverem sido automatizados.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- A sequência de integração das unidades deve ser planejada, como parte do planejamento das liberações. A sequência escolhida determina também a abordagem usada para os diversos testes de desenvolvimento, definindo a necessidade de componentes de teste, como cotos e controladores. Determina também as possibilidades de paralelismo de implementação, que será tanto mais necessário quanto menores forem os prazos em relação ao esforço previsto para o projeto. A subseção **Abordagens** trata dos tipos mais comuns de sequência.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Ambientes mais modernos de implementação, como o Eclipse, realizam os passos mais simples da integração já durante a compilação. Entretanto, o desenvolvimento de sistemas mais complexos, com paralelismo de implementação, exige processos mais sofisticados de integração, que é realizada juntamente com a realização de testes e a gestão de configurações, e dirigidos por scripts de ferramentas de automação.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **2.5.2 Abordagens**
- A integração *big-bang* é, provavelmente, o método de integração mais frequente na prática. As unidades são testadas individualmente e depois integradas de uma só vez. Esse tipo de integração é o que requer menos esforço, mas geralmente o resultado é desastroso. Diagnosticar um erro nessa situação é bastante complexo, pois é difícil isolar as causas. À medida que os erros são corrigidos, outros erros são introduzidos, por causa da desorganização do processo.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Em abordagens mais organizadas, a integração é feita dos níveis inferiores da arquitetura para os superiores (*bottom-up*) ou vice-versa (*top-down*). Nos diagramas de objetos seguintes, essas abordagens são ilustradas. Instâncias de unidades são denotadas pela letra U, seguida de um dígito que indica o nível (1 é o mais alto) e de um segundo dígito que indica a ordem dentro de cada nível.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

Análise da integração bottom-up

Principais características	As classes de entidade são integradas primeiro, seguidas pelas de controle.
	A ordem de implementação das unidades de nível mais baixo é mais flexível.
	Permite que unidades críticas partilhadas por casos de uso sejam testadas mais cedo.
Vantagens	Não há necessidade de cotos.
	Há maior facilidade para adaptar a implementação aos recursos humanos disponíveis para executá-los.
	Defeitos em unidades críticas partilhadas são detectados mais cedo.
	A produção de código avança mais rapidamente do que por meio da abordagem <i>top-down</i> .
	Em geral, essa abordagem é mais intuitiva.
Desvantagens	Há necessidade de controladores.
	Se a integração for horizontal, muitas unidades devem ser integradas antes de se obter um grau significativo de funcionalidade.
	Se a integração for horizontal, defeitos nas interfaces de usuário e com outros sistemas são detectados mais tarde.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Na integração *bottom-up*, as unidades nos níveis mais subordinados são testadas individualmente, usando-se controladores para simular as funções das unidades de nível superior. À medida que se desenvolve o processo de integração, as unidades de nível superior substituem os controladores.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Na integração *top-down*, os testes iniciais são realizados com a unidade principal, e cada nova unidade introduzida adiciona funcionalidade real ao sistema. As unidades subordinadas precisam ser simuladas por cotos; No início da integração, para que o teste seja eficaz, são necessários cotos muito sofisticados, que devem substituir um conjunto de unidades dos níveis inferiores.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Tanto a integração *top-down* quanto a integração *bottom-up* podem ser feitas de forma vertical ou horizontal. Essas formas são bem caracterizadas em uma arquitetura em que as unidades são divididas por camadas (estrutura horizontal) e por função implementada (estrutura vertical).

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Na **integração horizontal**, cada camada completa é integrada, antes de passar-se à camada imediatamente inferior (*top-down*) ou superior (*bottom-up*).
- Na **integração vertical**, todas as unidades que colaboram para realizar uma função são implementadas, em ordem *top-down* ou *bottom-up*, antes de se passar à função seguinte.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

Análise da integração top-down

Principais características	As classes de fronteira são integradas primeiro, seguindo-se as de controle.
	É mais fácil integrar uma unidade de cada vez.
	Há maior ênfase nas interfaces de usuário e com outros sistemas.
Vantagens	Não há necessidade de controladores.
	Com classes de fronteira e de controle obtém-se mais cedo um protótipo de fachada com alguma funcionalidade, o que facilita as avaliações.
	Defeitos nas interfaces de usuário e com outros sistemas são detectados mais cedo.
	Facilita a verificação em relação ao comportamento especificado pelos casos de uso.
Desvantagens	Há necessidade de cortes.
	Se a integração for horizontal, defeitos em unidades críticas nos níveis mais baixos da arquitetura são encontrados tardiamente.
	É mais difícil alocar os recursos humanos necessários.
	Na prática, é muito difícil utilizar uma abordagem puramente <i>top-down</i> .

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- É possível também realizar uma integração mista. Por exemplo, pode-se, na integração da realização de cada caso de uso, implementar em paralelo as camadas de fronteira e entidade, unindo-as depois por meio da camada de controle. Pode-se também implementar alguns casos de uso de forma *top-down* e outros de forma *bottom-up*. Nesses casos, pode haver algum ganho em paralelismo de implementação, mas a falta de uma ordem padronizada de integração dificulta a gestão.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- O processo *Praxis* adota a integração *bottom-up* vertical por caso de uso, considerando-se as vantagens relativas das várias abordagens, e principalmente o fato de que essa ordem de integração é mais adequada para a implementação dirigida por testes. Portanto, cotos não são utilizados, e os controladores são formados por colaborações das classes de testes de caixa-cinza.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Normalmente, as classes concretas de fronteira e controle são específicas da realização de cada caso de uso, sendo as classes de entidade compartilhadas entre realizações de casos de uso. Superclasses de fronteira e controle poderiam ser também compartilhadas entre casos de uso similares, mas, normalmente, elas deveriam fazer parte da plataforma reutilizável.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Casos de uso podem ser implementados em paralelo; nesse caso, é recomendável que sejam casos de uso que não têm classes implementadas em comum, para diminuir os conflitos de controle de versões. Classes que são apenas reutilizadas não trazem problemas de integração, naturalmente. Por outro lado, casos de uso complexos podem ter fluxos implementados separadamente, talvez até em liberações diferentes; esses casos devem ser previstos no **Plano das liberações**.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **2.5.3 Ambientes de integração**
- Como o processo de integração é incremental, é necessário controlar as várias **linhas de base** de código resultantes de cada passo da integração. Para os objetivos deste capítulo, será considerada como linha de base de implementação um conjunto de unidades formalmente designado e fixado em um instante específico, durante o ciclo de vida de um projeto.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- As linhas de base de implementação devem ser formadas nas construções de integração, e atualizadas ao final de cada nova integração. Para isso, elas devem ser facilmente acessíveis aos implementadores, por meio de um sistema de gestão de configurações, baseado em uma ferramenta que automatize a manutenção das linhas de base.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Linhas de base oficiais são aquelas cujo conteúdo foi aprovado em apreciações formais, como as inspeções de implementação. No Praxis, as linhas de base oficiais são formadas apenas no final das iterações. Entre as linhas de base oficiais, são intercaladas as linhas de base de trabalho, que são usadas e atualizadas pelos implementadores no dia a dia. Das linhas de base de trabalho, os implementadores podem inserir e extrair unidades, de acordo com regras específicas de cada equipe, de formalismo muito menor que o das linhas de base oficiais.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- É possível, inclusive, adotar métodos de propriedade conjunta de unidades. Essa filosofia é preconizada pelo método XP, no qual, em troca de liberdade de alteração das linhas de base sempre que conveniente, os desenvolvedores se comprometem com rigorosos testes de unidade e de integração e com o procedimento de integração contínua, descrito em subseção seguinte.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Além das ferramentas usadas na codificação, como compiladores e analisadores, e do sistema de gestão de configurações, o ambiente de integração inclui também as ferramentas de teste. Durante a integração, é preciso executar regressões tanto com os testes de caixa-cinza quanto com os testes de caixa-preta, sendo necessários os tipos de ferramentas que venham a ser requeridos por ambos.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Além dessas ferramentas, que devem estar disponíveis na estação de trabalho de cada implementador, é preciso dispor, em ambientes de trabalho em equipe, de **servidores de integração**. Esses servidores permitem executar scripts que invocam as demais ferramentas, em máquinas designadas para a integração do código da equipe. Exemplos de servidores de integração são o *CruiseControl* e o *Jenkins*.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **2.5.4 Integração contínua**
- Nos processos mais tradicionais, a integração era feita depois da implementação de todas as unidades isoladas, ou, na melhor das hipóteses, por grandes agrupamentos de unidades. Em contraste, a **integração contínua**, introduzida como uma das práticas do método XP, é efetuada depois que cada unidade é completada, ou mesmo várias vezes durante a programação de cada unidade. Numa sequência típica de integração contínua, um programador ou par de programadores segue estes passos:
- **1.** Completa o desenho detalhado, codificação e testes de unidade de uma unidade de código, ou mesmo de uma parte significativa de uma unidade mais complexa.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **2.**Extraí cópia atualizada da linha de base de trabalho mais recente para sua estação de trabalho.
- **3.**Substitui, nessa cópia da linha de base, as unidades em que estava trabalhando e os respectivos testes de unidade pelo material que acabou de completar.
- **4.**Executa na estação de trabalho uma construção de integração, seguindo um procedimento que incluirá a execução automatizada de compilação, análises estáticas, ligação, testes de unidades novos e regressões dos testes de unidade e testes de sistema restantes.
- **5.**Resolve todos os problemas encontrados nessa construção. Caso uma das unidades alteradas esteja sendo usada por outros programadores, a resolução dos conflitos deve ser negociada.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **6.** Repete os passos 2 a 5 (cópia da linha de base, substituição das unidades alteradas, construção de integração e resolução dos problemas), até que a construção de integração passe sem defeitos. Isso é necessário porque, durante a resolução dos problemas, a linha de base pode ter sido novamente alterada.
- **7.** Executa uma construção de integração na máquina de integração. Essa máquina conterá um ambiente de referência, evitando que problemas sejam mascarados por particularidades do ambiente de cada programador.
- **8.** Resolve todos os problemas encontrados nessa construção, até que a construção na máquina de integração passe sem defeitos.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Nas estações de trabalho dos programadores, é possível fazer as construções de integração no ambiente de desenvolvimento, enquanto na máquina de integração normalmente é usado um servidor de integração. Tipicamente, esse servidor pode ser invocado remotamente, possivelmente de dentro do próprio ambiente de desenvolvimento. Por exemplo, os servidores *CruiseControl* e *Jenkins*, citados anteriormente, dispõem de suplementos (*plug-ins*) que permitem invocá-los de dentro do ambiente Eclipse.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Tipicamente, uma equipe que trabalha com integração contínua faz várias integrações por dia. Se uma construção de integração falha, espera-se que o problema seja resolvido rapidamente, no máximo em algumas horas. Para isso, deve ser possível executar essa construção rapidamente; uma diretriz usual é a de que não deva ultrapassar dez minutos. Isso pode ser difícil de conseguir, pois alguns dos passos da construção, como os testes de regressão de sistema, podem ser demorados, mesmo em máquinas no estado da arte.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Construções de integração mais pesadas podem ser feitas em estágios. O estágio primário, que é usado em toda construção, pode usar uma versão reduzida das regressões, ou uma base de dados menor. Um estágio secundário é executado, por exemplo, durante a noite. Se o estágio secundário for muito grande, ele pode ser dividido em partes, que são executadas em paralelo, em máquinas diferentes. Caso o estágio secundário revele problemas, eles também devem ser resolvidos, e os testes que falharam devem ser incluídos no estágio primário, sempre que possível.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- O ambiente dos servidores de integração deve ser o mais próximo possível do ambiente de produção, ou, se existirem ambientes de produção diferentes, de um ambiente considerado representativo deles. O script de integração deve cobrir os passos da implantação no ambiente de produção, realizando, por exemplo, as cópias de arquivos e bancos de dados necessários. Assim, scripts similares poderão ser usados durante a implantação do produto em seu(s) ambiente(s) definitivo(s).

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **2.6 Inspeções de implementação**
- Encontram-se na literatura muitas posições discordantes a respeito das inspeções de implementação. [Humphrey90], [Humphrey95] e [McConnell96] apresentam dados significativos quanto à superioridade das revisões e inspeções em relação aos testes na remoção de defeitos. O principal argumento é que inspeções localizam diretamente o defeito, enquanto os testes apenas mostram sintomas; a partir desses, a identificação do defeito pode ser bastante demorada.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- No outro extremo de formalidade, seria possível usar os métodos mais tradicionais de implementação, nos quais os testes de unidade são feitos depois da codificação e, possivelmente, até depois da inspeção de implementação. Essa linha é seguida nos processos PSP ([Humphrey95], [Humphrey97]) e TSP [Humphrey99].

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- No PSP, chega-se a recomendar que a revisão do código seja feita antes da compilação, a partir de dados experimentais que apontam para maior eficácia da revisão quando feita sobre o código ainda não compilado. No TSP, os desenvolvedores fazem uma revisão pessoal antes da compilação e uma inspeção em grupo depois da compilação e antes dos testes de unidade. Comparações de eficácia entre os métodos tradicionais e os métodos ágeis têm sido publicadas na literatura.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Outros autores argumentam que as inspeções de código estariam superadas no atual estado da arte.
[Royce98] argumenta que as inspeções de código não são capazes de encontrar defeitos de desenho, que são os mais graves. No *Praxis*, isso é remediado pelo fato de que as inspeções de implementação incluem a revisão do desenho detalhado e de que o desenho de alto nível também é sujeito a inspeções, como parte das atividades de **Desenho**.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Os proponentes da metodologia XP, como [Beck99], substituem as inspeções pela combinação das práticas de programação em pares, testes de unidade, integração contínua e testes de regressão. O uso rigoroso dos testes de unidade faria com que os defeitos fossem geralmente mortos no nascedouro, já que unidades bem-desenhadas são pequenas o suficiente para permitir sua localização rápida. Um estudo bastante abrangente e baseado em dados empíricos é apresentado em [El Emam01]. Os dados apresentados nesse estudo justificam as inspeções em termos de custo e benefício em grande variedade de situações.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Listas de conferência para inspeção de implementação, cobrindo desenho detalhado e código, são apresentadas no padrão para desenho detalhado e codificação. As listas ali apresentadas são orientadas para a linguagem Java, e devem ser adaptadas caso sejam usadas outras linguagens.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- É conveniente remover das listas itens não pertinentes à linguagem inspecionada, corretos por construção (por exemplo, gerados de forma automatizada), ou que correspondam a defeitos raramente introduzidos pelos programadores da organização. Do mesmo modo, defeitos frequentes podem ser mais bem detalhados. As mesmas considerações de personalização valem para as classificações dos defeitos.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Inspeções detalhadas de todo o código podem ser dispendiosas, principalmente quando os inspetores têm pouca experiência e demoram tempos excessivos para realizar a inspeção. Existem muitos dados que comprovam que o tempo gasto nas inspeções é recuperado com a eliminação do trabalho acarretado pela descoberta tardia de defeitos e com a diminuição do esforço de manutenção durante a vida útil do produto.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Entretanto, pode ser difícil, nos primeiros estágios de implantação de um processo, reservar um orçamento de recursos adequado para as inspeções. Mesmo nesse caso, ainda é recomendável que as inspeções sejam realizadas parcialmente, concentrando-se no desenho detalhado para todas as unidades e fazendo as inspeções de código em unidades mais críticas ou selecionadas por amostragem.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Se, mesmo assim, o esforço dessas inspeções for considerado excessivo em um projeto com grande volume de código produzido, uma opção é inspecionar por amostragem. Nesse caso, se uma unidade for sorteada na amostragem e não passar na inspeção, o código produzido por seu programador deve ser automaticamente selecionado para as próximas inspeções. Se isso for devidamente administrado, pode ser um estímulo adicional para que os programadores produzam código capaz de passar nas inspeções. De qualquer maneira, quanto menos as inspeções forem usadas, mais recomendável é a programação em pares.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **2.7 Documentação para usuários**
- **2.7.1 Visão geral**
- Técnicas relativas à documentação de usuário são também tratadas aqui. A documentação de usuário, seja na forma de manuais ou de ajuda on-line, é muitas vezes tão necessária para o sucesso de um produto quanto o código executável. Em alguns tipos de produto, como sítios www, programas de treinamento baseado em computador e outros produtos de multimídia, sequer existe uma fronteira nítida entre programa e documentação.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- A produção de documentação de usuário compreende passos análogos aos da produção de código. Ela tem de ser desenhada, revisada e codificada. No caso de ajuda on-line e de sítios www, é evidente a necessidade de ambientes de desenvolvimento, mas mesmo os modernos processadores de textos são ferramentas sofisticadas de programação.
- A próxima subseção trata do desenho e da confecção dos manuais de usuário, impressos ou on-line. A subseção seguinte trata de questões específicas dos documentos em hipertexto.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **2.7.2 Manuais de usuário**
- As recomendações desta seção se aplicam a documentos destinados a ajudar o usuário a instalar, configurar, operar e gerir produtos de software. Elas não se aplicam a documentos destinados à manutenção de software, à instalação e configuração de hardware, e nem aos seguintes outros tipos de documentação de usuário:
 - material de treinamento;
 - material de marketing;
 - material de ajuda on-line.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- *Desenho dos documentos de usuário*
- Recomenda-se o seguinte procedimento para desenhar os documentos de usuário que deverão ser produzidos:
- •Rever a missão do produto de software, suas interfaces de usuário, suas funções e as tarefas em que será aplicado.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- •Determinar o público dos documentos, estabelecendo, para cada grupo de usuários, o respectivo perfil, tal como descrito nas seções relativas aos usuários do **Modelo do problema** ou do **Modelo da solução**. Os usuários podem ser divididos em públicos diferentes, e estilo e nível de detalhe da documentação devem ser escolhidos em função do respectivo público.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- •Determinar o conjunto de documentos de usuário. A documentação pode ser dividida em múltiplos conjuntos, conforme o público, e cada conjunto pode conter um ou mais volumes, dependendo da quantidade de material. Quando um produto tiver mais de um público, devem-se usar conjuntos diferentes de documentos, ou pelo menos, caso se use um único conjunto, indicar claramente quais as partes de interesse de cada público.
- •Determinar os modos de uso dos documentos. Vide descrição dos modos de uso na próxima subseção.
- •Determinar a estrutura dos documentos, seguindo as recomendações aplicáveis contidas no padrão para documentação para usuários.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- •Determinar os modos de uso dos documentos. Vide descrição dos modos de uso na próxima subseção.
- •Determinar a estrutura dos documentos, seguindo as recomendações aplicáveis contidas no padrão para documentação para usuários.
- *Modos de uso dos documentos de usuário*
- *Modo instrucional*

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Os documentos de modo **instrucional** destinam-se a ajudar o usuário a **aprender** o que for necessário a respeito de um produto. Esse tipo de documento pode ser **orientado para informação**, quando tem por objetivo oferecer ao leitor informação necessária para entendimento do produto e de suas funções, ou **orientado para tarefa**, quando tem por objetivo mostrar ao leitor como realizar uma tarefa ou atingir um objetivo.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- A informação típica de documentos orientados para informação inclui:
 - visão geral;
 - teoria de operação;
 - material tutorial.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- A informação típica de documentos orientados para tarefa inclui:
 - procedimentos de instalação;
 - procedimentos de operação;
 - procedimentos de diagnóstico.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- *Modo referencial*
- Os documentos de modo **referencial** destinam-se a armazenar e organizar informação necessária sobre um produto, facilitando a pesquisa por palavras chaves. São exemplos de documentos referenciais típicos:
 - manual de comandos;
 - manual de mensagens de erro;
 - manual de interface de programação;
 - cartela de referência rápida.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- *Modos mistos*
- Um manual pode combinar os modos instrucional e referencial e as orientações para informação ou para tarefa, desde que cada um deles esteja em uma seção separada do corpo do documento, com um título que identifique claramente o tipo de material.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- *Manuais on-line*
- Os manuais on-line devem, de preferência, ser apresentados em um formato imagem de impressão, como o formato .PDF do Adobe Acrobat. Nesse caso, eles podem ser distribuídos em um título em mídia ótica, ou disponibilizados em um sítio Web de suporte. Deve-se fornecer também o aplicativo para leitura, ou indicar um sítio de onde ele possa ser baixado.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- *Garantia da qualidade*
- Os manuais de usuário devem ser revistos quanto à consistência com os artefatos precursores, como a **Visão de uso** do **Modelo da solução**, e à clareza, completeza e conformidade com as normas aplicáveis. Devem ser mantidos sob gestão de configurações e, após operações de manutenção, sofrer revisão da consistência com o resultado das atividades de manutenção.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- No *Praxis*, a validação de cada função do produto inclui a revisão das respectivas partes do manual do usuário. Aconselha-se também uma revisão no final da fase de Transição, procurando resolver as dúvidas levantadas pelos usuários durante a Operação Piloto.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **2.7.3 Documentação em hipertexto**
- Os títulos em mídia ótica e sítios Web de suporte são muito úteis para os seguintes propósitos:
 - resumir informação importante sobre o produto;
 - fornecer cópias dos manuais on-line;
 - fornecer um ponto de contato para suporte e manutenção;

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Fornecer informação de atualização do produto ou de sua documentação;
- •distribuir correções urgentes para o produto;
- •manter listas de perguntas frequentes;
- •distribuir demonstrações e outros materiais promocionais;
- •fornecer ajuda on-line, quando o ambiente de desenvolvimento não dispuser de ferramentas para gerá-la.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- É importante notar que a estrutura de hipermídia adotada em um título ou sítio é muito diferente da estrutura dos manuais impressos ou imprimíveis. Os seguintes pontos devem ser observados:
- A estrutura de hipermídia consiste em uma rede de nós contendo informações, conectados por hiperligações (*hyperlinks*) que ligam nós por meio de uma palavra-chave. O tamanho dos tópicos não deverá exceder uma página, e essa página deve conter hiperligações agrupadas por temas, para evitar que o usuário fique consultando diversas páginas sobre um mesmo tópico.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- •Outro aspecto importante diz respeito à facilidade de localização da informação desejada. O título ou sítio deve permitir fácil navegação, o que pode ser obtido utilizando-se cadeias reduzidas de hiperligações para alcançar a informação e um número reduzido de hiperligações por página.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Devem-se levar em conta os navegadores (*browsers*) que poderão ser usados, informando o leitor sobre quais navegadores e quais versões permitem melhor leitura. Devem-se evitar usar recursos não padronizados e específicos de um navegador, embora não valha a pena preocupar-se com navegadores mais antigos ou raros.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- •Imagens e recursos de multimídia devem ser usados com cuidado, considerando-se aspectos estéticos, de clareza visual e principalmente da capacidade das linhas utilizadas pelos leitores. Por exemplo, uma intranet normalmente permite velocidades de transmissão muito maiores que as ligações por modem. A distribuição em mídia ótica pode ser útil no caso de material que contenha volumes muito grandes de imagens, som e animação. No caso de uso da Web, deve-se informar ao usuário qual a resolução gráfica mínima recomendada, quais suplementos do navegador são necessários e como obtê-los.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- •Considerações de segurança e confidencialidade são particularmente importantes no caso de acesso via Internet. Os leitores devem ser devidamente informados das restrições aplicáveis. Os mecanismos de distribuição e atualização de senhas devem ser seguros, mas não devem prejudicar a produtividade dos usuários.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- É cada vez mais comum que hipertextos sejam usados também para ajuda on-line, e muitos produtos oferecem essa ajuda tanto através de visualizadores especializados quanto de hipertexto e arquivos PDF. Outros materiais importantes para o suporte a um produto incluem materiais de treinamento e de marketing, que podem usar recursos similares. O assunto de desenho de títulos e sítios é extenso e tratado em publicações especializadas, como Nielsen *et al.* [Nielsen+06]. Recomenda-se consultar a literatura existente e recorrer a profissionais especializados, conforme o porte do sistema desenvolvido.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **2.8 Variações**
- Há controvérsias sobre qual seria a fronteira entre o desenho lógico e o desenho detalhado. Neste livro assume-se que aquele é feito pelo desenhista lógico, como tarefa da disciplina de **Desenho**, e este pelo programador, como tarefa da disciplina de **Implementação**, mas há também controvérsia sobre até que ponto esses papéis podem realmente ser separados. Glass [Glass03] discute os problemas que acontecem tanto quando o desenho lógico é detalhado demais como quando o é de menos. As soluções para isso dependem também da natureza das aplicações e da cultura da organização produtora.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Quanto à documentação para usuários, o grau de formalismo e profundidade dependerá também das características do produto e das exigências do cliente. Uma abordagem mais tradicional seria deixar a produção dessa documentação para depois que o produto completo estivesse pronto, o que tem as desvantagens de não permitir que a avaliação dela faça parte das avaliações intermediárias do produto e de deixar para o final tarefas que podem consumir grande volume de trabalho.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **2.5 Integração + 2.5.1 Visão Geral**
- Na **Integração**, as unidades codificadas e verificadas por testes de unidade são integradas com outras unidades verificadas e implementadas na iteração corrente e com o conjunto implementado nas iterações anteriores, verificando-se o resultado com testes de integração.
- Numa **construção de integração** (*build*), os arquivos de código objeto (*.class*, em Java) das unidades recém-implementadas são ligados com os arquivos de código objeto já existentes para formar um conjunto executável, que, dependendo da linguagem e do ambiente, pode incluir várias formas de arquivos executáveis (por exemplo, *.exe*, *.jar* ou *.war*), arquivos de dados e de configuração e bibliotecas de componentes.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- O conjunto é submetido aos **testes de integração**.
- Os testes de integração devem incluir a execução dos testes manuais da função, produzidos durante o desenho dos testes, principalmente se os testes da camada de fronteira não tiverem sido automatizados.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- A sequência de integração das unidades deve ser planejada, como parte do planejamento das liberações. A sequência escolhida determina também a abordagem usada para os diversos testes de desenvolvimento, definindo a necessidade de componentes de teste, como cotos e controladores. Determina também as possibilidades de paralelismo de implementação, que será tanto mais necessário quanto menores forem os prazos em relação ao esforço previsto para o projeto. A subseção **Abordagens** trata dos tipos mais comuns de sequência.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Ambientes mais modernos de implementação, como o Eclipse, realizam os passos mais simples da integração já durante a compilação. Entretanto, o desenvolvimento de sistemas mais complexos, com paralelismo de implementação, exige processos mais sofisticados de integração, que é realizada juntamente com a realização de testes e a gestão de configurações, e dirigidos por scripts de ferramentas de automação.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **2.5.2 Abordagens**
- A integração *big-bang* é, provavelmente, o método de integração mais frequente na prática. As unidades são testadas individualmente e depois integradas de uma só vez. Esse tipo de integração é o que requer menos esforço, mas geralmente o resultado é desastroso. Diagnosticar um erro nessa situação é bastante complexo, pois é difícil isolar as causas. À medida que os erros são corrigidos, outros erros são introduzidos, por causa da desorganização do processo.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Em abordagens mais organizadas, a integração é feita dos níveis inferiores da arquitetura para os superiores (*bottom-up*) ou vice-versa (*top-down*). Nos diagramas de objetos seguintes, essas abordagens são ilustradas. Instâncias de unidades são denotadas pela letra U, seguida de um dígito que indica o nível (1 é o mais alto) e de um segundo dígito que indica a ordem dentro de cada nível.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

Análise da integração bottom-up

Principais características	As classes de entidade são integradas primeiro, seguidas pelas de controle.
	A ordem de implementação das unidades de nível mais baixo é mais flexível.
	Permite que unidades críticas partilhadas por casos de uso sejam testadas mais cedo.
Vantagens	Não há necessidade de cotos.
	Há maior facilidade para adaptar a implementação aos recursos humanos disponíveis para executá-los.
	Defeitos em unidades críticas partilhadas são detectados mais cedo.
	A produção de código avança mais rapidamente do que por meio da abordagem <i>top-down</i> .
	Em geral, essa abordagem é mais intuitiva.
Desvantagens	Há necessidade de controladores.
	Se a integração for horizontal, muitas unidades devem ser integradas antes de se obter um grau significativo de funcionalidade.
	Se a integração for horizontal, defeitos nas interfaces de usuário e com outros sistemas são detectados mais tarde.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Na integração *bottom-up*, as unidades nos níveis mais subordinados são testadas individualmente, usando-se controladores para simular as funções das unidades de nível superior. À medida que se desenvolve o processo de integração, as unidades de nível superior substituem os controladores.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Na integração *top-down*, os testes iniciais são realizados com a unidade principal, e cada nova unidade introduzida adiciona funcionalidade real ao sistema. As unidades subordinadas precisam ser simuladas por cotos; No início da integração, para que o teste seja eficaz, são necessários cotos muito sofisticados, que devem substituir um conjunto de unidades dos níveis inferiores.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Tanto a integração *top-down* quanto a integração *bottom-up* podem ser feitas de forma vertical ou horizontal. Essas formas são bem caracterizadas em uma arquitetura em que as unidades são divididas por camadas (estrutura horizontal) e por função implementada (estrutura vertical).

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Na **integração horizontal**, cada camada completa é integrada, antes de passar-se à camada imediatamente inferior (*top-down*) ou superior (*bottom-up*).
- Na **integração vertical**, todas as unidades que colaboram para realizar uma função são implementadas, em ordem *top-down* ou *bottom-up*, antes de se passar à função seguinte.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

Análise da integração top-down

Principais características	As classes de fronteira são integradas primeiro, seguindo-se as de controle.
	É mais fácil integrar uma unidade de cada vez.
	Há maior ênfase nas interfaces de usuário e com outros sistemas.
Vantagens	Não há necessidade de controladores.
	Com classes de fronteira e de controle obtém-se mais cedo um protótipo de fachada com alguma funcionalidade, o que facilita as avaliações.
	Defeitos nas interfaces de usuário e com outros sistemas são detectados mais cedo.
	Facilita a verificação em relação ao comportamento especificado pelos casos de uso.
Desvantagens	Há necessidade de cotos.
	Se a integração for horizontal, defeitos em unidades críticas nos níveis mais baixos da arquitetura são encontrados tardiamente.
	É mais difícil alocar os recursos humanos necessários.
	Na prática, é muito difícil utilizar uma abordagem puramente <i>top-down</i> .

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- É possível também realizar uma integração mista. Por exemplo, pode-se, na integração da realização de cada caso de uso, implementar em paralelo as camadas de fronteira e entidade, unindo-as depois por meio da camada de controle. Pode-se também implementar alguns casos de uso de forma *top-down* e outros de forma *bottom-up*. Nesses casos, pode haver algum ganho em paralelismo de implementação, mas a falta de uma ordem padronizada de integração dificulta a gestão.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- O processo *Praxis* adota a integração *bottom-up* vertical por caso de uso, considerando-se as vantagens relativas das várias abordagens, e principalmente o fato de que essa ordem de integração é mais adequada para a implementação dirigida por testes. Portanto, cotos não são utilizados, e os controladores são formados por colaborações das classes de testes de caixa-cinza.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Normalmente, as classes concretas de fronteira e controle são específicas da realização de cada caso de uso, sendo as classes de entidade compartilhadas entre realizações de casos de uso. Superclasses de fronteira e controle poderiam ser também compartilhadas entre casos de uso similares, mas, normalmente, elas deveriam fazer parte da plataforma reutilizável.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Casos de uso podem ser implementados em paralelo; nesse caso, é recomendável que sejam casos de uso que não têm classes implementadas em comum, para diminuir os conflitos de controle de versões. Classes que são apenas reutilizadas não trazem problemas de integração, naturalmente. Por outro lado, casos de uso complexos podem ter fluxos implementados separadamente, talvez até em liberações diferentes; esses casos devem ser previstos no **Plano das liberações**.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **2.5.3 Ambientes de integração**
- Como o processo de integração é incremental, é necessário controlar as várias **linhas de base** de código resultantes de cada passo da integração. Para os objetivos deste capítulo, será considerada como linha de base de implementação um conjunto de unidades formalmente designado e fixado em um instante específico, durante o ciclo de vida de um projeto.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- As linhas de base de implementação devem ser formadas nas construções de integração, e atualizadas ao final de cada nova integração. Para isso, elas devem ser facilmente acessíveis aos implementadores, por meio de um sistema de gestão de configurações, baseado em uma ferramenta que automatize a manutenção das linhas de base.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Linhas de base oficiais são aquelas cujo conteúdo foi aprovado em apreciações formais, como as inspeções de implementação. No Praxis, as linhas de base oficiais são formadas apenas no final das iterações. Entre as linhas de base oficiais, são intercaladas as linhas de base de trabalho, que são usadas e atualizadas pelos implementadores no dia a dia. Das linhas de base de trabalho, os implementadores podem inserir e extrair unidades, de acordo com regras específicas de cada equipe, de formalismo muito menor que o das linhas de base oficiais.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- É possível, inclusive, adotar métodos de propriedade conjunta de unidades. Essa filosofia é preconizada pelo método XP, no qual, em troca de liberdade de alteração das linhas de base sempre que conveniente, os desenvolvedores se comprometem com rigorosos testes de unidade e de integração e com o procedimento de integração contínua, descrito em subseção seguinte.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Além das ferramentas usadas na codificação, como compiladores e analisadores, e do sistema de gestão de configurações, o ambiente de integração inclui também as ferramentas de teste. Durante a integração, é preciso executar regressões tanto com os testes de caixa-cinza quanto com os testes de caixa-preta, sendo necessários os tipos de ferramentas que venham a ser requeridos por ambos.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Além dessas ferramentas, que devem estar disponíveis na estação de trabalho de cada implementador, é preciso dispor, em ambientes de trabalho em equipe, de **servidores de integração**. Esses servidores permitem executar scripts que invocam as demais ferramentas, em máquinas designadas para a integração do código da equipe. Exemplos de servidores de integração são o *CruiseControl* e o *Jenkins*.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **2.5.4 Integração contínua**
- Nos processos mais tradicionais, a integração era feita depois da implementação de todas as unidades isoladas, ou, na melhor das hipóteses, por grandes agrupamentos de unidades. Em contraste, a **integração contínua**, introduzida como uma das práticas do método XP, é efetuada depois que cada unidade é completada, ou mesmo várias vezes durante a programação de cada unidade. Numa sequência típica de integração contínua, um programador ou par de programadores segue estes passos:
- **1.** Completa o desenho detalhado, codificação e testes de unidade de uma unidade de código, ou mesmo de uma parte significativa de uma unidade mais complexa.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **2.**Extraí cópia atualizada da linha de base de trabalho mais recente para sua estação de trabalho.
- **3.**Substitui, nessa cópia da linha de base, as unidades em que estava trabalhando e os respectivos testes de unidade pelo material que acabou de completar.
- **4.**Executa na estação de trabalho uma construção de integração, seguindo um procedimento que incluirá a execução automatizada de compilação, análises estáticas, ligação, testes de unidades novos e regressões dos testes de unidade e testes de sistema restantes.
- **5.**Resolve todos os problemas encontrados nessa construção. Caso uma das unidades alteradas esteja sendo usada por outros programadores, a resolução dos conflitos deve ser negociada.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **6.** Repete os passos 2 a 5 (cópia da linha de base, substituição das unidades alteradas, construção de integração e resolução dos problemas), até que a construção de integração passe sem defeitos. Isso é necessário porque, durante a resolução dos problemas, a linha de base pode ter sido novamente alterada.
- **7.** Executa uma construção de integração na máquina de integração. Essa máquina conterá um ambiente de referência, evitando que problemas sejam mascarados por particularidades do ambiente de cada programador.
- **8.** Resolve todos os problemas encontrados nessa construção, até que a construção na máquina de integração passe sem defeitos.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Nas estações de trabalho dos programadores, é possível fazer as construções de integração no ambiente de desenvolvimento, enquanto na máquina de integração normalmente é usado um servidor de integração. Tipicamente, esse servidor pode ser invocado remotamente, possivelmente de dentro do próprio ambiente de desenvolvimento. Por exemplo, os servidores *CruiseControl* e *Jenkins*, citados anteriormente, dispõem de suplementos (*plug-ins*) que permitem invocá-los de dentro do ambiente Eclipse.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Tipicamente, uma equipe que trabalha com integração contínua faz várias integrações por dia. Se uma construção de integração falha, espera-se que o problema seja resolvido rapidamente, no máximo em algumas horas. Para isso, deve ser possível executar essa construção rapidamente; uma diretriz usual é a de que não deva ultrapassar dez minutos. Isso pode ser difícil de conseguir, pois alguns dos passos da construção, como os testes de regressão de sistema, podem ser demorados, mesmo em máquinas no estado da arte.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Construções de integração mais pesadas podem ser feitas em estágios. O estágio primário, que é usado em toda construção, pode usar uma versão reduzida das regressões, ou uma base de dados menor. Um estágio secundário é executado, por exemplo, durante a noite. Se o estágio secundário for muito grande, ele pode ser dividido em partes, que são executadas em paralelo, em máquinas diferentes. Caso o estágio secundário revele problemas, eles também devem ser resolvidos, e os testes que falharam devem ser incluídos no estágio primário, sempre que possível.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- O ambiente dos servidores de integração deve ser o mais próximo possível do ambiente de produção, ou, se existirem ambientes de produção diferentes, de um ambiente considerado representativo deles. O script de integração deve cobrir os passos da implantação no ambiente de produção, realizando, por exemplo, as cópias de arquivos e bancos de dados necessários. Assim, scripts similares poderão ser usados durante a implantação do produto em seu(s) ambiente(s) definitivo(s).

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **2.6 Inspeções de implementação**
- Encontram-se na literatura muitas posições discordantes a respeito das inspeções de implementação. [Humphrey90], [Humphrey95] e [McConnell96] apresentam dados significativos quanto à superioridade das revisões e inspeções em relação aos testes na remoção de defeitos. O principal argumento é que inspeções localizam diretamente o defeito, enquanto os testes apenas mostram sintomas; a partir desses, a identificação do defeito pode ser bastante demorada.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- No outro extremo de formalidade, seria possível usar os métodos mais tradicionais de implementação, nos quais os testes de unidade são feitos depois da codificação e, possivelmente, até depois da inspeção de implementação. Essa linha é seguida nos processos PSP ([Humphrey95], [Humphrey97]) e TSP [Humphrey99].

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- No PSP, chega-se a recomendar que a revisão do código seja feita antes da compilação, a partir de dados experimentais que apontam para maior eficácia da revisão quando feita sobre o código ainda não compilado. No TSP, os desenvolvedores fazem uma revisão pessoal antes da compilação e uma inspeção em grupo depois da compilação e antes dos testes de unidade. Comparações de eficácia entre os métodos tradicionais e os métodos ágeis têm sido publicadas na literatura.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Outros autores argumentam que as inspeções de código estariam superadas no atual estado da arte.
[Royce98] argumenta que as inspeções de código não são capazes de encontrar defeitos de desenho, que são os mais graves. No *Praxis*, isso é remediado pelo fato de que as inspeções de implementação incluem a revisão do desenho detalhado e de que o desenho de alto nível também é sujeito a inspeções, como parte das atividades de **Desenho**.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Os proponentes da metodologia XP, como [Beck99], substituem as inspeções pela combinação das práticas de programação em pares, testes de unidade, integração contínua e testes de regressão. O uso rigoroso dos testes de unidade faria com que os defeitos fossem geralmente mortos no nascedouro, já que unidades bem-desenhadas são pequenas o suficiente para permitir sua localização rápida. Um estudo bastante abrangente e baseado em dados empíricos é apresentado em [El Emam01]. Os dados apresentados nesse estudo justificam as inspeções em termos de custo e benefício em grande variedade de situações.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Listas de conferência para inspeção de implementação, cobrindo desenho detalhado e código, são apresentadas no padrão para desenho detalhado e codificação. As listas ali apresentadas são orientadas para a linguagem Java, e devem ser adaptadas caso sejam usadas outras linguagens.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- É conveniente remover das listas itens não pertinentes à linguagem inspecionada, corretos por construção (por exemplo, gerados de forma automatizada), ou que correspondam a defeitos raramente introduzidos pelos programadores da organização. Do mesmo modo, defeitos frequentes podem ser mais bem detalhados. As mesmas considerações de personalização valem para as classificações dos defeitos.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Inspeções detalhadas de todo o código podem ser dispendiosas, principalmente quando os inspetores têm pouca experiência e demoram tempos excessivos para realizar a inspeção. Existem muitos dados que comprovam que o tempo gasto nas inspeções é recuperado com a eliminação do trabalho acarretado pela descoberta tardia de defeitos e com a diminuição do esforço de manutenção durante a vida útil do produto.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Entretanto, pode ser difícil, nos primeiros estágios de implantação de um processo, reservar um orçamento de recursos adequado para as inspeções. Mesmo nesse caso, ainda é recomendável que as inspeções sejam realizadas parcialmente, concentrando-se no desenho detalhado para todas as unidades e fazendo as inspeções de código em unidades mais críticas ou selecionadas por amostragem.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Se, mesmo assim, o esforço dessas inspeções for considerado excessivo em um projeto com grande volume de código produzido, uma opção é inspecionar por amostragem. Nesse caso, se uma unidade for sorteada na amostragem e não passar na inspeção, o código produzido por seu programador deve ser automaticamente selecionado para as próximas inspeções. Se isso for devidamente administrado, pode ser um estímulo adicional para que os programadores produzam código capaz de passar nas inspeções. De qualquer maneira, quanto menos as inspeções forem usadas, mais recomendável é a programação em pares.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **2.7 Documentação para usuários**
- **2.7.1 Visão geral**
- Técnicas relativas à documentação de usuário são também tratadas aqui. A documentação de usuário, seja na forma de manuais ou de ajuda on-line, é muitas vezes tão necessária para o sucesso de um produto quanto o código executável. Em alguns tipos de produto, como sítios www, programas de treinamento baseado em computador e outros produtos de multimídia, sequer existe uma fronteira nítida entre programa e documentação.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- A produção de documentação de usuário compreende passos análogos aos da produção de código. Ela tem de ser desenhada, revisada e codificada. No caso de ajuda on-line e de sítios www, é evidente a necessidade de ambientes de desenvolvimento, mas mesmo os modernos processadores de textos são ferramentas sofisticadas de programação.
- A próxima subseção trata do desenho e da confecção dos manuais de usuário, impressos ou on-line. A subseção seguinte trata de questões específicas dos documentos em hipertexto.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **2.7.2 Manuais de usuário**
- As recomendações desta seção se aplicam a documentos destinados a ajudar o usuário a instalar, configurar, operar e gerir produtos de software. Elas não se aplicam a documentos destinados à manutenção de software, à instalação e configuração de hardware, e nem aos seguintes outros tipos de documentação de usuário:
 - material de treinamento;
 - material de marketing;
 - material de ajuda on-line.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- *Desenho dos documentos de usuário*
- Recomenda-se o seguinte procedimento para desenhar os documentos de usuário que deverão ser produzidos:
- •Rever a missão do produto de software, suas interfaces de usuário, suas funções e as tarefas em que será aplicado.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- •Determinar o público dos documentos, estabelecendo, para cada grupo de usuários, o respectivo perfil, tal como descrito nas seções relativas aos usuários do **Modelo do problema** ou do **Modelo da solução**. Os usuários podem ser divididos em públicos diferentes, e estilo e nível de detalhe da documentação devem ser escolhidos em função do respectivo público.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- •Determinar o conjunto de documentos de usuário. A documentação pode ser dividida em múltiplos conjuntos, conforme o público, e cada conjunto pode conter um ou mais volumes, dependendo da quantidade de material. Quando um produto tiver mais de um público, devem-se usar conjuntos diferentes de documentos, ou pelo menos, caso se use um único conjunto, indicar claramente quais as partes de interesse de cada público.
- •Determinar os modos de uso dos documentos. Vide descrição dos modos de uso na próxima subseção.
- •Determinar a estrutura dos documentos, seguindo as recomendações aplicáveis contidas no padrão para documentação para usuários.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- •Determinar os modos de uso dos documentos. Vide descrição dos modos de uso na próxima subseção.
- •Determinar a estrutura dos documentos, seguindo as recomendações aplicáveis contidas no padrão para documentação para usuários.
- *Modos de uso dos documentos de usuário*
- *Modo instrucional*

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Os documentos de modo **instrucional** destinam-se a ajudar o usuário a **aprender** o que for necessário a respeito de um produto. Esse tipo de documento pode ser **orientado para informação**, quando tem por objetivo oferecer ao leitor informação necessária para entendimento do produto e de suas funções, ou **orientado para tarefa**, quando tem por objetivo mostrar ao leitor como realizar uma tarefa ou atingir um objetivo.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- A informação típica de documentos orientados para informação inclui:
 - visão geral;
 - teoria de operação;
 - material tutorial.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- A informação típica de documentos orientados para tarefa inclui:
 - procedimentos de instalação;
 - procedimentos de operação;
 - procedimentos de diagnóstico.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- *Modo referencial*
- Os documentos de modo **referencial** destinam-se a armazenar e organizar informação necessária sobre um produto, facilitando a pesquisa por palavras chaves. São exemplos de documentos referenciais típicos:
 - manual de comandos;
 - manual de mensagens de erro;
 - manual de interface de programação;
 - cartela de referência rápida.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- *Modos mistos*
- Um manual pode combinar os modos instrucional e referencial e as orientações para informação ou para tarefa, desde que cada um deles esteja em uma seção separada do corpo do documento, com um título que identifique claramente o tipo de material.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- *Manuais on-line*
- Os manuais on-line devem, de preferência, ser apresentados em um formato imagem de impressão, como o formato .PDF do Adobe Acrobat. Nesse caso, eles podem ser distribuídos em um título em mídia ótica, ou disponibilizados em um sítio Web de suporte. Deve-se fornecer também o aplicativo para leitura, ou indicar um sítio de onde ele possa ser baixado.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- *Garantia da qualidade*
- Os manuais de usuário devem ser revistos quanto à consistência com os artefatos precursores, como a **Visão de uso** do **Modelo da solução**, e à clareza, completeza e conformidade com as normas aplicáveis. Devem ser mantidos sob gestão de configurações e, após operações de manutenção, sofrer revisão da consistência com o resultado das atividades de manutenção.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- No *Praxis*, a validação de cada função do produto inclui a revisão das respectivas partes do manual do usuário. Aconselha-se também uma revisão no final da fase de Transição, procurando resolver as dúvidas levantadas pelos usuários durante a Operação Piloto.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **2.7.3 Documentação em hipertexto**
- Os títulos em mídia ótica e sítios Web de suporte são muito úteis para os seguintes propósitos:
 - resumir informação importante sobre o produto;
 - fornecer cópias dos manuais on-line;
 - fornecer um ponto de contato para suporte e manutenção;

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Fornecer informação de atualização do produto ou de sua documentação;
- •distribuir correções urgentes para o produto;
- •manter listas de perguntas frequentes;
- •distribuir demonstrações e outros materiais promocionais;
- •fornecer ajuda on-line, quando o ambiente de desenvolvimento não dispuser de ferramentas para gerá-la.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- É importante notar que a estrutura de hipermídia adotada em um título ou sítio é muito diferente da estrutura dos manuais impressos ou imprimíveis. Os seguintes pontos devem ser observados:
- A estrutura de hipermídia consiste em uma rede de nós contendo informações, conectados por hiperligações (*hyperlinks*) que ligam nós por meio de uma palavra-chave. O tamanho dos tópicos não deverá exceder uma página, e essa página deve conter hiperligações agrupadas por temas, para evitar que o usuário fique consultando diversas páginas sobre um mesmo tópico.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- •Outro aspecto importante diz respeito à facilidade de localização da informação desejada. O título ou sítio deve permitir fácil navegação, o que pode ser obtido utilizando-se cadeias reduzidas de hiperligações para alcançar a informação e um número reduzido de hiperligações por página.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Devem-se levar em conta os navegadores (*browsers*) que poderão ser usados, informando o leitor sobre quais navegadores e quais versões permitem melhor leitura. Devem-se evitar usar recursos não padronizados e específicos de um navegador, embora não valha a pena preocupar-se com navegadores mais antigos ou raros.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- •Imagens e recursos de multimídia devem ser usados com cuidado, considerando-se aspectos estéticos, de clareza visual e principalmente da capacidade das linhas utilizadas pelos leitores. Por exemplo, uma intranet normalmente permite velocidades de transmissão muito maiores que as ligações por modem. A distribuição em mídia ótica pode ser útil no caso de material que contenha volumes muito grandes de imagens, som e animação. No caso de uso da Web, deve-se informar ao usuário qual a resolução gráfica mínima recomendada, quais suplementos do navegador são necessários e como obtê-los.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- •Considerações de segurança e confidencialidade são particularmente importantes no caso de acesso via Internet. Os leitores devem ser devidamente informados das restrições aplicáveis. Os mecanismos de distribuição e atualização de senhas devem ser seguros, mas não devem prejudicar a produtividade dos usuários.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- É cada vez mais comum que hipertextos sejam usados também para ajuda on-line, e muitos produtos oferecem essa ajuda tanto através de visualizadores especializados quanto de hipertexto e arquivos PDF. Outros materiais importantes para o suporte a um produto incluem materiais de treinamento e de marketing, que podem usar recursos similares. O assunto de desenho de títulos e sítios é extenso e tratado em publicações especializadas, como Nielsen *et al.* [Nielsen+06]. Recomenda-se consultar a literatura existente e recorrer a profissionais especializados, conforme o porte do sistema desenvolvido.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- **2.8 Variações**
- Há controvérsias sobre qual seria a fronteira entre o desenho lógico e o desenho detalhado. Neste livro assume-se que aquele é feito pelo desenhista lógico, como tarefa da disciplina de **Desenho**, e este pelo programador, como tarefa da disciplina de **Implementação**, mas há também controvérsia sobre até que ponto esses papéis podem realmente ser separados. Glass [Glass03] discute os problemas que acontecem tanto quando o desenho lógico é detalhado demais como quando o é de menos. As soluções para isso dependem também da natureza das aplicações e da cultura da organização produtora.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

#Auditoria de Configuração (implementação) - (Fonte: PAULA FILHO, Wilson de Pádua – Cap.8)

- Quanto à documentação para usuários, o grau de formalismo e profundidade dependerá também das características do produto e das exigências do cliente. Uma abordagem mais tradicional seria deixar a produção dessa documentação para depois que o produto completo estivesse pronto, o que tem as desvantagens de não permitir que a avaliação dela faça parte das avaliações intermediárias do produto e de deixar para o final tarefas que podem consumir grande volume de trabalho.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

... composição de nota bimestral

#Cronograma

- *Data de início –*
- *Data de Término –*
 - *Qualidade de Testes – Elaborar 05 (cinco) quadros de Teste Manual - com no mínimo 05 (cinco) procedimentos e 05 (cinco) resultados esperados para cada teste = (pontuação máxima até 2.5 pontos);*
 - *Plano de Mitigação de Riscos (redução da probabilidade + redução do efeito) - Elaborar 05 (cinco) quadros completos (riscos, probabilidades e efeitos) = (pontuação máxima até 2.5 pontos);*
 - *Entrega dos 10 (dez) relatórios ... Inserir na plataforma AVA;*
 - *Durante este período os alunos deverão apresentar os relatórios através de walkthrough.*
- *Data – avaliação (5.0)*

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

... composição de nota bimestral

#a Case de Análise deverá ser o T.C.C.

#tarefa individual (mesmo que o TCC seja em dupla)

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

... composição de nota bimestral

#Modelo = Qualidade de Testes – Teste Manual (criar 05 tabelas como estas)

Contador:	001
Localização:	Tela de abertura NFE.
Criticidade:	Alta
Objeto de Teste:	Verificar funcionamento da tela de abertura emissão nfe.
Caso de Teste:	Testar o funcionamento do botão "Emissão NF-e e campo fiscal"
Pré - Condição:	1.Operação devidamente cadastrada.
Procedimento:	1. Clicar no campo "Emissão de NF-e" 2. Digite o código da operação ou clique F3 para pesquisar, depois de identificar a operação desejada. Pressione ENTER. 2.1 Digite um carácter, por exemplo, letra "A", no campo operação. 3. Digite o código da empresa ou pressione F3 para pesquisar, depois de selecionada a empresa pressione ENTER. 4. Digite o código do emitente ou pressione F3 para pesquisar, depois de identificado o emitente, pressione ENTER. 5. Clicar no botão "Salvar".
Resultado Esperado:	1. O sistema deverá abrir a tela de abertura para NF-e. 2. O sistema deverá passar para campo EMPRESA carregar dados. 2.1 O Sistema deverá mostrar mensagem de erro. 3. O sistema deverá passar para campo EMITENTE carregar dados. 4. O sistema deverá autenticar o emitente selecionado. 5 . O sistema deverá abrir a tela para emissão da nota fiscal eletrônica.

GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

... composição de nota bimestral

#Modelo = Planos de Mitigação de Riscos (criar 05 conjuntos de tabelas como estas)

Id	Causa	Risco	Efeito	Prob.	Imp.	Exp.

O plano de redução de probabilidade de risco consiste nas ações identificadas como necessárias para diminuir a probabilidade de que um risco ocorra. Esse tipo de plano deve agir nas *causas* do risco.

Id	Causa do risco	Risco	Plano de redução de probabilidade

O plano de redução de impacto de risco é definido e aplicado de forma semelhante ao plano de redução de probabilidade, exceto pelo fato de que, nesse caso, as ações devem procurar diminuir o impacto do risco.

Id	Risco	Efeito	Plano de redução de impacto